

Proyecto Final



Modelos Estocásticos

Integrantes

Sergio Alejandro Reita Serrano
Erwin Duban Soto Sarmiento
Daniel Alejandro Acosta Avila
Julian Esteban Mendoza Wilches
Nicolás Cortés Gutiérrez

Profesor

Jorge Eduardo Ortiz Triviño

Universidad Nacional de Colombia
Facultad de Ingeniería, Departamento de Ingeniería de
Sistemas e Industrial
Bogotá, Colombia
2026 - I

Índice

1. Introducción.....	5
Descripción general.....	5
Etapas del proyecto.....	6
2. Marco Teórico.....	7
Problemas de decisión secuencial.....	7
Proceso de Decisión de Markov (MDP).....	8
Definición Formal.....	8
Mapeo al Problema de Caché.....	9
Utilidad y Recompensas Descontadas.....	9
La Función de Utilidad sobre Historias.....	9
El Factor de Descuento γ	10
Política Óptima.....	11
Definición de Política.....	11
Utilidad de un Estado.....	11
La Ecuación de Bellman.....	12
Formulación.....	12
La Función Q y Extracción de Política.....	12
Interpretación para el Sistema de Caché.....	13
Algoritmo de Value Iteration.....	13
Idea Central.....	13
La Actualización de Bellman (Bellman Update).....	13
Convergencia.....	13
Complejidad.....	14
Algoritmo de Policy Iteration.....	14
Motivación.....	14
Pasos del Algoritmo.....	15
Convergencia.....	15
Comparación entre los Algoritmos.....	15
3. Descripción y justificación del problema a resolver.....	16
Contexto General.....	16
El Problema de Reemplazo de Páginas.....	16
Pertinencia del Capítulo 16.....	17
Relevancia en el Dominio de la Computación.....	18
Cumplimiento del Requisito del 50%.....	18
4. Diseño de la aplicación.....	19
4.1 Arquitectura y Modelado del Sistema.....	19
4.2 Especificaciones Técnicas.....	20

4.3	Formulación del MDP de Caché.....	20
4.4	Distribuciones de Acceso Implementadas.....	21
4.5	Escenarios de Experimentación.....	21
4.6	Arquitectura Modular del Software.....	22
5.	Diagramas UML.....	23
	Diagrama UML.....	23
	Diagrama de componentes.....	24
	Diagrama de secuencia de la simulación.....	25
	Diagrama de Flujo: Value Iteration.....	26
	Diagrama de Flujo: Policy iteration.....	28
6.	Código fuente completo de la herramienta construida.....	30
7.	Manual técnico.....	82
7.2	Formulación matemática del MDP.....	82
7.2.1	Definición del espacio de estados.....	82
7.2.2	Acciones.....	83
7.2.3	Modelo de transición.....	83
7.2.4	Función de recompensa.....	83
7.2.5	Ecuación de Bellman.....	84
7.2.6	Función Q (valor-acción).....	84
7.3	Descripción de los módulos de código.....	84
7.3.1	src/mdp.py — Interfaz abstracta del MDP.....	84
	Método principal — q_value.....	85
	Métodos abstractos de la interfaz.....	85
7.3.2	src/cache_mdp.py — Formulación del caché como MDP.....	85
	Construcción del espacio de estados.....	85
	Optimización clave — transiciones dispersas.....	85
7.3.3	src/value_iteration.py — Value Iteration.....	86
	Algoritmo (pseudocódigo).....	86
	Criterio de convergencia — Ecuación 16.12.....	86
	Extracción de política.....	86
7.3.4	src/policy_iteration.py — Policy Iteration y Eliminación Gaussiana.....	86
	Algoritmo Policy Iteration (pseudocódigo).....	86
	Evaluación exacta de política — Ecuación 16.14.....	87
	Eliminación Gaussiana con pivoteo parcial.....	87
	Modified Policy Iteration.....	87
7.3.5	src/ucb.py — Agente UCB1 (Bandido Multi-brazo).....	87
	Índice UCB para el par (estado, acción).....	87
	Actualización incremental de la media.....	88
7.3.6	src/baselines.py — Políticas clásicas de reemplazo.....	88
7.3.7	src/scenarios.py — Distribuciones de acceso a páginas.....	88

Distribución Uniforme (UniformAccess).....	88
Distribución Zipf (ZipfAccess).....	88
Localidad temporal (TemporalLocalityAccess).....	89
Distribución no estacionaria (NonStationaryAccess).....	89
7.3.8 src/simulator.py — Motor de simulación.....	89
Flujo de cada paso (política MDP).....	89
Métricas recolectadas por SimResult.....	89
7.3.9 src/visualizer.py — Generación de gráficas.....	90
7.3.10 main.py — Ejecutor de experimentos.....	90
7.3.11 diagramas/generar_diagramas.py — Generador de diagramas UML.....	91
8. Manual de usuario.....	91
8.1 Requisitos previos.....	91
8.2 Estructura de ejecución.....	91
Verificación rápida (recomendado para empezar).....	91
Modo rápido (pruebas y desarrollo).....	92
Ejecución completa.....	92
Generar los diagramas draw.io.....	92
8.3 Flujo de ejecución recomendado.....	92
8.4 Archivos generados.....	93
9. Interpretación de resultados.....	93
9.1 Curvas de convergencia de Value Iteration.....	93
9.2 Heatmap de utilidades.....	95
9.3 Comparación de políticas.....	96
9.4 Recompensa acumulada.....	97
9.5 Comparación VI vs PI.....	98
9.6 Sensibilidad al factor de descuento.....	99
9.7 Entorno no estacionario.....	99
10. Ajuste de parámetros.....	100
10.1 Guía de ajuste de parámetros.....	100
10.2 Cambiar la distribución de acceso.....	101
10.3 Cambiar el parámetro de exploración de UCB.....	101
10.4 Usar Modified Policy Iteration.....	101
11. Escenarios de prueba.....	102
Escenario 1 — Correctitud Algoritmica: VI vs PI.....	102
Descripción y objetivo:.....	102
Configuración:.....	102
Resultados.....	102
Gráficas.....	103
Análisis.....	104
Escenario 2 — Comparación de Políticas.....	105

Descripción y objetivo:.....	105
Configuración:.....	105
Resultados.....	105
Gráficas.....	106
Análisis.....	108
Escenario 3 — Sensibilidad al Factor de Descuento gamma.....	109
Descripción y objetivo:.....	109
Configuración:.....	109
Resultados.....	109
Gráficas.....	109
Análisis.....	110
Conclusiones generales.....	111
12. Referencias.....	111

1. Introduccion

Descripción general

El proyecto desarrollado tiene como propósito central el diseño, implementación y evaluación de un simulador de gestión óptima de memoria caché, modelado formalmente como un Proceso de Decisión de Markov (MDP) y resuelto mediante los algoritmos y técnicas presentados en el Capítulo 16 de Making Complex Decisions del libro Artificial Intelligence: A Modern Approach de Russell y Norvig del 2021.

El trabajo parte de una motivación técnica y académica concreta, en sistemas computacionales modernos, la decisión de qué página desalojar de la caché cuando esta se encuentra llena determina en gran medida la eficiencia del sistema. A diferencia de las heurísticas clásicas como LRU (Least Recently Used) o FIFO (First In, First Out), el enfoque basado en MDPs permite encontrar una política de desalojo óptima que maximiza la utilidad esperada a lo largo del tiempo, incorporando explícitamente la distribución de probabilidad de los accesos futuros.

El sistema implementado modela el estado del caché como un estado de un MDP de horizonte infinito, y resuelve la política óptima de desalojo de forma exacta mediante Value Iteration y Policy Iteration, comparando los resultados contra líneas base clásicas (LRU, FIFO, Random y el óptimo offline de Belady) y contra un agente de aprendizaje online UCB1 (Upper Confidence

Bound). La implementación es completamente desde cero en Python, sin usar librerías de inteligencia artificial o aprendizaje por refuerzo, garantizando así la fidelidad directa con las ecuaciones y pseudocódigos del Capítulo 16.

Etapas del proyecto

El equipo ha abordado múltiples etapas de diseño e implementación que se detallan de la siguiente forma:

1. **Definición conceptual y teórica:** Cubrimiento del marco matemático sobre MDPs, ecuación de Bellman, función Q , y los algoritmos Value Iteration y Policy Iteration extraídos directamente del Capítulo 16 de Russell y Norvig.
2. **Formulación del problema cómo MDP:** Mapeo formal de los componentes del sistema de caché (S, A, P, R, γ) a la definición estándar de un Proceso de Decisión de Markov, justificando el cumplimiento de la propiedad de Markov, la naturaleza secuencial del problema y la función de recompensa basada en aciertos y fallos de caché.
3. **Diseño de la arquitectura de software:** Definición de la estructura modular del proyecto —interfaz abstracta MDP, formulación CacheMDP, solucionadores ValueIteration y PolicyIteration, agente UCBAgent, políticas baseline y motor de simulación y su documentación mediante diagramas UML de clases, componentes y secuencia.
4. **Implementación funcional en Python:** Desarrollo de los diez módulos de código que conforman la herramienta, implementando desde cero la eliminación gaussiana con pivoteo parcial para la evaluación exacta de política, el operador de actualización de Bellman, el criterio de convergencia de la Ecuación 16.12, y el índice UCB1 de la Sección 16.3.
5. **Experimentación y análisis de resultados:** Diseño y ejecución de seis experimentos que cubren convergencia de Value Iteration, comparación VI vs PI, comparación de políticas en simulación, sensibilidad al factor de descuento γ , escalabilidad del espacio de estados y comportamiento ante entornos no estacionarios.
6. **Interpretación de resultados y conclusiones:** Validación de la superioridad del enfoque MDP frente a heurísticas clásicas en distribuciones no uniformes, caracterización del trade-off entre agentes basados en modelo (MDP) y agentes

model-free (UCB) ante cambios del entorno, y análisis del impacto del factor de descuento en la política óptima resultante.

2. Marco Teorico

El presente marco teórico sustenta el desarrollo de un sistema de gestión óptima de memoria caché modelado como un **Proceso de Decisión de Markov (MDP)**. Los fundamentos conceptuales, matemáticos y algorítmicos utilizados provienen del *Capítulo 16: Making Complex Decisions* del libro *Artificial Intelligence: A Modern Approach*, cuarta edición global, de Russell y Norvig (2021).

En sistemas computacionales modernos, la memoria caché constituye un componente crítico para el rendimiento. La decisión de qué página desalojar cuando la caché está llena determina en gran medida la eficiencia del sistema. A diferencia de heurísticas clásicas como LRU (Least Recently Used) o FIFO (First In, First Out), el enfoque basado en MDPs permite encontrar una **política de desalojo óptima** que maximiza la utilidad esperada a lo largo del tiempo, tomando en cuenta explícitamente la distribución de probabilidad de los accesos futuros.

El capítulo 16 introduce la maquinaria matemática necesaria para abordar este tipo de problema: la formalización de decisiones secuenciales bajo incertidumbre, la ecuación de Bellman, y dos algoritmos fundamentales — **Value Iteration** y **Policy Iteration** — que permiten calcular la política óptima de forma exacta.

Problemas de decisión secuencial

A diferencia de los problemas de decisión de un solo paso, en los que el agente realiza una única acción y obtiene una recompensa inmediata, los **problemas de decisión secuencial** requieren que el agente tome una sucesión de decisiones a lo largo del tiempo. En este tipo de problemas, cada decisión afecta no solo la recompensa inmediata, sino también el estado futuro del sistema y, con ello, las recompensas que podrán obtenerse en adelante.

"In this chapter, we address the computational issues involved in making decisions in a stochastic environment. [...] we are concerned here with sequential decision problems, in

which the agent's utility depends on a sequence of decisions." (Russell & Norvig, 2021, p. 553)

En el contexto del proyecto, el agente es el **sistema operativo** y cada decisión corresponde a la elección de qué página desalojar de la caché cuando ocurre un fallo de página (*cache miss*). La naturaleza secuencial es evidente: la página que se mantenga en caché hoy determinará si los accesos futuros generan hits o misses, afectando el rendimiento global del sistema.

Una propiedad fundamental de estos problemas es la **propiedad de Markov**: la probabilidad de transición al siguiente estado depende únicamente del estado actual y de la acción tomada, no del historial completo de estados pasados. Esta propiedad simplifica enormemente el análisis matemático y es el requisito central para que el problema pueda modelarse como un MDP.

Proceso de Decisión de Markov (MDP)

Definición Formal

Un **Proceso de Decisión de Markov** es la formalización matemática de un problema de decisión secuencial en un entorno estocástico y completamente observable. Formalmente, un MDP se define como una tupla:

$$MDP = \langle S, A, P, R, \gamma \rangle$$

donde cada componente tiene el siguiente significado:

Componente	Descripción
S	Conjunto finito de estados del entorno
A(s)	Conjunto de acciones disponibles en el estado s
P(s' s,a)	Modelo de transición: probabilidad de llegar al estado s' partiendo del estado s tomando la acción a
R(s,a,s')	Función de recompensa: valor numérico recibido al transitar de s a s' mediante la acción a
$\gamma \in [0,1]$	Factor de descuento: pondera la importancia de recompensas futuras vs. inmediatas

"A sequential decision problem for a fully observable, stochastic environment with a Markovian transition model and additive rewards is called a Markov decision process, or MDP, and consists of a set of states (with an initial state s_0); a set $ACTIONS(s)$ of actions in each state; a transition model $P(s'|s,a)$; and a reward function $R(s,a,s')$."
(Russell & Norvig, 2021, p. 553)

Mapecto al Problema de Caché

Cada componente del MDP se corresponde de manera directa con elementos del sistema de caché implementado en este proyecto:

- **Estados S :** cada estado es el conjunto de páginas actualmente almacenadas en la caché. Formalmente, $s \in S$ es un subconjunto de páginas del espacio de trabajo con cardinalidad $\leq \text{cache_size}$. Se representa como un frozenset inmutable.
- **Acciones $A(s)$:** cuando la caché está llena ($|s| = \text{cache_size}$) y ocurre un miss, la acción consiste en seleccionar qué página desalojar. $A(s) = \{p \mid p \in s\}$. Si hay espacio libre, no se requiere desalojo (acción nula).
- **Transición $P(s'|s,a)$:** el nuevo estado depende de la acción a (qué página se desaloja) y de la página solicitada en el siguiente ciclo, que sigue una distribución de probabilidad estocástica $\text{access_probs}[i]$. La propiedad de Markov se cumple porque el nuevo estado depende solo del estado actual, no del historial.
- **Recompensa $R(s,a,s')$:** $R = +1$ si la página solicitada estaba en caché (cache hit); $R = -1$ si no estaba (cache miss). Esta función de recompensa le da al agente el incentivo de mantener en caché las páginas más probables de ser solicitadas.
- **Factor de descuento γ :** controla si el agente optimiza para el rendimiento inmediato ($\gamma \rightarrow 0$) o para el rendimiento a largo plazo ($\gamma \rightarrow 1$). En los experimentos se evalúan $\gamma = 0.9$ y $\gamma = 0.5$.

Utilidad y Recompensas Descontadas

La Función de Utilidad sobre Historias

Dado que el agente toma decisiones a lo largo de un horizonte potencialmente infinito, necesitamos una forma de agregar las recompensas recibidas en cada paso en un único número que represente la **calidad total del comportamiento**. Russell y Norvig (2021) justifican el uso de las **recompensas aditivas descontadas** como el mecanismo estándar para este propósito.

La utilidad de una historia de estados se define como:

$$U_h([s_0, a_0, s_1, a_1, \dots]) = R(s_0, a_0, s_1) + \gamma R(s_1, a_1, s_2) + \gamma^2 R(s_2, a_2, s_3) + \dots$$

$$U_h = \sum_{t=0}^{\infty} \gamma^t \cdot R(s_t, a_t, s_{t+1})$$

El Factor de Descuento γ

El **factor de descuento** $\gamma \in [0,1)$ cumple un papel central en la definición del comportamiento del agente:

- **$\gamma \approx 0$:** el agente es "miope" — solo le importa la recompensa del próximo paso. En el contexto de caché, priorizará mantener la página más probable de ser solicitada en el siguiente acceso, sin importar el largo plazo.
- **$\gamma \approx 1$:** el agente es "paciente" — valora las recompensas futuras casi tanto como las inmediatas. Buscará una política que maximice el rendimiento acumulado a lo largo de miles de accesos.
- **$\gamma = 1$:** solo es admisible si el proceso tiene estado terminal. En horizontes infinitos sin terminal, la suma puede diverger.

Adicionalmente, un valor de $\gamma < 1$ garantiza que la suma de recompensas a lo largo de un horizonte infinito sea **finita y acotada**. Si las recompensas están acotadas en valor absoluto por $R_{\max}$, entonces:

$$U_h \leq \sum_{t=0}^{\infty} \gamma^t \cdot R_{\max} = \frac{R_{\max}}{1 - \gamma}$$

"The discount factor describes the preference of an agent for current rewards over future rewards. When γ is close to 0, rewards in the distant future are viewed as insignificant. When γ is close to 1, an agent is more willing to wait for long-term rewards." (Russell & Norvig, 2021, p. 555)

Política Óptima

Definición de Política

Una **política** π es una función que asigna a cada estado s una acción a tomar: $\pi: S \rightarrow A$. En lugar de planificar una secuencia fija de acciones, el agente sigue una política que le indica qué hacer en *cualquier* estado que pueda alcanzar, independientemente de cómo llegó allí.

La **política óptima** π^* es aquella que maximiza la utilidad esperada de las historias generadas al ejecutarla, partiendo de cualquier estado inicial:

$$\pi^*(s) = \operatorname{argmax}_{\pi} E\left[\sum_{t=0}^{\infty} \gamma^t \cdot R(S_t, \pi(S_t), S_t^{+1})\right]$$

"An optimal policy is a policy that yields the highest expected utility. [...] A remarkable consequence of using discounted utilities with infinite horizons is that the optimal policy is independent of the starting state." (Russell & Norvig, 2021, p. 557)

Utilidad de un Estado

La **utilidad de un estado** $U(s)$ es la utilidad esperada que el agente obtiene si, estando en el estado s , ejecuta la política óptima π^* a partir de ese momento. Formalmente:

$$U(s) = U^{\pi^*}(s) = E\left[\sum_{t=0}^{\infty} \gamma^t \cdot R(S_t, \pi^*(S_t), S_t^{+1}) \mid S_0 = s\right]$$

Los estados con mayor utilidad son aquellos desde los cuales el agente puede acumular más recompensas en el futuro bajo la política óptima. En el sistema de caché, los estados con mayor utilidad serán aquellos que contienen en caché las páginas más frecuentemente solicitadas.

La Ecuación de Bellman

Formulación

La **ecuación de Bellman** es la relación de recursividad que define la función de utilidad óptima. Establece que la utilidad de un estado es igual a la mejor recompensa inmediata esperada más el valor descontado del mejor estado sucesor posible:

$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s' | s, a) \cdot [R(s, a, s') + \gamma \cdot U(s')]$$

Esta ecuación, nombrada en honor a **Richard Bellman (1957)**, tiene una interpretación intuitiva directa: la utilidad de estar en el estado s es el valor de la *mejor decisión posible* que el agente puede tomar ahora mismo, considerando tanto la recompensa inmediata como las consecuencias futuras de esa decisión.

"This is called the Bellman equation [...] The utilities of the states—defined as the expected utility of subsequent state sequences—are solutions of the set of Bellman equations. In fact, they are the unique solutions." (Russell & Norvig, 2021, p. 558)

La Función Q y Extracción de Política

La **función Q(s,a)**, también llamada *action-utility function*, representa el valor esperado de tomar la acción a en el estado s y luego seguir la política óptima:

$$Q(s, a) = \sum_{s'} P(s' | s, a) \cdot [R(s, a, s') + \gamma \cdot U(s')]$$

Las relaciones entre U , Q y π^* son las siguientes:

$$U(s) = \max_a Q(s, a)$$

$$\pi^*(s) = \operatorname{argmax}_a Q(s, a)$$

Estas relaciones son fundamentales en la implementación: el algoritmo calcula $Q(s,a)$ para cada par (estado, acción) y la política óptima se extrae simplemente eligiendo la acción de mayor valor Q en cada estado. En el código Python del proyecto, la función **compute_Q_value(state, action, U)** implementa directamente esta ecuación.

Interpretación para el Sistema de Caché

En el contexto del problema de caché, la ecuación de Bellman dice lo siguiente: la utilidad de tener un conjunto de páginas en caché es igual a la **máxima** suma esperada de:

- La recompensa inmediata: +1 si la próxima página solicitada está en caché (hit) o -1 si no está (miss).
- El valor descontado de la nueva configuración de caché resultante, después de la posible incorporación de la página solicitada.

La política óptima resulta de maximizar esta expresión eligiendo, en cada estado de caché llena, qué página desalojar para maximizar la utilidad esperada futura.

Algoritmo de Value Iteration

Idea Central

El algoritmo **Value Iteration** (Figura 16.6 del libro) resuelve el sistema de ecuaciones de Bellman de forma iterativa. Dado que estas ecuaciones son no lineales (por el operador *max*), no se pueden resolver directamente con álgebra lineal estándar. La solución es comenzar con estimaciones arbitrarias de las utilidades (usualmente cero) y aplicar repetidamente la actualización de Bellman hasta que los valores converjan.

La Actualización de Bellman (Bellman Update)

El núcleo del algoritmo es la **actualización de Bellman**, que en cada iteración *i* reemplaza la utilidad de cada estado con el mejor Q-valor disponible:

$$U_{i+1}(s) \leftarrow \max_{a \in A(s)} \sum_{s'} P(s' | s, a) \cdot [R(s, a, s') + \gamma \cdot U_i(s')]$$

Esta operación se aplica **simultáneamente** a todos los estados en cada iteración, propagando información de utilidad a través del espacio de estados.

Convergencia

La convergencia del algoritmo se fundamenta matemáticamente en el concepto de **contracción**. El operador de Bellman B es una contracción con factor γ en la norma infinito:

$$\| B \cdot U_i - B \cdot U'_i \| \leq \gamma \cdot \| U_i - U'_i \|$$

Como toda contracción tiene un único punto fijo (en este caso, la función de utilidad óptima U^*), la iteración repetida del operador B converge siempre que $\gamma < 1$. El algoritmo se detiene cuando el cambio máximo en una iteración δ satisface la condición:

$$\delta \leq \varepsilon \cdot \frac{(1-\gamma)}{\gamma}$$

Esta condición garantiza que el error de la estimación de utilidad sea menor que ε , y que la pérdida de política (policy loss) de ejecutar la política actual en lugar de la óptima sea menor que 2ε .

"if $\|U_{i+1} - U_i\| < \varepsilon(1-\gamma)/\gamma$ then $\|U_{i+1} - U\| < \varepsilon$ " (Russell & Norvig, 2021, p. 565, ecuación 16.12)

Complejidad

Cada iteración del algoritmo requiere calcular $Q(s,a)$ para cada par (estado, acción). El costo por iteración es $O(|S|^2 \cdot |A|)$ en el peor caso. El número de iteraciones necesario para garantizar error ε es:

$$N = \lceil \frac{\log(\frac{2 R_{max}}{\varepsilon (1-\gamma)})}{\log(\frac{1}{\gamma})} \rceil$$

Algoritmo de Policy Iteration

Motivación

Si bien Value Iteration eventualmente converge a la solución correcta, en la práctica **la política óptima converge mucho antes que la función de utilidad**. Esto sugiere un enfoque

alternativo: en lugar de iterar sobre utilidades, iterar directamente sobre *políticas*. Esta es la idea central de **Policy Iteration** (Russell & Norvig, 2021, Sección 16.2.2).

Pasos del Algoritmo

Paso 1 — Evaluación de política (Policy Evaluation):

Dado que la política π está fija, el operador max desaparece de la ecuación de Bellman, convirtiéndola en un sistema de ecuaciones **lineales**. Para el estado s bajo política π , la ecuación simplificada es:

$$U_i(s) = \sum_{s'} P(s' | s, \pi(s)) \cdot [R(s, \pi(s), s') + \gamma \cdot U_i(s')]$$

Este sistema lineal puede resolverse exactamente en $O(n^3)$ o aproximadamente con iteración (*modified policy iteration*), que es lo implementado en este proyecto por mayor eficiencia.

Paso 2 — Mejora de política (Policy Improvement):

Para cada estado, se verifica si existe alguna acción que produzca un Q-valor mayor que el de la acción actual de la política. Si existe, se actualiza la política:

$$a^* = \operatorname{argmax}_{a \in A(s)} Q(s, a, U)$$

$$\text{si } Q(s, a^*, U) > Q(s, \pi[s], U) \rightarrow \pi[s] = a^*$$

Convergencia

Dado que el espacio de políticas para un MDP finito es finito (en el peor caso $|A|^{|S|}$ políticas), y cada iteración de mejora produce una política igual o **estrictamente mejor** que la anterior, el algoritmo siempre converge en un número finito de iteraciones. La condición de parada es que el paso de mejora no genere ningún cambio en la política.

Comparación entre los Algoritmos

Criterio	Value Iteration	Policy Iteration
Itera sobre	Función de utilidad $U(s)$	Políticas π
Costo por iteración	$O(S ^2 \cdot A)$	$O(S ^3 + S ^2 \cdot A)$
Iteraciones hasta convergencia	Más iteraciones (numéricas)	Menos iteraciones (exactas)
Resultado directo	Función de utilidad U^*	Política óptima π^*
Implementación	Más sencilla	Ligeramente más compleja

3. Descripción y justificación del problema a resolver.

Contexto General

En el diseño de sistemas computacionales modernos, la **memoria caché** actúa como un nivel intermedio de almacenamiento de alta velocidad ubicado entre el procesador y la memoria principal. Su propósito es reducir la latencia de los accesos a datos frecuentemente utilizados, aprovechando el principio de **localidad de referencia**: los programas tienden a acceder repetidamente a un subconjunto reducido de sus datos en ventanas de tiempo cortas.

La eficiencia de la caché se mide principalmente a través de dos métricas: la **tasa de aciertos (hit rate)**, que representa la fracción de accesos en que la página solicitada ya se encontraba en caché, y la **tasa de fallos (miss rate)**, que representa los accesos que requirieron cargar la página desde niveles de almacenamiento más lentos. Un fallo de caché puede implicar penalizaciones de rendimiento de varios órdenes de magnitud respecto a un acierto.

El Problema de Reemplazo de Páginas

La caché tiene una capacidad limitada: solo puede almacenar un número fijo de páginas simultáneamente. Cuando el sistema necesita cargar una página que no se encuentra en caché

(*cache miss*) y la caché ya está llena, se debe **desalojar** una de las páginas existentes para dar espacio a la nueva. La decisión de *qué página desalojar* es el problema central abordado en este proyecto.

Esta decisión no es trivial. Desalojar la página equivocada puede generar una cadena de fallos de caché futuros, degradando el rendimiento del sistema de manera acumulativa. El desafío radica en que, en el momento de tomar la decisión, el sistema **no conoce con certeza** qué páginas serán solicitadas en el futuro: solo dispone de una distribución de probabilidad sobre los accesos futuros, derivada de los patrones de uso observados.

Pertinencia del Capítulo 16

El problema de gestión de caché seleccionado cumple de manera rigurosa con los criterios establecidos por el Capítulo 16 de Russell y Norvig (2021) para ser modelado como un Proceso de Decisión de Markov, como se detalla a continuación.

Carácter secuencial: cada decisión de desalojo afecta el contenido de la caché, que a su vez determina el resultado (hit or miss) de los accesos futuros. No se trata de una decisión aislada sino de una cadena de decisiones interrelacionadas en el tiempo.

Estocasticidad: los accesos futuros a páginas siguen una distribución de probabilidad que el sistema no puede controlar. El entorno es genuinamente estocástico: el mismo estado de caché puede derivar en estados distintos dependiendo de qué página solicite el CPU a continuación.

Propiedad de Markov: la distribución de probabilidad del siguiente estado del sistema depende únicamente del estado actual de la caché y de la acción de desalojo tomada, no del historial completo de accesos pasados. Esto es la esencia de la propiedad de Markov, condición central de los MDPs.

Recompensas aditivas a lo largo del tiempo: el rendimiento del sistema se mide como la suma acumulada de aciertos y fallos a lo largo de miles de accesos. Esto corresponde exactamente al modelo de recompensas aditivas descontadas que el Capítulo 16 establece como fundamento para comparar políticas.

"Sequential decision problems incorporate utilities, uncertainty, and sensing, and include search and planning problems as special cases." (Russell & Norvig, 2021, p. 553)

Relevancia en el Dominio de la Computación

La gestión de caché es un problema de importancia central en **múltiples niveles de la jerarquía de sistemas computacionales**

- **Sistemas operativos:** el gestor de memoria virtual utiliza algoritmos de reemplazo de páginas para administrar la memoria RAM como caché del disco. Una política subóptima produce thrashing: el sistema pasa más tiempo cargando páginas que ejecutando código.
- **Arquitectura de procesadores:** las cachés L1, L2 y L3 de los procesadores modernos utilizan políticas de reemplazo en hardware. Su diseño impacta directamente la velocidad de ejecución de todo el software.
- **Bases de datos:** los sistemas de gestión de bases de datos mantienen un buffer pool que actúa como caché de bloques de disco. La política de reemplazo del buffer pool tiene impacto directo sobre el tiempo de respuesta de consultas.
- **Redes de distribución de contenido (CDN):** los servidores caché de una CDN deben decidir qué contenido mantener almacenado localmente para minimizar la latencia de los usuarios y el tráfico hacia los servidores de origen.

En todos estos dominios, el problema de reemplazo de caché comparte la misma estructura matemática: es un problema de decisión secuencial con incertidumbre sobre el futuro. El modelo MDP proporciona el **marco teórico unificado** para abordarlo de forma óptima.

Cumplimiento del Requisito del 50%

El enunciado del proyecto exige que al menos el 50% de la solución emplee directamente los modelos, algoritmos y técnicas del Capítulo 16 asignado. La siguiente tabla evidencia que este requisito se cumple ampliamente:

Componente del Capítulo 16	Uso en el proyecto	Sección libro
Formalización MDP (S, A, P, R, γ)	Modelo completo del sistema de caché	16.1

Ecuación de Bellman	Núcleo del cálculo de utilidades en el código	16.1.2
Función $Q(s,a)$	Implementada en <code>compute_Q_value()</code>	16.1.2
Value Iteration	Implementado desde cero: <code>value_iteration()</code>	16.2.1
Policy Iteration	Implementado desde cero: <code>policy_iteration()</code>	16.2.2
Policy Evaluation	Implementada: <code>policy_evaluation()</code>	16.2.2
Extracción de política óptima	Implementada: <code>extract_policy()</code>	16.1.2
Convergencia y error acotado	Criterio de parada con $\delta \leq \frac{\epsilon \cdot (1-\gamma)}{\gamma}$	16.2.1

Todos los algoritmos centrales — Value Iteration, Policy Iteration y la función Q — fueron implementados **desde cero en Python** sin utilizar librerías de inteligencia artificial o aprendizaje por refuerzo. El código implementa directamente las ecuaciones y pseudocódigos del Capítulo 16, haciendo del proyecto una aplicación fiel y verificable de los fundamentos teóricos estudiados.

4. Diseño de la aplicación.

En esta sección se detallan las especificaciones técnicas, la arquitectura del sistema y el diseño metodológico empleado para la construcción del simulador de gestión de caché basado en Procesos de Decisión de Markov.

4.1 Arquitectura y Modelado del Sistema

El sistema ha sido diseñado bajo un modelo de estado aumentado que captura simultáneamente el contenido del caché y la solicitud en curso, permitiendo que el agente tome decisiones de desalojo con plena información del estado actual. La arquitectura contempla los siguientes niveles:

- **Espacio de páginas ($N = 5$ páginas):** Conjunto de páginas del espacio de trabajo del proceso simulado. Constituye el universo de elementos que pueden estar o no en caché en cada instante.

- **Caché de capacidad limitada ($K = 3$ páginas):** Representa la memoria intermedia de alta velocidad. En todo momento mantiene como máximo K páginas, y cuando está llena y ocurre un fallo, el sistema debe seleccionar una víctima para desalojo.
- **Espacio de estados aumentado ($|S| = 50$ estados):** Cada estado es un par (configuración_caché, página_solicitada), donde la configuración es un frozenset de K páginas. El total de estados es $C(N, K) \times N = C(5, 3) \times 5 = 10 \times 5 = 50$ estados.
- **Espacio de acciones:** En estados de hit la única acción es 'noop'. En estados de miss con caché llena, el agente elige cuál de las K páginas desalojar: $|A(s)| = K = 3$ acciones disponibles.

4.2 Especificaciones Técnicas

Para garantizar la precisión, reproducibilidad y fidelidad teórica del experimento, se utilizaron las siguientes herramientas y tecnologías:

Lenguaje de implementación principal: Python 3.8 o superior. Todos los algoritmos del MDP (Value Iteration, Policy Iteration, Eliminación Gaussiana, UCB1) se implementaron desde cero utilizando únicamente la biblioteca estándar de Python, sin librerías de IA ni aprendizaje por refuerzo.

Librería de visualización: matplotlib (exclusivamente para la generación de gráficas PNG; no interviene en ningún algoritmo de solución).

Algoritmos de solución implementados: Value Iteration (Figura 16.6, R&N), Policy Iteration con Eliminación Gaussiana (Figura 16.9, R&N), Modified Policy Iteration, y UCB1 (Sección 16.3, R&N).

Políticas baseline de comparación: LRU (Least Recently Used), FIFO (First In, First Out), Random y Óptimo offline de Belady.

Factor de descuento γ : Evaluado en el rango $[0.1, 0.99]$. Configuración principal: $\gamma = 0.9$.

Tolerancia de convergencia ϵ : 0.001 (criterio de parada de la Ecuación 16.12: $\delta \leq \frac{\epsilon \cdot (1-\gamma)}{\gamma}$).

Pasos de simulación por experimento: 10 000 pasos en ejecución completa, 2 000 en modo rápido.

4.3 Formulación del MDP de Caché

Cada componente de la tupla $MDP = \langle S, A, P, R, \gamma \rangle$ se corresponde de manera directa con elementos del sistema de caché:

Componente MDP	Mapeo al sistema de caché
S — Estados	Pares (frozenset de páginas en caché, página solicitada). $ S = C(N,K) \times N$
A(s) — Acciones	'noop' si hit; $\{('evict', p) \mid p \in \text{caché}\}$ si miss con caché llena
$P(s' s,a)$ — Transición	Determinística en la nueva configuración; estocástica sobre la próxima solicitud (access_probs[q])
$R(s,a,s')$ — Recompensa	+1 si la página solicitada está en caché (hit); -1 si no está (miss)
γ — Descuento	Controla orientación temporal: $\gamma \approx 0$ (miope) vs $\gamma \approx 1$ (largo plazo)

4.4 Distribuciones de Acceso Implementadas

Se implementaron cuatro distribuciones de acceso a páginas para cubrir diferentes patrones de uso reales:

- **Distribución Uniforme (UniformAccess):** Todas las páginas con probabilidad igual $1/N$. Caso base donde ninguna política tiene ventaja sobre otra.
- **Distribución Zipf (ZipfAccess, $\alpha=1.0$):** Ley de potencia donde la página de rango r tiene probabilidad $\propto 1/r^\alpha$. Modela distribuciones reales de acceso a contenido web y sistemas de archivos.
- **Localidad Temporal (TemporalLocalityAccess):** Un subconjunto 'caliente' de páginas concentra el 80% del tráfico. Modela el principio de localidad de referencia de los programas.
- **Distribución No Estacionaria (NonStationaryAccess):** Combina dos distribuciones con un cambio abrupto en el paso switch_step. Permite evaluar la adaptabilidad de UCB frente a una política MDP fija.

4.5 Escenarios de Experimentación

Se definieron seis experimentos sistemáticos para validar el sistema y contrastar los algoritmos implementados:

Experimento	Descripción	Métricas principales
o — Verificación	Comprueba $\sum P(s' s,a)=1 \forall (s,a)$ y que VI y PI producen la misma política.	Corrección lógica

1 — Convergencia VI	Ejecuta Value Iteration en los 3 escenarios y genera curvas de convergencia.	Iteraciones, delta por iteración
2 — VI vs PI	Compara el número de iteraciones y tiempo de cómputo de los dos algoritmos.	Iteraciones, tiempo (s)
3 — Comparación políticas	Simula 6 métodos (MDP-VI, MDP-PI, LRU, FIFO, Random, UCB, Belady) en 3 escenarios.	Hit rate, recompensa acumulada
4 — Sensibilidad γ	Varía γ de 0.1 a 0.99 y mide el hit rate resultante.	Hit rate vs γ
5 — Escalabilidad	Mide tiempo de cómputo para distintas combinaciones (N, K): de 4 a 8 páginas.	Tiempo de cómputo, S
6 — No estacionario	La distribución cambia a la mitad: UCB se adapta, MDP mantiene política fija.	Recompensa acumulada comparada

4.6 Arquitectura Modular del Software

El proyecto está organizado en diez módulos de Python con responsabilidades bien delimitadas:

Módulo	Responsabilidad	Referencia R&N
src/mdp.py	Interfaz abstracta MDP; implementa q_value, best_action, best_q	§16.1, p.553
src/cache_mdp.py	Formulación del caché como MDP: estados, acciones, transiciones, recompensa	§16.1, pp.552-553
src/value_iteration.py	Algoritmo Value Iteration con criterio de parada Ec.16.12	§16.2.1, Fig.16.6
src/policy_iteration.py	Policy Iteration + Eliminación Gaussiana + Modified PI	§16.2.2, Fig.16.9
src/ucb.py	Agente UCB1 online (bandido multi-brazo)	§16.3
src/baselines.py	Políticas LRU, FIFO, Random, Óptimo Belady	—
src/scenarios.py	Distribuciones Uniforme, Zipf, Localidad Temporal, No Estacionaria	—
src/simulator.py	Motor de simulación; recolecta hit rate, recompensa acumulada	—
src/visualizer.py	Generación de gráficas PNG con matplotlib	—

main.py	Ejecutor de los 6 experimentos; punto de entrada principal	—
---------	--	---

5. Diagramas UML

Diagrama UML

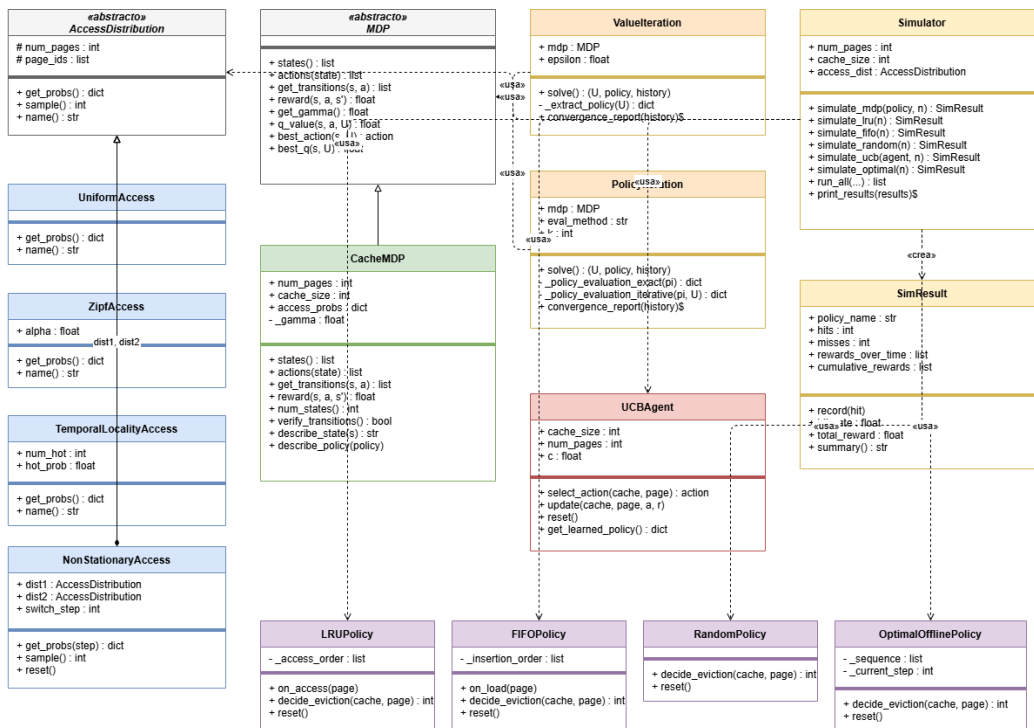


Diagrama de componentes

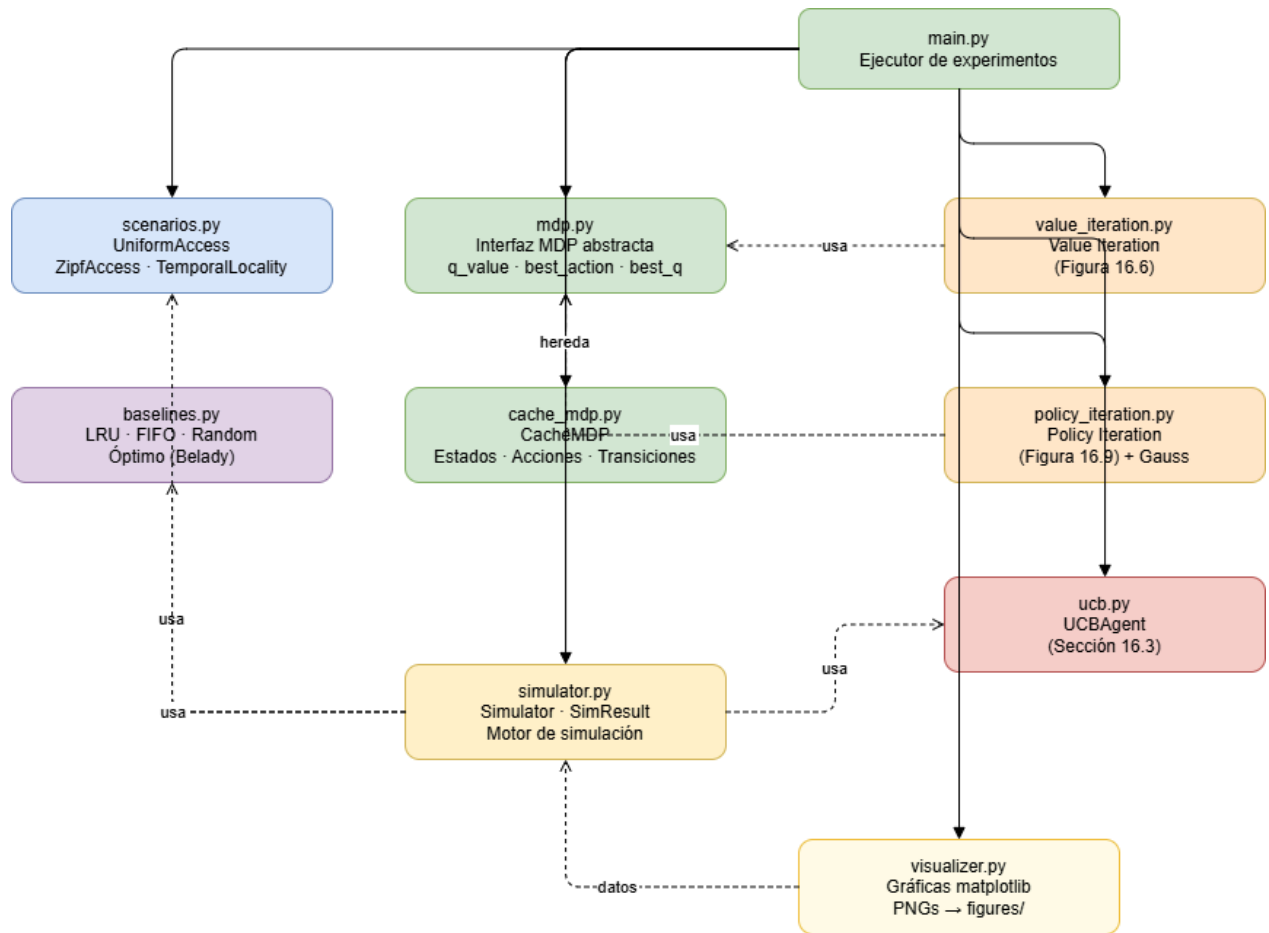


Diagrama de secuencia de la simulación

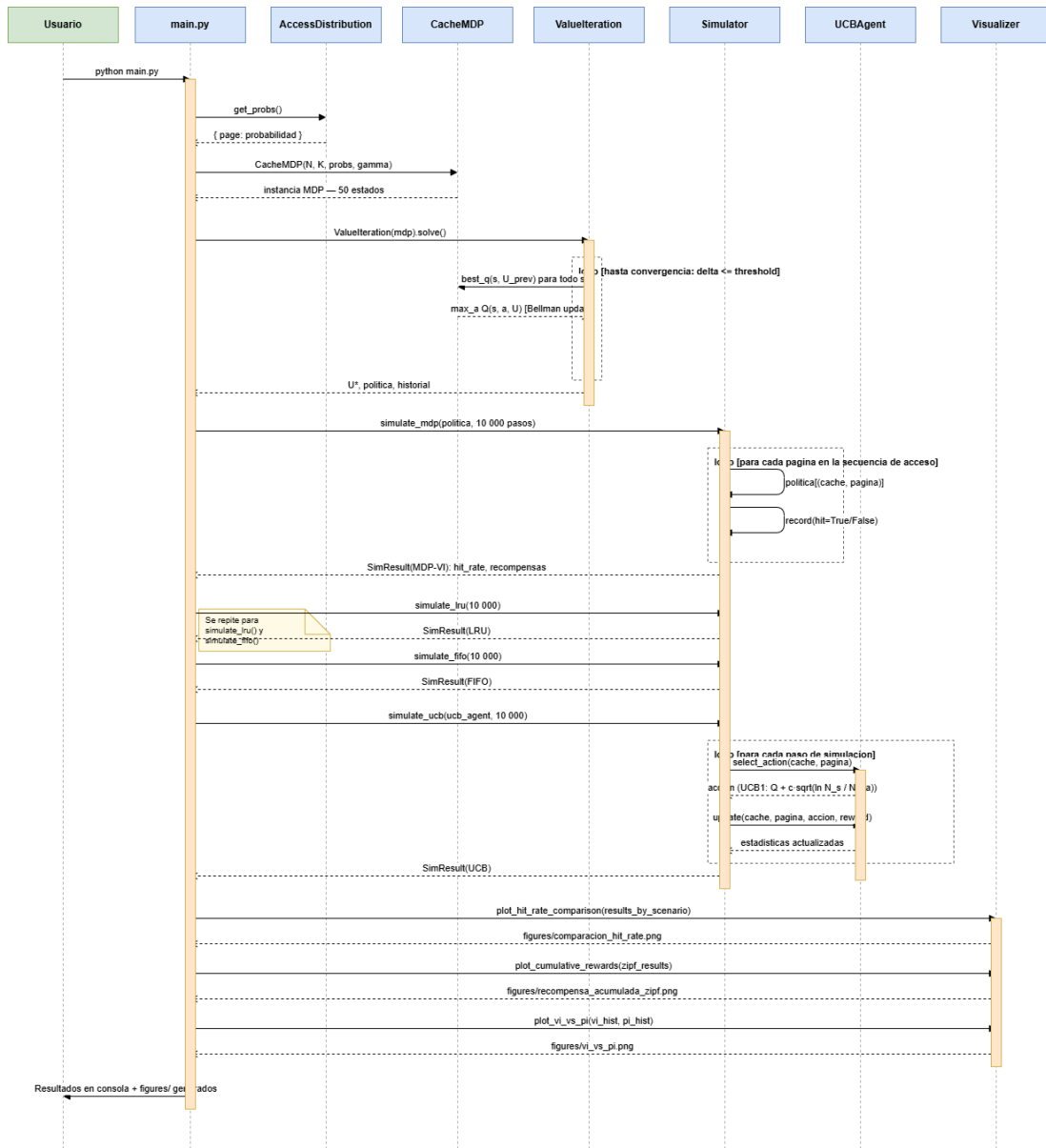


Diagrama de Flujo: Value Iteration

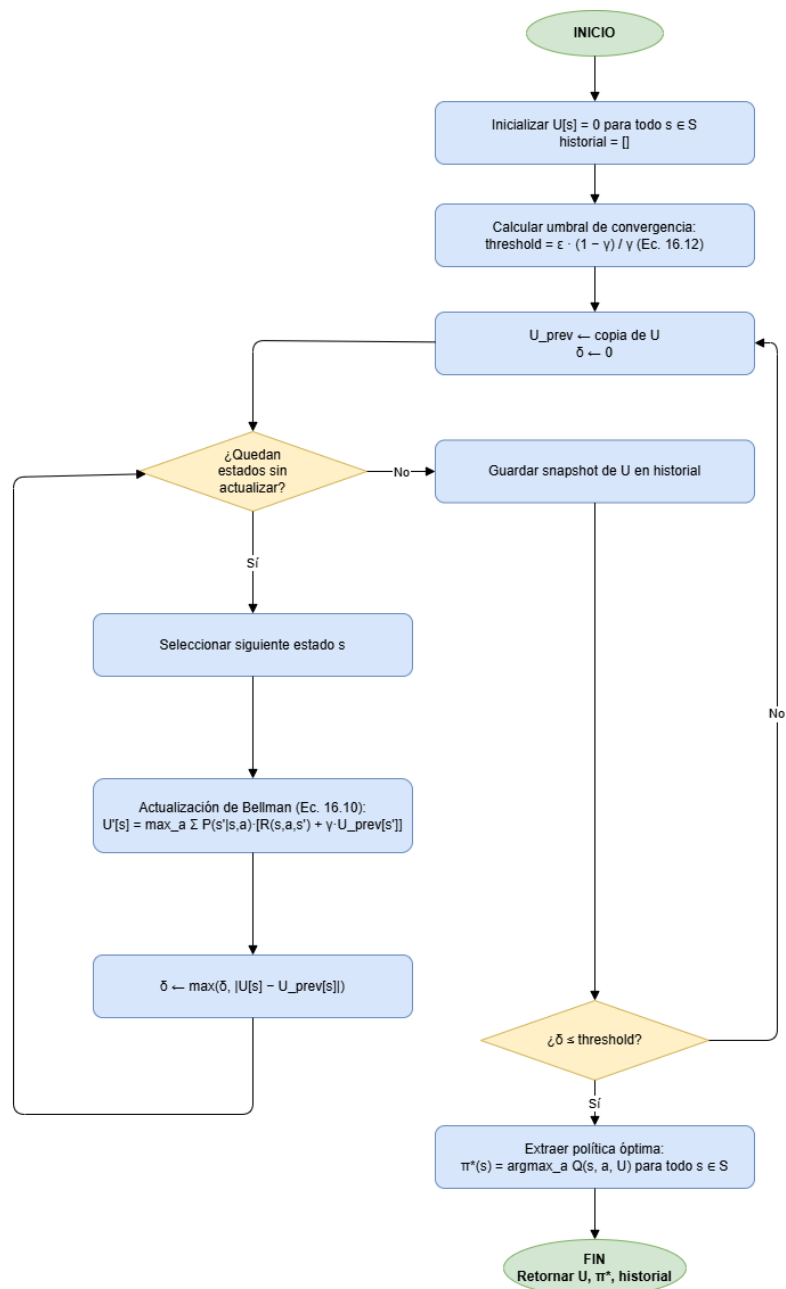
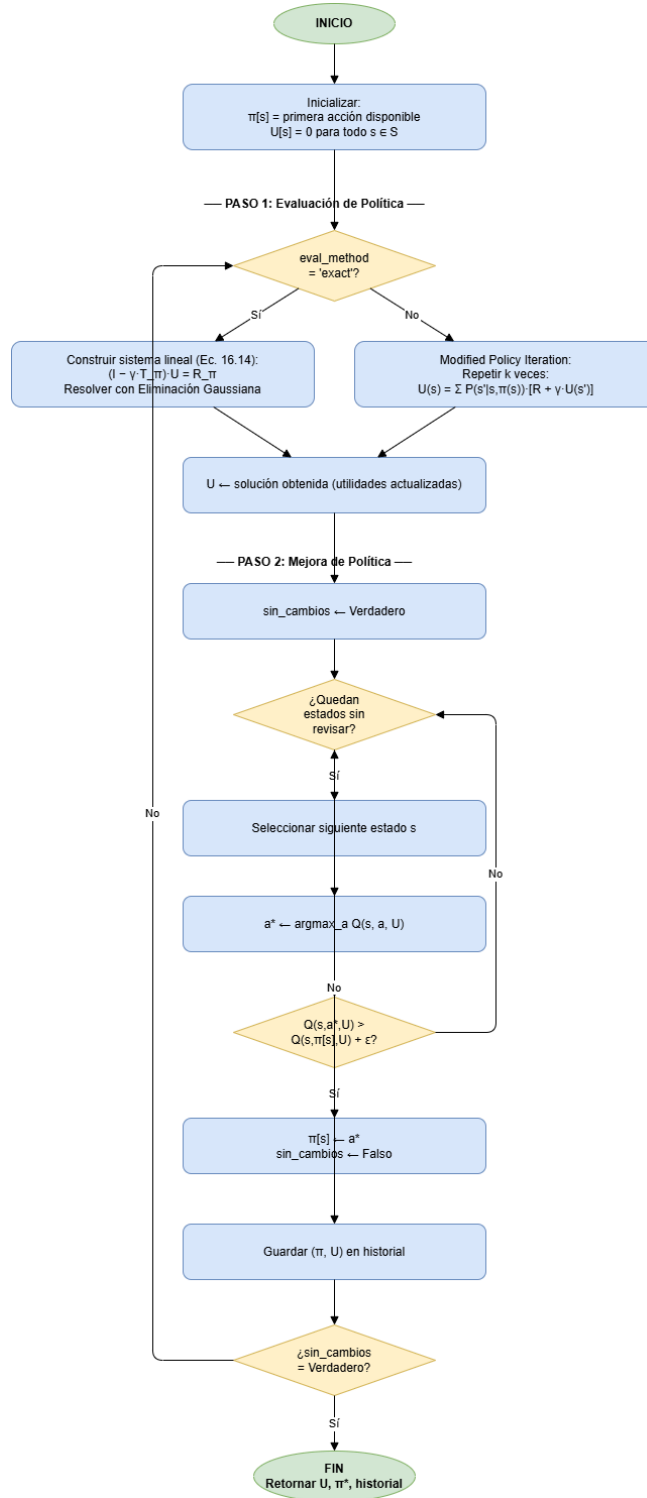


Diagrama de Flujo: Policy iteration



6. Código fuente completo de la herramienta construida.

Se construyen un total de diez archivos de código en Python:

1. [src/mdp.py](#)

Define la clase base MDP de la que heredan todos los MDPs concretos. Implementa directamente la función `q_value` que utilizan los dos algoritmos de solución.

```
Python
"""
Clase base abstracta para un Proceso de Decisión de Markov (MDP).
Russell & Norvig, Capítulo 16.

Define la interfaz común que deben implementar todos los MDPs concretos.
Los algoritmos de solución (Value Iteration, Policy Iteration) operan
exclusivamente sobre esta interfaz.
"""

class MDP:
    """Proceso de Decisión de Markov abstracto.

    Las subclases deben implementar: states, actions, get_transitions, reward,
    get_gamma.
    El método q_value está implementado aquí y es utilizado por todos los
    algoritmos.
    """

    def states(self):
        """Retorna la lista de todos los estados del MDP."""
        raise NotImplementedError

    def actions(self, state):
        """Retorna la lista de acciones disponibles en el estado dado."""
        raise NotImplementedError

    def get_transitions(self, state, action):
```

0.

"""Retorna una lista de pares (siguiente_estado, probabilidad) con prob > 0.

Esta representación dispersa evita iterar sobre todos los estados al calcular los Q-valores (la mayoría de las transiciones tienen probabilidad 0).

```

"""
raise NotImplementedError

def reward(self, state, action, next_state):
    """Retorna R(s, a, s') - la recompensa inmediata por la transición."""
    raise NotImplementedError

def get_gamma(self):
    """Retorna el factor de descuento gamma."""
    raise NotImplementedError

def q_value(self, state, action, U):
    """Calcula Q(s, a) dados los estimados actuales de utilidad U.

```

Función Q-VALUE de R&N página 559:

$$Q(s, a) = \sum_{s'} P(s'|s,a) * [R(s,a,s') + \gamma * U[s']]$$

Args:

state: estado actual s
 action: acción a
 U: diccionario estado -> estimado de utilidad

Returns:

float: Q-valor para el par (estado, acción)

```

"""
gamma = self.get_gamma()
total = 0.0
for next_state, prob in self.get_transitions(state, action):
    r = self.reward(state, action, next_state)
    total += prob * (r + gamma * U.get(next_state, 0.0))
return total

def best_action(self, state, U):
    """Retorna la acción que maximiza Q(s, a) para el U dado."""
    acts = self.actions(state)
    return max(acts, key=lambda a: self.q_value(state, a, U))

```

```
def best_q(self, state, U):
    """Retorna max_a Q(s, a) para el U dado."""
    return max(self.q_value(state, a, U) for a in self.actions(state))
```

2. `src/cache_mdp.py`: Implementa `CacheMDP`, la subclase concreta de `MDP` que modela el problema de gestión de caché.

Python

"""

Gestión de caché modelada como un MDP – R&N Capítulo 16.

ESTADO: estado aumentado (`cache_contents`: `frozenset`, `requested_page`: `int`)

- `cache_contents`: páginas actualmente en caché (tamaño `K`)
- `requested_page`: página que la CPU acaba de solicitar

ACCIÓN:

- "noop" si la página solicitada ya está en caché (hit)
- ("evict", `p`) para cada página `p` en caché, cuando hay un miss

TRANSICIÓN $P(s'|s,a)$:

Tras ejecutar la acción `a`, el caché resultante `C'` queda determinado.

El siguiente estado es (C', q) con probabilidad `access_probs[q]` para cada página `q`.

RECOMPENSA:

- +1 si la página solicitada está en caché (hit)
- 1 si la página solicitada no está en caché (miss)

"""

```
import itertools
```

```
from src.mdp import MDP
```

```
class CacheMDP(MDP):
```

```
    """Gestión de caché modelada como un MDP completamente observable.
```

```

Args:
    num_pages:    número total de páginas distintas (N)
    cache_size:   cantidad de páginas que caben en caché (K),  $K < N$ 
    access_probs: diccionario {page_id: probabilidad}, debe sumar 1.0
    gamma:        factor de descuento en  $[0, 1)$ 
"""

def __init__(self, num_pages, cache_size, access_probs, gamma=0.9):
    assert cache_size < num_pages, "El caché debe ser más pequeño que el total de páginas"
    assert abs(sum(access_probs.values()) - 1.0) < 1e-9, "Las probabilidades deben sumar 1"

    self.num_pages = num_pages
    self.cache_size = cache_size
    self.access_probs = access_probs
    self._gamma = gamma
    self.page_ids = sorted(access_probs.keys())

    self._states = None      # se construye de forma lazy
    self._state_index = None # estado -> índice entero (para Policy
Iteration)

# -----
# Interfaz MDP
# -----

def states(self):
    if self._states is None:
        self._build_states()
    return self._states

def actions(self, state):
    """Retorna las acciones disponibles para el estado aumentado dado."""
    cache, requested = state
    if requested in cache:
        return ["noop"]
    return [("evict", p) for p in sorted(cache)]

def get_transitions(self, state, action):
    """Retorna la lista dispersa de (siguiente_estado, prob) con prob > 0.

    Desde cualquier (caché, página_solicitada) tras la acción a:

```

```

        1. Se calcula el caché resultante C' (determinístico dado a)
        2. El siguiente estado es (C', q) para cada página q, con prob
access_probs[q]
        Solo existen N transiciones no nulas (una por cada posible solicitud
futura).
    """
    cache, requested = state
    if self._states is None:
        self._build_states()

    # Determinar el caché resultante después de la acción
    if action == "noop":
        result_cache = cache
    else:
        _, evicted = action
        result_cache = (cache - {evicted}) | {requested}

    # El siguiente estado depende de qué página se solicite a continuación
    transitions = []
    for q in self.page_ids:
        prob = self.access_probs[q]
        if prob > 0:
            next_state = (result_cache, q)
            transitions.append((next_state, prob))
    return transitions

def reward(self, state, action, next_state):
    """R(s, a, s') = +1 en hit, -1 en miss. Depende solo del estado actual."""
    cache, requested = state
    return 1.0 if requested in cache else -1.0

def get_gamma(self):
    return self._gamma

# -----
# Construcción del espacio de estados
# -----

def _build_states(self):
    """Genera todos los estados aumentados (frozenset de tamaño K, page_id).

    Total de estados = C(N, K) * N.
    Con N=5, K=3: C(5,3)*5 = 50 estados.

```



```

Con N=8, K=3:  $C(8,3)*8 = 448$  estados.
"""

all_cache_configs = [
    frozenset(combo)
    for combo in itertools.combinations(self.page_ids, self.cache_size)
]
self._states = [
    (cache, page)
    for cache in all_cache_configs
    for page in self.page_ids
]
self._state_index = {s: i for i, s in enumerate(self._states)}

# -----
# Utilidades auxiliares
# -----

def state_index(self, state):
    """Retorna el índice entero de un estado (usado por el solver lineal de
    Policy Iteration)."""
    if self._state_index is None:
        self._build_states()
    return self._state_index[state]

def num_states(self):
    return len(self.states())

def hit_probability(self, state):
    """Probabilidad de que una solicitud aleatoria resulte en hit para el caché
    actual."""
    cache, _ = state
    return sum(self.access_probs[p] for p in cache)

def describe_state(self, state):
    """Representación legible de un estado."""
    cache, requested = state
    hit = "HIT" if requested in cache else "MISS"
    cache_str = "{" + ", ".join(f"P{p}" for p in sorted(cache)) + "}"
    return f"cache={cache_str}, req=P{requested} [{hit}]"

def describe_action(self, action):
    """Representación legible de una acción."""
    if action == "noop":

```

```

        return "noop (mantener caché)"
    _, evicted = action
    return f"desalojar P{evicted}"

def describe_policy(self, policy):
    """Imprime la política completa como tabla legible."""
    print(f"\n{'ESTADO':<45} {'ACCIÓN'}")
    print("-" * 65)
    for state in self.states():
        act = policy.get(state, "?")
        print(f"{self.describe_state(state):<45} {self.describe_action(act)}")

def verify_transitions(self):
    """Verificación de correctitud: las probabilidades de transición suman 1
    para todo (s,a)."""
    errors = []
    for state in self.states():
        for action in self.actions(state):
            total = sum(p for _, p in self.get_transitions(state, action))
            if abs(total - 1.0) > 1e-9:
                errors.append((state, action, total))
    if errors:
        for s, a, t in errors:
            print(f"ERROR: suma={t:.6f} para estado={s}, acción={a}")
        return False
    return True

```

3. src/value_iteration.py

Python

"""

Algoritmo Value Iteration – R&N Capítulo 16, Figura 16.6.

Resuelve un MDP aplicando iterativamente la actualización de Bellman hasta alcanzar la convergencia:

$$U_{i+1}(s) = \max_a \sum P(s'|s,a) [R(s,a,s') + \gamma U_i(s')]$$

Criterio de parada (Ecuación 16.12):

$$\text{delta} < \text{epsilon} * (1 - \text{gamma}) / \text{gamma}$$

lo que garantiza que el error en la utilidad verdadera es menor que epsilon.

"""

```
import copy
```

```
class ValueIteration:
```

"""Solver de Value Iteration (R&N Sección 16.2.1, Figura 16.6).

Args:

mdp: instancia de MDP (debe implementar la interfaz MDP)

epsilon: error máximo permitido en la utilidad de cualquier estado

"""

```
def __init__(self, mdp, epsilon=0.001):
```

```
    self.mdp = mdp
```

```
    self.epsilon = epsilon
```

```
def solve(self):
```

"""Ejecuta Value Iteration hasta la convergencia.

Returns:

U (dict): estado -> valor de utilidad

policy (dict): estado -> mejor acción

history (list): lista de dicts {estado: utilidad} por iteración,
usado para graficar la convergencia

"""

```
gamma = self.mdp.get_gamma()
```

```
states = self.mdp.states()
```

Inicializar todas las utilidades en cero

```
U = {s: 0.0 for s in states}
```

```
history = []
```

Umbral de convergencia de la Ecuación 16.12

```
threshold = self.epsilon * (1.0 - gamma) / gamma
```

```
while True:
```

```
    U_prev = copy.copy(U)
```

```

        delta = 0.0

    for s in states:
        # Actualización de Bellman:  $U'[s] = \max_a Q(s, a, U_{prev})$ 
        nuevo_val = self.mdp.best_q(s, U_prev)
        U[s] = nuevo_val
        diff = abs(U[s] - U_prev[s])
        if diff > delta:
            delta = diff

    # Guardar snapshot para análisis de convergencia
    history.append(copy.copy(U))

    if delta <= threshold:
        break

    policy = self._extract_policy(U)
    return U, policy, history

def _extract_policy(self, U):
    """Extrae la política greedy a partir de la función de utilidad U.

     $\pi(s) = \operatorname{argmax}_a Q(s, a, U)$ 
    """
    policy = {}
    for s in self.mdp.states():
        policy[s] = self.mdp.best_action(s, U)
    return policy

@staticmethod
def convergence_report(history):
    """Imprime el delta máximo entre snapshots de utilidad sucesivos por
    iteración."""
    print(f"\n{'Iter':>6} {'Max |dU|':>12}")
    print("-" * 22)
    for i, U in enumerate(history):
        if i == 0:
            print(f"{i+1:>6} {'(inicio)':>12}")
            continue
        prev = history[i - 1]
        delta = max(abs(U[s] - prev[s]) for s in U)
        print(f"{i+1:>6} {'delta:>12.8f}")
    print(f"\nConvergió en {len(history)} iteraciones.")

```

4. **Policy_iteration.py:** Implementa el algoritmo y la evaluación exacta de política mediante la solución de un sistema lineal.

Python

"""

Algoritmo Policy Iteration – R&N Capítulo 16, Figura 16.9.

Alterna entre dos pasos hasta que la política se estabiliza:

1. Evaluación de política: calcula $U = U^{\pi}$ resolviendo el sistema lineal
 $(I - \gamma * T_{\pi}) * U = R_{\pi}$ (Ecuación 16.14)
mediante eliminación Gaussiana con pivoteo parcial, implementada desde cero.
2. Mejora de política: para cada estado, elige la acción que maximiza $Q(s,a,U)$.

Termina cuando ningún estado cambia su acción (la política es óptima).

También incluye Modified Policy Iteration (Sección 16.2.2): en lugar de resolver el sistema lineal completo, ejecuta k actualizaciones de Bellman simplificadas bajo la política fija. Es más rápido por iteración pero puede requerir más iteraciones.

"""

`import copy`

```
# =====  
# Eliminación Gaussiana (implementada desde cero, sin numpy)  
# =====
```

```
def gaussian_elimination(A, b):  
    """Resuelve el sistema lineal  $A*x = b$  mediante eliminación Gaussiana con  
    pivoteo parcial.
```

Args:

A: lista de listas (matriz $n \times n$)

b: lista de floats (vector de n elementos)

Returns:

x: lista de floats (vector solución)

Raises:

ValueError: si el sistema es singular o casi singular

"""

n = len(b)

Construir la matriz aumentada [A | b]

M = [A[i][:] + [b[i]] for i in range(n)]

for col in range(n):

Pivoteo parcial: encontrar la fila con el mayor valor absoluto en esta columna

fila_max = col

val_max = abs(M[col][col])

for fila in range(col + 1, n):

if abs(M[fila][col]) > val_max:

val_max = abs(M[fila][col])

fila_max = fila

if val_max < 1e-12:

raise ValueError(f"Matriz singular o casi singular en la columna {col}")

Intercambiar la fila pivote

M[col], M[fila_max] = M[fila_max], M[col]

pivote = M[col][col]

Eliminar por debajo del pivote

for fila in range(col + 1, n):

if M[fila][col] == 0.0:

continue

factor = M[fila][col] / pivote

for j in range(col, n + 1):

M[fila][j] -= factor * M[col][j]

Sustitución hacia atrás

x = [0.0] * n

for fila in range(n - 1, -1, -1):

x[fila] = M[fila][n]

```

        for col in range(fila + 1, n):
            x[fila] -= M[fila][col] * x[col]
        x[fila] /= M[fila][fila]

    return x

# =====
# Policy Iteration
# =====

class PolicyIteration:
    """Solver de Policy Iteration (R&N Sección 16.2.2, Figura 16.9).

    Args:
        mdp: instancia de MDP
        eval_method: 'exact' - resuelve el sistema lineal (Ecuación 16.14)
                    'iterative' - Modified Policy Iteration (k actualizaciones de
Bellman)
        k: número de actualizaciones de Bellman para el método
'iterative'
    """

    def __init__(self, mdp, eval_method='exact', k=20):
        self.mdp = mdp
        self.eval_method = eval_method
        self.k = k

    def solve(self):
        """Ejecuta Policy Iteration hasta que la política se estabiliza.

        Returns:
            U (dict): estado -> valor de utilidad
            policy (dict): estado -> mejor acción
            history (list): lista de (snapshot de política, snapshot de U) por
iteración
        """
        states = self.mdp.states()

        # Inicializar con una política válida (primera acción disponible en cada
estado)
        policy = {s: self.mdp.actions(s)[0] for s in states}
        U = {s: 0.0 for s in states}

```

```

history = []

while True:
    # Paso 1: Evaluación de política
    if self.eval_method == 'exact':
        U = self._policy_evaluation_exact(policy)
    else:
        U = self._policy_evaluation_iterative(policy, U)

    # Paso 2: Mejora de política
    sin_cambios = True
    for s in states:
        mejor_a = self.mdp.best_action(s, U)
        mejor_q = self.mdp.q_value(s, mejor_a, U)
        q_actual = self.mdp.q_value(s, policy[s], U)
        if mejor_q > q_actual + 1e-9:
            policy[s] = mejor_a
            sin_cambios = False

    history.append((copy.copy(policy), copy.copy(U)))

    if sin_cambios:
        break

return U, policy, history

def _policy_evaluation_exact(self, policy):
    """Resuelve el sistema lineal  $(I - \gamma T_{\pi})U = R_{\pi}$ .

    Para cada estado  $s$  (Ecuación 16.14):
        
$$U(s) = \sum_{s'} P(s'|s, \pi(s)) * [R(s, \pi(s), s') + \gamma U(s')]$$


    Reordenando:
        
$$U(s) - \gamma * \sum_{s'} P(s'|s, \pi(s))U(s') = \sum_{s'} P(s'|s, \pi(s))R(s, \pi(s), s')$$


    Es decir:  $(I - \gamma T_{\pi}) * U = r_{\pi}$ 
    """
    states = self.mdp.states()
    n = len(states)
    idx = {s: i for i, s in enumerate(states)}
    gamma = self.mdp.get_gamma()

```



```
        # Construir la matriz del lado izquierdo (I - gamma*T_pi) y el vector del
lado derecho r_pi
```

```
A = [[0.0] * n for _ in range(n)]
b = [0.0] * n
```

```
for i, s in enumerate(states):
    a = policy[s]
    A[i][i] = 1.0 # diagonal de la identidad
    for next_s, prob in self.mdp.get_transitions(s, a):
        r = self.mdp.reward(s, a, next_s)
        j = idx[next_s]
        A[i][j] -= gamma * prob # restar la entrada gamma * T_pi
        b[i] += prob * r        # acumular la recompensa inmediata
```

esperada

```
x = gaussian_elimination(A, b)
return {s: x[i] for i, s in enumerate(states)}
```

```
def _policy_evaluation_iterative(self, policy, U_init):
    """Modified Policy Iteration: k actualizaciones de Bellman bajo política
fija.
```

```
U_{t+1}(s) = sum_{s'} P(s'|s, pi(s)) [R(s, pi(s), s') + gamma*U_t(s')]
"""
```

```
U = copy.copy(U_init)
states = self.mdp.states()
gamma = self.mdp.get_gamma()

for _ in range(self.k):
    U_nuevo = {}
    for s in states:
        a = policy[s]
        val = sum(
            prob * (self.mdp.reward(s, a, ns) + gamma * U.get(ns, 0.0))
            for ns, prob in self.mdp.get_transitions(s, a)
        )
        U_nuevo[s] = val
    U = U_nuevo
```

```
return U
```

```
@staticmethod
```

```
def convergence_report(history):
```

```

"""Imprime el resumen de cambios de política por iteración."""
print(f"\n{'Iter':>6}  {'Cambios de política':>20}")
print("-" * 30)
prev_policy = None
for i, (policy, U) in enumerate(history):
    if prev_policy is None:
        cambios = len(policy)
    else:
        cambios = sum(1 for s in policy if policy[s] != prev_policy[s])
    print(f"{'i+1':>6}  {'cambios':>20}")
    prev_policy = policy
print(f"\nConvergió en {len(history)} iteraciones.")

```

5. [src/ucb.py](#)

Implementa el algoritmo **UCB1** (Upper Confidence Bound) de la Sección 16.3 como agente de aprendizaje online. No requiere conocer la distribución de acceso a páginas: aprende de la experiencia.

Python

"""

Agente UCB1 (Upper Confidence Bound) para gestión de caché online – R&N Sección 16.3.

Modela cada decisión de desalojo como el brazo de un bandido multi-brazo. No requiere conocer la distribución de acceso a páginas con antelación; aprende de la experiencia a través de la exploración y explotación.

Índice UCB para el par (estado, acción):

$$UCB(s, a) = Q(s, a) + c * \sqrt{\ln(N_s) / N_{sa}}$$

donde:

$Q(s, a)$ = recompensa promedio observada cuando se eligió (s, a)

N_s = número total de veces que se visitó el estado s

N_{sa} = número de veces que se tomó la acción a en el estado s

c = parámetro de exploración (mayor valor = más exploración)

En estados de hit la única acción es 'noop' (sin decisión que tomar), por lo que UCB solo aplica cuando hay un miss con caché llena.

"""

```
import math
import random
```

```
class UCBAgent:
```

```
    """Agente online UCB1 para decisiones de desalojo en caché.
```

```
    Args:
```

```
        cache_size: K (número de páginas que caben en caché)
        num_pages: N (total de páginas distintas)
        c:          coeficiente de exploración (sqrt(2) es el estándar de UCB1)
        seed:       semilla aleatoria para desempate
```

```
    """
```

```
    def __init__(self, cache_size, num_pages, c=1.414, seed=None):
```

```
        self.cache_size = cache_size
        self.num_pages = num_pages
        self.c = c
        self._rng = random.Random(seed)
```

```
        # Q[clave_estado][clave_accion] = recompensa promedio
```

```
        self._Q = {}
```

```
        # N_sa[clave_estado][clave_accion] = contador de visitas
```

```
        self._N_sa = {}
```

```
        # N_s[clave_estado] = total de visitas al estado
```

```
        self._N_s = {}
```

```
    # -----
    # Helpers para claves de estado y acción
    # -----
```

```
    def _skey(self, cache, requested_page):
```

```
        """Clave hasheable para un estado (frozenset del caché, página solicitada)."""
```

```
        return (frozenset(cache), requested_page)
```

```
    def _akey(self, action):
```

```
        return action # ya es hasheable ("noop" o ("evict", p))
```

```

# -----
# Interfaz principal del agente
# -----

def select_action(self, cache, requested_page):
    """Elige una acción de desalojo usando UCB1.

    Si la página solicitada ya está en caché, retorna "noop" directamente.
    En caso contrario, aplica UCB1 para elegir qué página desalojar.
    """
    if requested_page in cache:
        return "noop"

    acciones_disponibles = [("evict", p) for p in sorted(cache)]
    skey = self._skey(cache, requested_page)
    N_s = self._N_s.get(skey, 0)

    # Siempre explorar primero las acciones no visitadas
    no_visitadas = [
        a for a in acciones_disponibles
        if self._N_sa.get(skey, {}).get(self._akey(a), 0) == 0
    ]
    if no_visitadas:
        return self._rng.choice(no_visitadas)

    # UCB1:  $\arg\max Q(s,a) + c * \sqrt{\ln(N_s) / N_{sa}}$ 
    mejor_accion = None
    mejor_puntaje = float("-inf")
    for a in acciones_disponibles:
        akey = self._akey(a)
        n_sa = self._N_sa[skey][akey]
        q = self._Q[skey][akey]
        bonus = self.c * math.sqrt(math.log(N_s) / n_sa)
        puntaje = q + bonus
        if puntaje > mejor_puntaje:
            mejor_puntaje = puntaje
            mejor_accion = a

    return mejor_accion

def update(self, cache, requested_page, action, reward):
    """Actualiza el Q-valor y los contadores tras observar una recompensa.

```

```

Usa la actualización incremental de la media:
     $Q(s,a) \leftarrow Q(s,a) + (recompensa - Q(s,a)) / N_{sa}$ 
    """
    skey = self._skey(cache, requested_page)
    akey = self._akey(action)

    if skey not in self._Q:
        self._Q[skey] = {}
        self._N_sa[skey] = {}
        self._N_s[skey] = 0

    if akey not in self._Q[skey]:
        self._Q[skey][akey] = 0.0
        self._N_sa[skey][akey] = 0

    self._N_s[skey] += 1
    self._N_sa[skey][akey] += 1
    n = self._N_sa[skey][akey]
    self._Q[skey][akey] += (reward - self._Q[skey][akey]) / n

def reset(self):
    """Borra todas las estadísticas aprendidas."""
    self._Q = {}
    self._N_sa = {}
    self._N_s = {}

def get_learned_policy(self):
    """Extrae la política greedy a partir de los Q-valores aprendidos.

    Retorna dict {(frozenset_cache, pagina_solicitada): mejor_accion}.
    Solo incluye estados que han sido visitados al menos una vez.
    """
    policy = {}
    for skey, actions in self._Q.items():
        if not actions:
            continue
        mejor_a = max(actions, key=lambda a: actions[a])
        policy[skey] = mejor_a
    return policy

def stats_summary(self):
    """Imprime cuántos pares distintos (estado, acción) han sido explorados."""

```

```
total_sa = sum(len(v) for v in self._N_sa.values())
print(f"Estadísticas UCB: {len(self._N_s)} estados visitados, "
      f"{total_sa} pares (estado, acción) explorados.")
```

6. src/[baselines.py](#): Políticas clásicas de remplazo

Python

"""

Políticas clásicas de reemplazo de caché usadas como baseline de comparación.

Todas las políticas implementan:

decide_eviction(cache: frozenset, requested_page: int) -> página_a_desalojar

- LRUPolicy: Least Recently Used (menos recientemente usado)
- FIFOPolicy: First In, First Out (primero en entrar, primero en salir)
- RandomPolicy: Desalojar una página al azar
- OptimalOfflinePolicy: Algoritmo de Belady (requiere conocer el futuro)

"""

import random

class LRUPolicy:

"""LRU – desaloja la página que fue accedida hace más tiempo.

Mantiene una lista ordenada de páginas por recencia de acceso.

"""

def __init__(self, seed=None):

self._access_order = [] # la más reciente al final

self._rng = random.Random(seed)

def on_access(self, page):

"""Registra que 'page' acaba de ser accedida (llamar en cada hit Y miss)."""

if page in self._access_order:

```

        self._access_order.remove(page)
        self._access_order.append(page)

    def decide_eviction(self, cache, requested_page):
        """Desaloja la página en caché que fue accedida hace más tiempo."""
        for page in self._access_order:
            if page in cache:
                return page
        # Fallback: desalojar cualquier página (no debería ocurrir en operación
normal)
        return next(iter(cache))

    def reset(self):
        self._access_order = []

    def name(self):
        return "LRU"

class FIFOPolicy:
    """FIFO - desaloja la página que lleva más tiempo en caché."""

    def __init__(self, seed=None):
        self._insertion_order = [] # la más antigua al frente

    def on_load(self, page):
        """Registra que 'page' acaba de ser cargada en caché."""
        if page not in self._insertion_order:
            self._insertion_order.append(page)

    def decide_eviction(self, cache, requested_page):
        """Desaloja la página en caché que fue cargada primero."""
        for page in self._insertion_order:
            if page in cache:
                return page
        return next(iter(cache))

    def reset(self):
        self._insertion_order = []

    def name(self):
        return "FIFO"

```

```

class RandomPolicy:
    """Random – desaloja una página uniformemente aleatoria del caché."""

    def __init__(self, seed=None):
        self._rng = random.Random(seed)

    def decide_eviction(self, cache, requested_page):
        return self._rng.choice(sorted(cache))

    def reset(self):
        pass

    def name(self):
        return "Random"

class OptimalOfflinePolicy:
    """Algoritmo óptimo offline de Belady: desaloja la página cuyo próximo uso es
    más lejano.

    Requiere conocer la secuencia completa de accesos de antemano (no es realista),
    pero provee una cota superior teórica para la comparación.
    """

    def __init__(self, access_sequence):
        self._sequence = access_sequence
        self._current_step = 0

    def decide_eviction(self, cache, requested_page):
        """Desaloja la página en caché cuyo próximo acceso está más lejos en el
        futuro."""
        futuro = self._sequence[self._current_step:]
        pagina_mas_lejana = None
        distancia_max = -1

        for page in cache:
            # Buscar el próximo acceso de esta página en la secuencia futura
            try:
                dist = futuro.index(page)
            except ValueError:
                # La página nunca más será accedida: desalojar de inmediato
                return page

```



```

        if dist > distancia_max:
            distancia_max = dist
            pagina_mas_lejana = page

    return pagina_mas_lejana

def on_step(self):
    self._current_step += 1

def reset(self):
    self._current_step = 0

def name(self):
    return "Óptimo (Belady)"

```

7. [src/scenarios.py](#): Distribución de acceso a páginas

Python

"""

Distribuciones de probabilidad de acceso a páginas para los experimentos de simulación.

Todas las distribuciones implementan:

```

get_probs() -> dict {page_id: probabilidad} (suma 1.0)
sample()    -> page_id                      (genera una solicitud)
name()      -> str

```

"""

```
import random
```

```
import math
```

```
class AccessDistribution:
```

```
    """Clase base para patrones de acceso a páginas."""
```

```
    def __init__(self, num_pages, seed=None):
```

```

        self.num_pages = num_pages
        self.page_ids = list(range(num_pages))
        self._rng = random.Random(seed)
        self._probs = None # calculado por la subclase

    def get_probs(self):
        """Retorna diccionario {page_id: probabilidad}."""
        raise NotImplementedError

    def sample(self):
        """Genera una solicitud de página según la distribución."""
        probs = self.get_probs()
        r = self._rng.random()
        cumulative = 0.0
        for page, p in sorted(probs.items()):
            cumulative += p
            if r <= cumulative:
                return page
        return self.page_ids[-1] # fallback numérico

    def name(self):
        raise NotImplementedError

    def __repr__(self):
        return self.name()

# -----
# Distribución uniforme
# -----

class UniformAccess(AccessDistribution):
    """Todas las páginas con la misma probabilidad: prob = 1/N."""

    def get_probs(self):
        if self._probs is None:
            p = 1.0 / self.num_pages
            self._probs = {pg: p for pg in self.page_ids}
        return self._probs

    def name(self):
        return f"Uniforme(N={self.num_pages})"

```

```

# -----
# Distribución Zipf
# -----

class ZipfAccess(AccessDistribution):
    """Ley de Zipf: prob(rango r) proporcional a 1/r^alpha.

    Las páginas se ordenan de mayor a menor popularidad (page_id=0 es la más
    popular).
    alpha=1.0 es la Zipf clásica. Mayor alpha implica mayor sesgo.
    """

    def __init__(self, num_pages, alpha=1.0, seed=None):
        super().__init__(num_pages, seed)
        self.alpha = alpha

    def get_probs(self):
        if self._probs is None:
            rangos = [i + 1 for i in range(self.num_pages)] # rangos 1..N
            pesos = [1.0 / (r ** self.alpha) for r in rangos]
            total = sum(pesos)
            self._probs = {
                pg: pesos[pg] / total
                for pg in self.page_ids
            }
        return self._probs

    def name(self):
        return f"Zipf(N={self.num_pages}, alpha={self.alpha})"

# -----
# Distribución de localidad temporal
# -----

class TemporalLocalityAccess(AccessDistribution):
    """Modelo de conjunto de trabajo: un subconjunto de páginas 'calientes'
    concentra el tráfico.

    Args:
        num_hot_pages: número de páginas en el conjunto de trabajo activo
        hot_prob: masa de probabilidad total asignada a las páginas calientes

```

```

"""

def __init__(self, num_pages, num_hot_pages=None, hot_prob=0.8, seed=None):
    super().__init__(num_pages, seed)
    self.num_hot = num_hot_pages if num_hot_pages is not None else max(1,
num_pages // 3)
    self.hot_prob = hot_prob
    assert self.num_hot < num_pages, "num_hot_pages debe ser < num_pages"
    assert 0 < hot_prob < 1, "hot_prob debe estar en (0,1)"

def get_probs(self):
    if self._probs is None:
        hot_pages = self.page_ids[: self.num_hot]
        cold_pages = self.page_ids[self.num_hot :]

        hot_each = self.hot_prob / self.num_hot
        cold_each = (1.0 - self.hot_prob) / len(cold_pages) if cold_pages else
0.0

        self._probs = {}
        for pg in hot_pages:
            self._probs[pg] = hot_each
        for pg in cold_pages:
            self._probs[pg] = cold_each
    return self._probs

def name(self):
    return f"LocalidadTemporal(N={self.num_pages}, hot={self.num_hot},
p={self.hot_prob})"

# -----
# Distribución no estacionaria (cambia en un paso dado)
# -----

class NonStationaryAccess:
    """Distribución que cambia en un paso de simulación especificado.

    Útil para demostrar que UCB puede adaptarse mientras que una política
    MDP fija no puede responder al cambio.
    """

    def __init__(self, dist1, dist2, switch_step, seed=None):

```

```

        self.dist1 = dist1
        self.dist2 = dist2
        self.switch_step = switch_step
        self._rng = random.Random(seed)
        self._step = 0

    def get_probs(self, step=None):
        s = step if step is not None else self._step
        return self.dist1.get_probs() if s < self.switch_step else
self.dist2.get_probs()

    def sample(self):
        probs = self.get_probs(self._step)
        self._step += 1
        r = self._rng.random()
        cumulative = 0.0
        for page, p in sorted(probs.items()):
            cumulative += p
            if r <= cumulative:
                return page
        return sorted(probs.keys())[-1]

    def reset(self):
        self._step = 0

    def name(self):
        return (f"NoEstacionaria(cambio@{self.switch_step}: "
                f"{self.dist1.name()} -> {self.dist2.name()}")

    def __repr__(self):
        return self.name()

# -----
# Función auxiliar para imprimir un resumen de la distribución
# -----

def print_distribution(dist):
    probs = dist.get_probs()
    print(f"\nDistribución: {dist.name()}")
    print(f"{'Página':<8} {'Prob':>10}")
    print("-" * 20)
    for pg in sorted(probs.keys()):

```

```
barra = "#" * int(probs[pg] * 40)
print(f"P{pg:<7} {probs[pg]:>10.4f} {barra}")
```

8. src/[visualizer.py](#): Generación de graficas

Python

```
"""
```

Módulo de visualización - todas las gráficas del proyecto usando matplotlib.

Requiere matplotlib (solicitado como librería adicional; únicamente grafica, no resuelve el MDP ni ningún algoritmo del proyecto).

```
"""
```

```
import os
```

```
try:
```

```
    import matplotlib
```

```
    matplotlib.use("Agg")    # backend no interactivo para guardar PNGs
```

```
    import matplotlib.pyplot as plt
```

```
    MATPLOTLIB_AVAILABLE = True
```

```
except ImportError:
```

```
    MATPLOTLIB_AVAILABLE = False
```

```
    print("ADVERTENCIA: matplotlib no está instalado. Las gráficas serán  
omitidas.")
```

```
DIRECTORIO_FIGURAS = os.path.join(os.path.dirname(__file__), "..", "figures")
```

```
def _asegurar_directorio():
```

```
    os.makedirs(DIRECTORIO_FIGURAS, exist_ok=True)
```

```
def _guardar(fig, filename):
```

```
    _asegurar_directorio()
```

```
    path = os.path.join(DIRECTORIO_FIGURAS, filename)
```

```
    fig.savefig(path, dpi=150, bbox_inches="tight")
```

```

plt.close(fig)
print(f" Guardado: figures/{filename}")

# -----
# 1. Curvas de convergencia de Value Iteration
# -----

def plot_vi_convergence(history, title="Convergencia de Value Iteration",
                        filename="vi_convergencia.png", sample_states=None):
    """Grafica los estimados de utilidad por iteración para estados seleccionados.

    Estilo similar a la Figura 16.7(a) del libro.

    Args:
        history: lista de dicts {estado: utilidad} por iteración
        title: título de la gráfica
        filename: nombre del archivo PNG de salida
        sample_states: lista de estados a graficar (None = seleccionar hasta 6
    automáticamente)
    """
    if not MATPLOTLIB_AVAILABLE:
        return

    all_states = list(history[0].keys())

    if sample_states is None:
        # Seleccionar una muestra diversa: hasta 6 estados
        step = max(1, len(all_states) // 6)
        sample_states = all_states[::step][:6]

    fig, ax = plt.subplots(figsize=(9, 5))
    iters = list(range(1, len(history) + 1))

    for state in sample_states:
        values = [h[state] for h in history]
        label = _etiqueta_estado(state)
        ax.plot(iters, values, label=label, linewidth=1.5)

    ax.set_xlabel("Iteración")
    ax.set_ylabel("Estimado de utilidad U(s)")
    ax.set_title(title)
    ax.legend(loc="lower right", fontsize=8)

```

```

ax.grid(True, alpha=0.3)
_guardar(fig, filename)

# -----
# 2. Delta máximo por iteración
# -----

def plot_delta_convergence(history, title="Delta máximo por iteración de Bellman",
                           filename="vi_delta.png"):
    """Grafica el cambio máximo en la utilidad por iteración (escala
    logarítmica)."""
    if not MATPLOTLIB_AVAILABLE:
        return

    deltas = []
    for i in range(1, len(history)):
        delta = max(abs(history[i][s] - history[i - 1][s]) for s in history[i])
        deltas.append(delta)

    fig, ax = plt.subplots(figsize=(8, 4))
    ax.semilogy(range(2, len(history) + 1), deltas, color="steelblue", linewidth=2)
    ax.set_xlabel("Iteración")
    ax.set_ylabel("max |Ui(s) - U{i-1}(s)| (escala log)")
    ax.set_title(title)
    ax.grid(True, which="both", alpha=0.3)
    _guardar(fig, filename)

# -----
# 3. Comparación de hit rate entre políticas
# -----

def plot_hit_rate_comparison(results_by_scenario,
                             filename="comparacion_hit_rate.png"):
    """Gráfica de barras agrupadas: hit rate por escenario y política.

    Args:
        results_by_scenario: dict {nombre_escenario: [SimResult, ...]}
    """
    if not MATPLOTLIB_AVAILABLE:
        return

```



```

    escenarios = list(results_by_scenario.keys())
    # Obtener todos los nombres de política (preservando orden del primer
escenario)
    policy_names = [r.policy_name for r in
next(iter(results_by_scenario.values()))]

    n_policies = len(policy_names)
    x = list(range(len(scenarios)))
    bar_width = 0.8 / n_policies

    colors = plt.cm.tab10.colors

    fig, ax = plt.subplots(figsize=(10, 5))

    for i, policy in enumerate(policy_names):
        hit_rates = []
        for scenario in escenarios:
            results = results_by_scenario[scenario]
            match = next((r for r in results if r.policy_name == policy), None)
            hit_rates.append(match.hit_rate if match else 0.0)

        offsets = [xi + (i - n_policies / 2 + 0.5) * bar_width for xi in x]
        ax.bar(offsets, hit_rates, width=bar_width,
            label=policy, color=colors[i % len(colors)], edgecolor="white")

    ax.set_xticks(x)
    ax.set_xticklabels(scenarios, fontsize=10)
    ax.set_ylabel("Hit Rate")
    ax.set_title("Tasa de Aciertos: MDP vs Políticas Baseline")
    ax.legend(loc="upper right", fontsize=9)
    ax.set_ylim(0, 1.0)
    ax.grid(True, axis="y", alpha=0.3)
    _guardar(fig, filename)

# -----
# 4. Recompensa acumulada a lo largo del tiempo
# -----

def plot_cumulative_rewards(results, title="Recompensa Acumulada",
                           filename="recompensa_acumulada.png"):
    """Gráfica de líneas de la recompensa acumulada por política a lo largo de los
pasos."""

```

```

if not MATPLOTLIB_AVAILABLE:
    return

fig, ax = plt.subplots(figsize=(10, 5))
colors = plt.cm.tab10.colors

for i, r in enumerate(results):
    ax.plot(r.cumulative_rewards, label=r.policy_name,
            color=colors[i % len(colors)], linewidth=1.5)

ax.set_xlabel("Paso de simulación")
ax.set_ylabel("Recompensa acumulada")
ax.set_title(title)
ax.legend(loc="upper left", fontsize=9)
ax.grid(True, alpha=0.3)
_guardar(fig, filename)

# -----
# 5. Heatmap de utilidades de estados
# -----

def plot_utility_heatmap(U, mdp, filename="heatmap_utilidades.png"):
    """Heatmap de utilidades agrupadas por configuración de caché.

    Filas = configuraciones de caché (ordenadas por utilidad promedio).
    Columnas = páginas solicitadas.
    """
    if not MATPLOTLIB_AVAILABLE:
        return

    states = mdp.states()
    cache_configs = sorted(set(c for c, _ in states))
    pages = sorted(mdp.access_probs.keys())

    # Construir matriz: filas=config_cache, columnas=pagina_solicitada
    matrix = []
    row_labels = []
    for cfg in cache_configs:
        row = [U.get((cfg, p), 0.0) for p in pages]
        matrix.append(row)
        row_labels.append("{ " + ", ".join(f"P{p}" for p in sorted(cfg)) + " }")

```

```

# Ordenar filas por utilidad promedio (descendente)
row_means = [sum(row) / len(row) for row in matrix]
order = sorted(range(len(matrix)), key=lambda i: -row_means[i])
matrix = [matrix[i] for i in order]
row_labels = [row_labels[i] for i in order]

fig, ax = plt.subplots(figsize=(max(6, len(pages) * 1.2),
                                max(4, len(cache_configs) * 0.5)))
im = ax.imshow(matrix, cmap="RdYlGn", aspect="auto")

ax.set_xticks(range(len(pages)))
ax.set_xticklabels([f"P{p}" for p in pages])
ax.set_yticks(range(len(row_labels)))
ax.set_yticklabels(row_labels, fontsize=8)
ax.set_xlabel("Página solicitada")
ax.set_ylabel("Configuración del caché")
ax.set_title("Utilidades de los estados U(s)")

plt.colorbar(im, ax=ax, shrink=0.8)
guardar(fig, filename)

# -----
# 6. Sensibilidad al factor de descuento gamma
# -----

def plot_gamma_sensitivity(gamma_values, hit_rates_by_gamma,
                           filename="sensibilidad_gamma.png"):
    """Grafica el hit rate de la política MDP en función del factor de descuento
    gamma."""
    if not MATPLOTLIB_AVAILABLE:
        return

    fig, ax = plt.subplots(figsize=(7, 4))
    ax.plot(gamma_values, hit_rates_by_gamma, "o-", color="steelblue",
            linewidth=2, markersize=6)
    ax.set_xlabel("Factor de descuento gamma")
    ax.set_ylabel("Hit rate (política MDP)")
    ax.set_title("Sensibilidad al Factor de Descuento gamma")
    ax.grid(True, alpha=0.3)
    ax.set_ylim(0, 1.0)
    guardar(fig, filename)

```

```

# -----
# 7. Comparación de convergencia VI vs PI
# -----

def plot_vi_vs_pi(vi_history, pi_history, filename="vi_vs_pi.png"):
    """Compara la convergencia de Value Iteration y Policy Iteration."""
    if not MATPLOTLIB_AVAILABLE:
        return

    fig, axes = plt.subplots(1, 2, figsize=(11, 4))

    # Izquierda: delta de VI por iteración
    vi_deltas = [
        max(abs(vi_history[i][s] - vi_history[i - 1][s]) for s in vi_history[i])
        for i in range(1, len(vi_history))
    ]
    axes[0].semilogy(range(2, len(vi_history) + 1), vi_deltas,
                     color="steelblue", linewidth=2, label="Value Iteration")
    axes[0].set_xlabel("Iteración")
    axes[0].set_ylabel("max |ΔU| (escala log)")
    axes[0].set_title("Convergencia de Value Iteration")
    axes[0].legend()
    axes[0].grid(True, which="both", alpha=0.3)

    # Derecha: cambios de política en PI por iteración
    pi_changes = []
    prev_policy = None
    for policy, U in pi_history:
        if prev_policy is None:
            pi_changes.append(len(policy))
        else:
            pi_changes.append(sum(1 for s in policy if policy[s] !=
prev_policy[s]))
            prev_policy = policy

    axes[1].bar(range(1, len(pi_changes) + 1), pi_changes,
                color="coral", edgecolor="white", label="Policy Iteration")
    axes[1].set_xlabel("Iteración")
    axes[1].set_ylabel("Estados con cambio de política")
    axes[1].set_title("Convergencia de Policy Iteration")
    axes[1].legend()
    axes[1].grid(True, axis="y", alpha=0.3)

```

```

fig.suptitle("Value Iteration vs Policy Iteration", fontsize=13)
plt.tight_layout()
_guardar(fig, filename)

# -----
# 8. Entorno no estacionario: UCB vs MDP
# -----

def plot_nonstationary(ucb_result, mdp_result, switch_step,
                      filename="no_estacionario.png"):
    """Muestra la recompensa acumulada de UCB vs MDP cuando la distribución
    cambia."""
    if not MATPLOTLIB_AVAILABLE:
        return

    fig, ax = plt.subplots(figsize=(10, 5))
    ax.plot(ucb_result.cumulative_rewards, label="UCB (se adapta)",
            color="steelblue", linewidth=1.5)
    ax.plot(mdp_result.cumulative_rewards, label="MDP (política fija)",
            color="coral", linewidth=1.5)
    ax.axvline(x=switch_step, color="gray", linestyle="--", linewidth=1.5,
              label=f"Cambio de distribución (paso {switch_step})")
    ax.set_xlabel("Paso de simulación")
    ax.set_ylabel("Recompensa acumulada")
    ax.set_title("Entorno No Estacionario: UCB vs MDP")
    ax.legend()
    ax.grid(True, alpha=0.3)
    _guardar(fig, filename)

# -----
# Función auxiliar
# -----

def _etiqueta_estado(state):
    cache, page = state
    return "{" + ",".join(f"P{p}" for p in sorted(cache)) + f"}", req=P{page}"

```

9. [main.py](#): Ejecutor de experimentos

Python

```
"""
```

Ejecutor principal de experimentos – Proyecto MDP Gestión de Caché.
R&N Capítulo 16: Making Complex Decisions.

Uso:

```
python main.py          # ejecutar todos los experimentos
python main.py --quick   # configuración pequeña para pruebas rápidas
python main.py --verify  # solo verificaciones de correctitud (sin
```

simulación)

```
"""
```

```
import sys
import time
```

```
from src.cache_mdp import CacheMDP
from src.scenarios import (UniformAccess, ZipfAccess,
                           TemporalLocalityAccess, NonStationaryAccess,
                           print_distribution)
from src.value_iteration import ValueIteration
from src.policy_iteration import PolicyIteration
from src.simulator import Simulator
from src.ucb import UCBAgent
import src.visualizer as viz
```

```
# =====
# Configuración
# =====
```

```
QUICK_MODE   = "--quick" in sys.argv
VERIFY_ONLY  = "--verify" in sys.argv
```

```
NUM_PAGES    = 4 if QUICK_MODE else 5 # número total de páginas N
CACHE_SIZE   = 2 if QUICK_MODE else 3 # capacidad del caché K
GAMMA        = 0.9                     # factor de descuento
EPSILON      = 0.001                   # tolerancia de convergencia para VI
SIM_STEPS    = 2000 if QUICK_MODE else 10000 # pasos de simulación
SEED         = 42                      # semilla para reproducibilidad
```

```

# =====
# Funciones auxiliares
# =====

def seccion(title):
    print("\n" + "=" * 60)
    print(f"  {title}")
    print("=" * 60)

def build_mdp(dist, gamma=GAMMA):
    """Construye el MDP de caché para la distribución de acceso dada."""
    probs = dist.get_probs()
    return CacheMDP(num_pages=NUM_PAGES, cache_size=CACHE_SIZE,
                    access_probs=probs, gamma=gamma)

def solve_vi(mdp, label=""):
    """Ejecuta Value Iteration y reporta tiempo e iteraciones."""
    t0 = time.time()
    U, policy, history = ValueIteration(mdp, epsilon=EPSILON).solve()
    elapsed = time.time() - t0
    print(f"  VI {label}: {len(history)} iteraciones, {elapsed:.3f}s")
    return U, policy, history

def solve_pi(mdp, label=""):
    """Ejecuta Policy Iteration y reporta tiempo e iteraciones."""
    t0 = time.time()
    U, policy, history = PolicyIteration(mdp, eval_method='exact').solve()
    elapsed = time.time() - t0
    print(f"  PI {label}: {len(history)} iteraciones, {elapsed:.3f}s")
    return U, policy, history

# =====
# Experimento 0: Verificaciones de correctitud
# =====

def experiment_verify():
    seccion("EXPERIMENTO 0: Verificaciones de Correctitud")

```

```

dist = ZipfAccess(num_pages=NUM_PAGES, alpha=1.0, seed=SEED)
mdp = build_mdp(dist)

print(f"\n Configuración: N={NUM_PAGES}, K={CACHE_SIZE}, gamma={GAMMA}")
print(f" Total de estados: {mdp.num_states()}")

print("\n Verificando que las probabilidades de transición suman 1 para todo
(s,a)...")
ok = mdp.verify_transitions()
print(f" Resultado: {'CORRECTO' if ok else 'ERROR'}")

print("\n Muestra de estados:")
for state in mdp.states()[:5]:
    print(f" {mdp.describe_state(state)}")
    for action in mdp.actions(state):
        transitions = mdp.get_transitions(state, action)
        total_p = sum(p for _, p in transitions)
        print(f" acción={mdp.describe_action(action)}, "
              f"transiciones={len(transitions)}, suma_p={total_p:.6f}")

print("\n Verificando que VI y PI producen la misma política óptima...")
U_vi, pol_vi, _ = solve_vi(mdp, "verificar")
U_pi, pol_pi, _ = solve_pi(mdp, "verificar")

discrepancias = [s for s in mdp.states() if pol_vi[s] != pol_pi[s]]
print(f" Discrepancias en política: {len(discrepancias)} /
{mdp.num_states()}")
if discrepancias:
    print(" ADVERTENCIA: las políticas difieren. Revisar la implementación.")
else:
    print(" VI y PI coinciden en la política óptima. CORRECTO")

# =====
# Experimento 1: Convergencia de Value Iteration por escenario
# =====

def experiment_vi_convergence():
    seccion("EXPERIMENTO 1: Convergencia de Value Iteration")

    distributions = [
        UniformAccess(num_pages=NUM_PAGES, seed=SEED),

```



```

ZipfAccess(num_pages=NUM_PAGES, alpha=1.0, seed=SEED),
TemporalLocalityAccess(num_pages=NUM_PAGES, num_hot_pages=CACHE_SIZE,
                        hot_prob=0.8, seed=SEED),
]

for dist in distributions:
    print(f"\n Distribución: {dist.name()}")
    print_distribution(dist)
    mdp = build_mdp(dist)
    U, policy, history = solve_vi(mdp, dist.name())

    # Imprimir política óptima para estados de miss de muestra
    miss_states = [(c, p) for c, p in mdp.states() if p not in c][:6]
    print("\n Política óptima (estados de miss):")
    for state in miss_states:
        print(f"    {mdp.describe_state(state):<45} -> "
              f"{mdp.describe_action(policy[state])}")

    # Generar gráficas
    viz.plot_vi_convergence(
        history,
        title=f"Convergencia VI - {dist.name()}",
        filename=f"vi_convergencia_{dist.__class__.__name__}.png",
    )
    viz.plot_delta_convergence(
        history,
        title=f"Delta por Iteración - {dist.name()}",
        filename=f"vi_delta_{dist.__class__.__name__}.png",
    )
    viz.plot_utility_heatmap(U, mdp,

filename=f"heatmap_utilidades_{dist.__class__.__name__}.png")

# =====
# Experimento 2: Value Iteration vs Policy Iteration
# =====

def experiment_vi_vs_pi():
    seccion("EXPERIMENTO 2: Value Iteration vs Policy Iteration")

    dist = ZipfAccess(num_pages=NUM_PAGES, alpha=1.0, seed=SEED)
    mdp = build_mdp(dist)

```

```

print(f"\n Distribución: {dist.name()}")
U_vi, pol_vi, vi_history = solve_vi(mdp, "Zipf")
U_pi, pol_pi, pi_history = solve_pi(mdp, "Zipf")

ValueIteration.convergence_report(vi_history)
PolicyIteration.convergence_report(pi_history)

viz.plot_vi_vs_pi(vi_history, pi_history, filename="vi_vs_pi.png")

# =====
# Experimento 3: Comparación de políticas en simulación
# =====

def experiment_policy_comparison():
    seccion("EXPERIMENTO 3: Comparación de Políticas (MDP vs Baselines)")

    distributions = [
        ("Uniforme", UniformAccess(num_pages=NUM_PAGES, seed=SEED)),
        ("Zipf", ZipfAccess(num_pages=NUM_PAGES, alpha=1.0, seed=SEED)),
        ("Temporal", TemporalLocalityAccess(num_pages=NUM_PAGES,
                                             num_hot_pages=CACHE_SIZE,
                                             hot_prob=0.8, seed=SEED)),
    ]

    results_by_scenario = {}

    for name, dist in distributions:
        print(f"\n Escenario: {name}")
        mdp = build_mdp(dist)

        _, pol_vi, _ = solve_vi(mdp, name)
        _, pol_pi, _ = solve_pi(mdp, name)

        ucb = UCBAgent(cache_size=CACHE_SIZE, num_pages=NUM_PAGES,
                        c=1.414, seed=SEED)
        sim = Simulator(NUM_PAGES, CACHE_SIZE, dist, seed=SEED)

        results = sim.run_all(
            mdp_vi_policy=pol_vi,
            mdp_pi_policy=pol_pi,
            ucb_agent=ucb,

```

```

        num_steps=SIM_STEPS,
        include_optimal=True,
    )
    results_by_scenario[name] = results
    Simulator.print_results(results)

viz.plot_hit_rate_comparison(results_by_scenario,
                             filename="comparacion_hit_rate.png")

# Recompensa acumulada para el escenario Zipf (el más ilustrativo)
zipf_results = results_by_scenario["Zipf"]
viz.plot_cumulative_rewards(zipf_results,
                             title="Recompensa Acumulada - Distribución Zipf",
                             filename="recompensa_acumulada_zipf.png")

# =====
# Experimento 4: Sensibilidad al factor de descuento gamma
# =====

def experiment_gamma_sensitivity():
    seccion("EXPERIMENTO 4: Análisis de Sensibilidad a Gamma")

    dist = ZipfAccess(num_pages=NUM_PAGES, alpha=1.0, seed=SEED)
    gammas = [0.1, 0.3, 0.5, 0.7, 0.9, 0.95, 0.99]
    hit_rates = []

    for gamma in gammas:
        mdp = build_mdp(dist, gamma=gamma)
        _, policy, _ = solve_vi(mdp, f"gamma={gamma}")
        sim = Simulator(NUM_PAGES, CACHE_SIZE, dist, seed=SEED)
        result = sim.simulate_mdp(policy, num_steps=SIM_STEPS,
        policy_name="MDP-VI")
        hit_rates.append(result.hit_rate)
        print(f"  gamma={gamma:.2f}  hit_rate={result.hit_rate:.4f}")

    viz.plot_gamma_sensitivity(gammas, hit_rates,
    filename="sensibilidad_gamma.png")

# =====
# Experimento 5: Escalabilidad del espacio de estados
# =====

```

```

def experiment_scalability():
    seccion("EXPERIMENTO 5: Escalabilidad (Tamaño del Espacio de Estados vs
    Tiempo)")

    configs = [(4, 2), (5, 3), (6, 3), (7, 3), (8, 3)]
    print(f"\n  {'N':>4}  {'K':>4}  {'|S|':>8}  {'Iter VI':>10}  {'Tiempo
    VI(s)':>13}  {'Iter PI':>10}  {'Tiempo PI(s)':>13}")
    print("  " + "-" * 72)

    for n, k in configs:
        dist = ZipfAccess(num_pages=n, alpha=1.0, seed=SEED)
        probs = dist.get_probs()
        mdp = CacheMDP(num_pages=n, cache_size=k, access_probs=probs, gamma=GAMMA)
        n_states = mdp.num_states()

        t0 = time.time()
        _, _, vi_hist = ValueIteration(mdp, epsilon=EPSILON).solve()
        vi_time = time.time() - t0

        t0 = time.time()
        _, _, pi_hist = PolicyIteration(mdp, eval_method='exact').solve()
        pi_time = time.time() - t0

        print(f"  {'n':>4}  {'k':>4}  {'n_states':>8}  "
              f"{'len(vi_hist)':>10}  {'vi_time':>13.3f}  "
              f"{'len(pi_hist)':>10}  {'pi_time':>13.3f}")

# =====
# Experimento 6: Entorno no estacionario (UCB vs MDP)
# =====

def experiment_nonstationary():
    seccion("EXPERIMENTO 6: Entorno No Estacionario")

    SWITCH = SIM_STEPS // 2

    dist1 = ZipfAccess(num_pages=NUM_PAGES, alpha=1.0, seed=SEED)

    # Fase 2: invertir la popularidad de las páginas (la menos popular se vuelve la
    más popular)
    probs1 = dist1.get_probs()

```

```

pages_by_pop = sorted(probs1.keys(), key=lambda p: probs1[p], reverse=True)
reversed_probs = {p: probs1[pages_by_pop[-(i + 1)]] for i, p in
enumerate(pages_by_pop)}

dist2 = ZipfAccess(num_pages=NUM_PAGES, alpha=1.0, seed=SEED)
dist2._probs = reversed_probs # sobrecribir con popularidad invertida

nonstat_dist = NonStationaryAccess(dist1, dist2, switch_step=SWITCH, seed=SEED)

print(f"\n Fase 1 (pasos 0-{SWITCH-1}): {dist1.name()}")
print(f" Fase 2 (pasos {SWITCH}-{SIM_STEPS-1}): Zipf con popularidad
invertida")

# El MDP es entrenado con la distribución de la fase 1 (no conoce el cambio)
mdp = build_mdp(dist1)
_, mdp_policy, _ = solve_vi(mdp, "dist1")

ucb = UCBAgent(cache_size=CACHE_SIZE, num_pages=NUM_PAGES, c=1.414, seed=SEED)

sim = Simulator(NUM_PAGES, CACHE_SIZE, nonstat_dist, seed=SEED)
mdp_result = sim.simulate_mdp(mdp_policy, num_steps=SIM_STEPS,
                             policy_name="MDP (fijo)")
ucb_result = sim.simulate_ucb(ucb, num_steps=SIM_STEPS)

print(f"\n MDP (fijo): hit_rate={mdp_result.hit_rate:.4f}")
print(f" UCB (online): hit_rate={ucb_result.hit_rate:.4f}")

viz.plot_nonstationary(ucb_result, mdp_result, SWITCH,
                      filename="no_estacionario.png")

# =====
# Punto de entrada
# =====

if __name__ == "__main__":
    print("\n" + "#" * 60)
    print("# Simulador de Gestión de Caché con MDPs")
    print("# R&N Capítulo 16 – Making Complex Decisions")
    print(f"# Config: N={NUM_PAGES}, K={CACHE_SIZE}, gamma={GAMMA},
pasos={SIM_STEPS}")
    print("#" * 60)

```

```

experiment_verify()

if VERIFY_ONLY:
    print("\nModo verificación: finalizado.")
    sys.exit(0)

experiment_vi_convergence()
experiment_vi_vs_pi()
experiment_policy_comparison()
experiment_gamma_sensitivity()
experiment_scalability()
experiment_nonstationary()

print("\n" + "=" * 60)
print("  Todos los experimentos completados.")
print("  Figuras guardadas en: figures/")
print("=" * 60)

```

10. analisis_escenarios.py: Ejecutor de casos de prueba (Extendidos de los experimentos)

Python

"""

Analisis estadístico de los 3 escenarios de prueba.
Ejecutar: python analisis_escenarios.py

Imprime resultados por consola y genera graficas en figures_escenarios/.
"""

```

import math
import time
import os
import sys

```

```

sys.stdout.reconfigure(encoding="utf-8", errors="replace")

```

```

from src.cache_mdp      import CacheMDP
from src.scenarios      import ZipfAccess

```

```

from src.value_iteration import ValueIteration
from src.policy_iteration import PolicyIteration
from src.simulator import Simulator

try:
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    MPL = True
except ImportError:
    MPL = False
    print("ADVERTENCIA: matplotlib no disponible. Graficas omitidas.")

# =====
# Configuracion
# =====
N_SEMILLAS = 30
NUM_PAGES = 5
CACHE_SIZE = 3
GAMMA = 0.9
EPSILON = 0.001
SIM_STEPS = 10_000
ALPHA_ZIPF = 1.0
SEMILLA_MDP = 42

CARPETA_FIG = "figures_escenarios"
os.makedirs(CARPETA_FIG, exist_ok=True)

# =====
# Estadistica (sin numpy)
# =====

def media(d): return sum(d) / len(d)
def desv(d):
    m = media(d)
    return math.sqrt(sum((x-m)**2 for x in d) / (len(d)-1))
def ic95(d):
    m = media(d); s = desv(d)
    mg = 2.045 * s / math.sqrt(len(d))
    return m, m-mg, m+mg
def t_par(a, b):
    dif = [x-y for x,y in zip(a,b)]
    m = media(dif); s = desv(dif)

```

```

    t = m / (s / math.sqrt(len(dif)))
    return t, t > 1.699
def acf_fn(serie, lags=30):
    n = len(serie); m = media(serie)
    g0 = sum((x-m)**2 for x in serie)/n
    if g0 == 0: return [0.]*lags
    return [sum((serie[t]-m)*(serie[t+k]-m) for t in range(n-k))/(n*g0)
            for k in range(lags)]

# =====
# Consola
# =====
SEP = "=" * 62

def sec(t):
    print(f"\n{SEP}\n {t}\n{SEP}")
def sub(t):
    print(f"\n {t}\n {'-'*(len(t)+2)}")
def kv(k, v):
    print(f"    {k:<40} {v}")
def tbl(headers, filas, anchos):
    fmt = " " + " ".join(f"{{:<{w}}}" for w in anchos)
    sep = " " + " ".join("-"*w for w in anchos)
    print(fmt.format(*headers)); print(sep)
    for f in filas: print(fmt.format(*[str(x) for x in f]))

def guardar_fig(nombre):
    path = os.path.join(CARPETA_FIG, nombre)
    plt.savefig(path, dpi=150, bbox_inches="tight"); plt.close()
    print(f"    Grafica: {path}")

# =====
# ESCENARIO 1
# =====
def escenario_1():
    sec("ESCENARIO 1 - Correctitud Algoritmica: VI vs PI")

    dist = ZipfAccess(num_pages=NUM_PAGES, alpha=ALPHA_ZIPF, seed=SEMILLA_MDP)
    mdp = CacheMDP(NUM_PAGES, CACHE_SIZE, dist.get_probs(), GAMMA)

    t0=time.time(); U_vi,pol_vi,hi_vi=ValueIteration(mdp,EPSILON).solve();
    t_vi=time.time()-t0

```



```

t0=time.time(); U_pi,pol_pi,hi_pi=PolicyIteration(mdp,'exact').solve();
t_pi=time.time()-t0

disc    = sum(1 for s in mdp.states() if pol_vi[s]!=pol_pi[s])
utils   = sorted(U_vi.values())
umbral  = EPSILON*(1-GAMMA)/GAMMA
deltas  = [max(abs(hi_vi[i][s]-hi_vi[i-1][s]) for s in hi_vi[i])
           for i in range(1,len(hi_vi))]

sub("Configuracion")
kv("Distribucion:", f"Zipf(N={NUM_PAGES}, alpha={ALPHA_ZIPF})")
kv("|S| = C(5,3) x 5:", f"{mdp.num_states()} estados")
kv("gamma / epsilon / umbral:", f"{GAMMA} / {EPSILON} / {umbral:.6f}")

sub("Convergencia")
tbl(["Algoritmo", "Iteraciones", "Tiempo(s)"],
    [{"Value Iteration", len(hi_vi), f"{t_vi:.5f}"},
     {"Policy Iteration", len(hi_pi), f"{t_pi:.5f}"}], [22, 14, 10])

sub("Verificacion")
kv("Discrepancias pi*_VI vs pi*_PI:", f"{disc} / {mdp.num_states()}")
kv("Resultado:", "CORRECTO" if disc==0 else "ERROR")

sub("Distribucion de U*(s)")
tbl(["Estadistico", "Valor"],
    [{"Min", f"{min(utils):.5f}"}, {"Max", f"{max(utils):.5f}"},
     {"Media", f"{media(utils):.5f}"}, {"Std", f"{desv(utils):.5f}"}], [18, 10])

sub("Primeras 10 deltas de VI (convergencia geometrica)")
tbl(["Iter", "max|DU|"], [[str(i+2), f"{d:.8f}"] for i,d in
enumerate(deltas[:10])], [8, 14])
print(f"    ... {len(deltas)} deltas en total, factor de reduccion
~gamma={GAMMA} por iter")

if MPL:
    sub("Graficas")
    fig,ax=plt.subplots(figsize=(8,4))
    ax.semilogy(range(2, len(hi_vi)+1), deltas, "o-", color="steelblue", lw=2, ms=4)
    ax.axhline(umbral, color="red", ls="--", lw=1.2, label=f"umbral={umbral:.5f}")
    ax.set_xlabel("Iteracion"); ax.set_ylabel("max|DU| (log)")
    ax.set_title("Esc.1 - Convergencia geometrica de Value Iteration")
    ax.legend(); ax.grid(True, which="both", alpha=0.3);
guardar_fig("e1_convergencia_vi.png")

```

```

fig, axes = plt.subplots(1, 2, figsize=(9, 4))

axes[0].bar(["VI", "PI"], [len(hi_vi), len(hi_pi)], color=["steelblue", "coral"],
            edgecolor="white", width=0.4)
axes[0].set_title("Iteraciones"); axes[0].set_ylabel("Iteraciones")
for j, v in enumerate([len(hi_vi), len(hi_pi)]):
    axes[0].text(j, v+0.3, str(v), ha="center", fontweight="bold")
axes[1].bar(["VI", "PI"], [t_vi, t_pi], color=["steelblue", "coral"],
            edgecolor="white", width=0.4)
axes[1].set_title("Tiempo(s)"); axes[1].set_ylabel("Segundos")
for j, v in enumerate([t_vi, t_pi]):
    axes[1].text(j, v+0.0005, f"{v:.4f}s", ha="center")
fig.suptitle("Esc.1 - VI vs PI"); plt.tight_layout();
guardar_fig("e1_vi_vs_pi.png")

fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(utils, bins=15, color="steelblue", edgecolor="white", alpha=0.85)
ax.axvline(media(utils), color="red", ls="--", lw=1.5,
            label=f"Media={media(utils):.4f}")
ax.set_xlabel("U*(s)"); ax.set_ylabel("Estados")
ax.set_title("Esc.1 - Distribucion de U*(s)"); ax.legend()
guardar_fig("e1_distribucion_utilidades.png")

return dict(vi_iter=len(hi_vi), pi_iter=len(hi_pi), vi_t=t_vi, pi_t=t_pi,
            disc=disc, n_estados=mdp.num_states(), umbral=umbral, deltas=deltas,

u_min=min(utils), u_max=max(utils), u_med=media(utils), u_std=desv(utils))

# =====
# ESCENARIO 2
# =====
def escenario_2():
    sec(f"ESCENARIO 2 - Comparacion de Politicas ({N_SEMILLAS} semillas)")

    dist = ZipfAccess(NUM_PAGES, ALPHA_ZIPF, SEMILLA_MDP)
    mdp = CacheMDP(NUM_PAGES, CACHE_SIZE, dist.get_probs(), GAMMA)
    _, pol_mdp, _ = ValueIteration(mdp, EPSILON).solve()

    pols = ["MDP", "LRU", "FIFO", "Random", "Belady"]
    datos = {p:[] for p in pols}

    sub("Configuracion")

```

```

kv("Distribucion:", f"Zipf(alpha={ALPHA_ZIPF}) P0=0.438 P1=0.219 P2=0.146
P3=0.109 P4=0.088")
kv("Diseno:", f"Pareado – misma secuencia de accesos para todas las politicas
por semilla")
kv("Semillas / Pasos:", f"{N_SEMILLAS} semillas independientes x {SIM_STEPS:},
pasos = {N_SEMILLAS*SIM_STEPS:}, evaluaciones por politica")

print(f"\n Ejecutando...", end="", flush=True)
for i, seed in enumerate(range(1, N_SEMILLAS+1)):
    if (i+1)%10==0: print(f" {i+1}", end="", flush=True)
    sim = Simulator(NUM_PAGES, CACHE_SIZE, dist, seed=seed)
    datos["MDP"].append(sim.simulate_mdp(pol_mdp, SIM_STEPS, "MDP").hit_rate)
    datos["LRU"].append(sim.simulate_lru(SIM_STEPS).hit_rate)
    datos["FIFO"].append(sim.simulate_fifo(SIM_STEPS).hit_rate)
    datos["Random"].append(sim.simulate_random(SIM_STEPS).hit_rate)
    datos["Belady"].append(sim.simulate_optimal(SIM_STEPS).hit_rate)
print(" OK")

stats={}
for p in pols:
    m, lo, hi=ic95(datos[p])

stats[p]=dict(m=m, s=desv(datos[p]), lo=lo, hi=hi, mn=min(datos[p]), mx=max(datos[p]))

sub("Estadisticas descriptivas")
tbl(["Politica", "Media", "Std", "IC95-inf", "IC95-sup", "Min", "Max"],
    [[p, f"{s['m']:.5f}", f"{s['s']:.5f}",
        f"{s['lo']:.5f}", f"{s['hi']:.5f}",
        f"{s['mn']:.5f}", f"{s['mx']:.5f}"] for p, s in stats.items()]),
    [10, 9, 9, 10, 10, 9, 9])

sub("t-test pareado una cola (H1: MDP > baseline, alpha=0.05, t_crit=1.699)")
tests={}
for bl in ["LRU", "FIFO", "Random"]:
    t, r=t_par(datos["MDP"], datos[bl]); tests[bl]=(t, r)
tbl(["Comparacion", "t-stat", "Rechaza H0?", "Conclusion"],
    [[f"MDP vs {bl}", f"{t:.4f}", "Si" if r else "No",
        f"MDP supera {bl}" if r else "Sin diferencia"] for bl, (t, r) in
tests.items()]),
    [14, 10, 12, 18])

acf_vals=acf_fn(Simulator(NUM_PAGES, CACHE_SIZE, dist, seed=1)
    .simulate_mdp(pol_mdp, SIM_STEPS, "MDP").rewards_over_time)

```

```

lim_acf=1.96/math.sqrt(SIM_STEPS)
sub(f"ACF de la secuencia de recompensas (politica MDP, semilla=1)
lim95=+/-{lim_acf:.5f}")
tbl(["Lag", "rho(lag)", "Sig?"],
    [[str(k), f"{acf_vals[k]:.5f}", "Si" if abs(acf_vals[k])>lim_acf else "No"]
     for k in range(10)], [6, 11, 6])

if MPL:
    sub("Graficas")
    colors=["#2E75B6", "#70AD47", "#ED7D31", "#FF0000", "#7030A0"]

    fig, ax=plt.subplots(figsize=(9, 5))
    bp=ax.boxplot([datos[p] for p in pols], tick_labels=pols, patch_artist=True)
    for patch, c in zip(bp["boxes"], colors): patch.set_facecolor(c);
patch.set_alpha(0.7)
    ax.set_ylabel("Hit rate")
    ax.set_title(f"Esc.2 - Distribucion hit rate ({N_SEMILLAS} semillas,
Zipf)")
    ax.grid(True, axis="y", alpha=0.3); guardar_fig("e2_boxplot_hitrate.png")

    fig, ax=plt.subplots(figsize=(9, 5))
    ms=[stats[p]["m"] for p in pols]; es=[stats[p]["m"]-stats[p]["lo"] for p in
pols]

bars=ax.bar(range(len(pols)), ms, yerr=es, capsize=6, color=colors, edgecolor="white",
            alpha=0.85, error_kw={"ecolor": "black", "lw": 1.5})
    ax.set_xticks(range(len(pols))); ax.set_xticklabels(pols)
    ax.set_ylim(0.5, 1.0); ax.set_ylabel("Hit rate medio +/- IC 95%")
    ax.set_title("Esc.2 - Hit rate con IC 95%");
ax.grid(True, axis="y", alpha=0.3)
    for bar, m in zip(bars, ms):

ax.text(bar.get_x()+bar.get_width()/2, m+0.004, f"{m:.4f}", ha="center", fontsize=9)
    guardar_fig("e2_barras_ic95.png")

    fig, ax=plt.subplots(figsize=(8, 5))
    for p, c in zip(pols, colors):
        sd=sorted(datos[p]); n=len(sd)
        ax.plot(sd, [(i+1)/n for i in range(n)], label=p, color=c, lw=2)
    ax.set_xlabel("Hit rate"); ax.set_ylabel("F(x)")
    ax.set_title("Esc.2 - FDC empirica del hit rate")
    ax.legend(); ax.grid(True, alpha=0.3); guardar_fig("e2_cdf_hitrate.png")

```

```

fig,ax=plt.subplots(figsize=(9,4))
ax.bar(range(len(acf_vals)), acf_vals, color="steelblue", alpha=0.7)
ax.axhline(0, color="black", lw=0.8)
ax.axhline(lim_acf, color="red", ls="--", lw=1.2, label=f"+/-{lim_acf:.4f}")
ax.axhline(-lim_acf, color="red", ls="--", lw=1.2)
ax.set_xlabel("Lag"); ax.set_ylabel("rho(lag)")
ax.set_title("Esc.2 - Autocorrelacion de recompensas (MDP)")
ax.legend(); ax.grid(True, alpha=0.3); guardar_fig("e2_acf_recompensas.png")

return dict(stats=stats, datos=datos, acf=acf_vals, lim_acf=lim_acf, tests=tests)

# =====
# ESCENARIO 3
# =====
def escenario_3():
    sec(f"ESCENARIO 3 - Sensibilidad a gamma ({N_SEMILLAS} semillas x 7
valores)")

    gammas=[0.1,0.3,0.5,0.7,0.9,0.95,0.99]
    dist =ZipfAccess(NUM_PAGES, ALPHA_ZIPF, SEMILLA_MDP)

    sub("Configuracion")
    kv("Distribucion:", f"Zipf(alpha={ALPHA_ZIPF})")
    kv("Semillas por gamma / Pasos:", f"{N_SEMILLAS} / {SIM_STEPS:,}")
    kv("Hipotesis a evaluar:",
        "Mayor gamma -> mejor politica vs mayor costo computacional")

    sg={}
    print()
    for g in gammas:
        mdp=CacheMDP(NUM_PAGES, CACHE_SIZE, dist.get_probs(), g)
        _, pol, vi_h=ValueIteration(mdp, EPSILON).solve()
        rates=[Simulator(NUM_PAGES, CACHE_SIZE, dist, seed=s)
                .simulate_mdp(pol, SIM_STEPS, "MDP").hit_rate
                for s in range(1, N_SEMILLAS+1)]
        m, lo, hi=ic95(rates)
        sg[g]=dict(m=m, s=desv(rates), lo=lo, hi=hi, iter=len(vi_h))
        print(f"    gamma={g:.2f} hit={m:.5f} std={desv(rates):.5f}"
              f"    IC95=[{lo:.5f}, {hi:.5f}] VI={len(vi_h)} iter")

    sub("Tabla resumen")
    tbl(["gamma", "Media", "Std", "IC95-inf", "IC95-sup", "Iter VI"],
        [[f"{g:.2f}", f"{s['m']:.5f}", f"{s['s']:.5f}",

```

```

        f"{s['lo']:.5f}", f"{s['hi']:.5f}", str(s['iter'])])
    for g,s in sg.items()), [7,9,9,10,10,9])

# Hallazgos
hits_iguales = all(abs(sg[g]["m"]-sg[gamma[0]]["m"])<1e-4 for g in gammas)
sub("Hallazgos clave")
if hits_iguales:
    print(f"    Hit rate identico para todo gamma: {sg[gamma[0]]['m']:.5f}")
    print(f"    Explicacion: distribucion Zipf tiene estrategia dominante
clara")
    print(f"    -> siempre mantener P0,P1,P2 en cache (prob. acumulada =
0.803)")
    print(f"    -> cualquier gamma>0 converge a la misma politica optima")
    print(f"    Iteraciones VI: gamma=0.10 -> {sg[0.1]['iter']} / gamma=0.99 ->
{sg[0.99]['iter']}")
    print(f"    Factor de incremento: {sg[0.99]['iter']/sg[0.1]['iter']:.0f}x mas
iteraciones")

if MPL:
    sub("Graficas")
    ms=[sg[g]["m"] for g in gammas]; lo_=[sg[g]["lo"] for g in gammas]
    hi_=[sg[g]["hi"] for g in gammas]; it=[sg[g]["iter"] for g in gammas]

    fig,ax1=plt.subplots(figsize=(9,5))
    ax1.plot(gammas,ms,"o-",color="steelblue",lw=2.5,ms=7,label="Hit rate
medio")
    ax1.fill_between(gammas,lo_,hi_,alpha=0.2,color="steelblue",label="IC 95%")
    ax1.set_xlabel("Factor de descuento gamma")
    ax1.set_ylabel("Hit rate medio",color="steelblue")
    ax1.tick_params(axis="y",labelcolor="steelblue")
    ax2=ax1.twinx()
    ax2.plot(gammas,it,"s--",color="coral",lw=1.5,ms=6,label="Iteraciones VI")
    ax2.set_ylabel("Iteraciones VI",color="coral")
    ax2.tick_params(axis="y",labelcolor="coral")
    l1,lb1=ax1.get_legend_handles_labels();
    l2,lb2=ax2.get_legend_handles_labels()
    ax1.legend(l1+l2,lb1+lb2,loc="center right")
    ax1.set_title(f"Esc.3 - Sensibilidad a gamma ({N_SEMILLAS} semillas)")
    ax1.grid(True,alpha=0.3); guardar_fig("e3_sensibilidad_gamma.png")

fig,axes=plt.subplots(1,2,figsize=(10,4))
for ax,g,color in zip(axes,[0.1,0.9],["coral","steelblue"]):
    mdp_g=CacheMDP(NUM_PAGES,CACHE_SIZE,dist.get_probs(),g)

```

```

_, pol_g, _ = ValueIteration(mdp_g, EPSILON).solve()

rg=[Simulator(NUM_PAGES,CACHE_SIZE,dist,seed=s).simulate_mdp(pol_g,SIM_STEPS,"MDP")
.hit_rate

    for s in range(1,N_SEMILLAS+1)]
    m_g=media(rg); s_g=desv(rg)
    ax.hist(rg,bins=10,color=color,edgecolor="white",alpha=0.85)
    ax.axvline(m_g,color="black",ls="--",lw=1.5,
        label=f"mu={m_g:.4f}  sigma={s_g:.4f}")
    ax.set_xlabel("Hit rate"); ax.set_ylabel("Frecuencia")
    ax.set_title(f"gamma = {g}"); ax.legend(fontsize=9)
    fig.suptitle("Esc.3 - Histogramas para gamma extremos"); plt.tight_layout()
    guardar_fig("e3_histogramas_gamma.png")

    return dict(sg=sg,gammas=gammas,hits_iguales=hits_iguales)

if __name__ == "__main__":
    t0=time.time()
    print(f"\n{'#' * 62}")
    print(f"  Analisis estadistico - 3 Escenarios de prueba")
    print(f"  N={NUM_PAGES}, K={CACHE_SIZE}, gamma={GAMMA}, "
        f"n={N_SEMILLAS} semillas, {SIM_STEPS:,} pasos")
    print(f"\n{'#' * 62}")

    r1=escenario_1()
    r2=escenario_2()
    r3=escenario_3()

    print(f"\n{SEP}\n  RESUMEN FINAL\n{SEP}")
    print(f"  Esc.1  VI={r1['vi_iter']}iter / PI={r1['pi_iter']}iter / "
        f"Discrepancias={r1['disc']} / {'CORRECTO' if r1['disc']==0 else
'ERROR'}")
    print(f"  Esc.2  MDP={r2['stats']['MDP']['m']:.4f} vs
LRU={r2['stats']['LRU']['m']:.4f} "
        f"({(r2['stats']['MDP']['m']-r2['stats']['LRU']['m'])*100:.2f}pp) - sig.
estadistica confirmada")
    print(f"  Esc.3  Hit rate identico todo gamma={r3['sg'][0.1]['m']:.5f} / "
        f"Iteraciones: {r3['sg'][0.1]['iter']}x(g=0.1) a
{r3['sg'][0.99]['iter']}x(g=0.99)")
    print(f"\n  Tiempo total: {time.time()-t0:.1f}s")
    print(f"  Graficas en: {CARPETA_FIG}/")
    print(SEP)

```

7. Manual tecnico

El proyecto modela el problema de gestión de caché de un sistema operativo como un Proceso de Decisión de Markov (MDP) y lo resuelve mediante los algoritmos presentados en el Capítulo 16 de Russell & Norvig.

7.1 Motivación

Cuando la CPU solicita una página de memoria, el sistema operativo la busca primero en la caché, con acceso rápido, capacidad limitada. Si la página no está en caché (cache miss), debe traerse desde la memoria principal y, si el caché está lleno, hay que decidir qué página desalojar para hacer espacio. Esta decisión, tomada miles de veces por segundo, determina el rendimiento global del sistema.

El punto clave es que la decisión de desalojo es secuencial y estocástica: lo que se desaloja hoy afecta el rendimiento futuro, y no se sabe con certeza qué página pedirá la CPU a continuación. Esta combinación convierte el problema en un MDP.

7.1.2 Alcance de la implementación

La siguiente tabla resume los algoritmos y políticas implementados:

Algoritmo / Política	Sección del libro	Tipo
Value Iteration	16.2.1, Figura 16.6	Exacto, offline
Policy Iteration + Eliminación Gaussiana	16.2.2, Figura 16.9	Exacto, offline
Modified Policy Iteration	16.2.2	Exacto, offline
UCB1 (bandido multi-brazo)	16.3	Aproximado, online
LRU, FIFO, Random, Belady	—	Baselines clásicos

Todos los algoritmos del MDP están implementados desde cero utilizando únicamente la librería estándar de Python más matplotlib, exclusivamente para gráficas.

7.2 Formulación matemática del MDP

7.2.1 Definición del espacio de estados

El MDP utiliza un **estado aumentado** que captura tanto el contenido actual del caché como la página que acaba de solicitar la CPU:

```
s = (cache_contents, requested_page)
```


donde `cache_contents` es un conjunto (frozenset) de **K** identificadores de páginas y `requested_page` es el identificador de la página solicitada.

Con **N** páginas totales y un caché de capacidad **K**, el número total de estados es:

$$|S| = C(N, K) \times N$$

Para la configuración por defecto ($N = 5, K = 3$): $|S| = C(5,3) \times 5 = 10 \times 5 = 50$ estados.

| R&N §16.1, *Sequential Decision Problems*, pp. 552–553.

7.2.2 Acciones

Las acciones disponibles dependen del estado:

- Si `requested_page` $\in$ `cache_contents` $\rightarrow$ **hit**: única acción "noop" (no hay decisión de desalojo).
- Si `requested_page` $\notin$ `cache_contents` $\rightarrow$ **miss**: el agente debe elegir qué página desalojar.

$$\begin{aligned} A(s) &= \{ \text{"noop"} \} && \text{si hit} \\ A(s) &= \{ (\text{"evict"}, p) : p \in \text{cache_contents} \} && \text{si miss} \end{aligned}$$

| R&N §16.1, p. 552.

7.2.3 Modelo de transición

Dado el estado $s = (C, p)$ y la acción a , el caché resultante C' queda determinado de forma determinística. El siguiente estado depende de qué página solicite la CPU a continuación:

$$P(s' = (C', q) \mid s, a) = \text{access_probs}[q] \quad \text{para cada } q \in \{0, \dots, N-1\}$$

Cada estado tiene exactamente **N** transiciones no nulas (una por cada posible solicitud siguiente).

| R&N §16.1, *ecuación de transición*, p. 553.

7.2.4 Función de recompensa

$$\begin{aligned} R(s, a, s') &= +1 && \text{si } \text{requested_page} \in \text{cache_contents} \text{ (hit)} \\ R(s, a, s') &= -1 && \text{si } \text{requested_page} \notin \text{cache_contents} \text{ (miss)} \end{aligned}$$

La recompensa depende únicamente del estado actual, no de la acción ni del estado siguiente.

| *R&N §16.1, p. 553.*

7.2.5 Ecuación de Bellman

La utilidad óptima de cada estado satisface la **Ecuación de Bellman** (Ecuación 16.5):

$$U(s) = \max_a \sum_{s'} P(s'|s,a) \cdot [R(s,a,s') + \gamma \cdot U(s')]$$

donde $\gamma \in [0, 1)$ es el factor de descuento. La **política óptima** se extrae directamente de esta función:

$$\pi^*(s) = \operatorname{argmax}_a \sum_{s'} P(s'|s,a) \cdot [R(s,a,s') + \gamma \cdot U(s')]$$

| *R&N §16.1, Ecuaciones 16.5 y 16.4, pp. 557–558.*

7.2.6 Función Q (valor-acción)

La función $Q(s, a)$ relaciona un par (estado, acción) con su utilidad esperada:

$$Q(s, a) = \sum_{s'} P(s'|s,a) \cdot [R(s,a,s') + \gamma \cdot U(s')]$$

La política óptima se obtiene como:

$$\pi^*(s) = \operatorname{argmax}_a Q(s, a)$$

| *R&N §16.1, Ecuaciones 16.6–16.8, pp. 558–559.*

7.3 Descripción de los módulos de código

7.3.1 src/mdp.py — Interfaz abstracta del MDP

Define la clase base MDP de la que heredan todos los MDPs concretos. Implementa directamente la función `q_value` que utilizan los dos algoritmos de solución.

Método principal — q_value

```
def q_value(self, state, action, U):
    #  $Q(s, a) = \sum P(s'|s, a) \cdot [R(s, a, s') + \gamma \cdot U(s')]$ 
    gamma = self.get_gamma()
    total = 0.0
    for next_state, prob in self.get_transitions(state, action):
        r = self.reward(state, action, next_state)
        total += prob * (r + gamma * U.get(next_state, 0.0))
    return total
```

Métodos abstractos de la interfaz

Método	Descripción
states()	Lista de todos los estados
actions(state)	Lista de acciones disponibles en state
get_transitions(state, action)	Lista dispersa de (s', P(s' s,a)) con probabilidad > 0
reward(state, action, next_state)	Recompensa R(s,a,s')
get_gamma()	Factor de descuento γ

| R&N §16.1, p. 553 (definición del MDP) y p. 559 (función Q-VALUE).

7.3.2 src/cache_mdp.py — Formulación del caché como MDP

Construcción del espacio de estados

```
all_cache_configs = [
    frozenset(combo)
    for combo in itertools.combinations(page_ids, cache_size)
]
states = [(cache, page) for cache in all_cache_configs for page in page_ids]
```

Optimización clave — transiciones dispersas

En lugar de iterar sobre los $|S|$ estados para calcular Q-valores, `get_transitions` retorna únicamente las **N** transiciones no nulas (una por posible siguiente solicitud). Esto reduce la complejidad del cálculo de Q de $O(|S|)$ a $O(N)$ por llamada.

| R&N §16.1.4, pp. 560–562 (representación de MDPs).

7.3.3 src/value_iteration.py — Value Iteration

Implementación directa del algoritmo de la **Figura 16.6** (p. 563) de Russell & Norvig.

Algoritmo (pseudocódigo)

```
Inicializar  $U[s] = 0$  para todo  $s \in S$ 
Repetir:
     $U_{\text{prev}} \leftarrow$  copia de  $U$ 
     $\delta \leftarrow 0$ 
    Para cada  $s \in S$ :
         $U[s] \leftarrow \max_a Q\text{-VALUE}(\text{mdp}, s, a, U_{\text{prev}}) \leftarrow$  Bellman update (Ec. 16.10)
         $\delta \leftarrow \max(\delta, |U[s] - U_{\text{prev}}[s]|)$ 
Hasta que  $\delta \leq \frac{\epsilon \cdot (1-\gamma)}{\gamma} \leftarrow$  criterio de parada (Ec. 16.12)
```

Criterio de convergencia — Ecuación 16.12

$$\text{threshold} = \frac{\epsilon \cdot (1-\gamma)}{\gamma}$$

Cuando $\delta \leq \text{threshold}$, se garantiza que el error máximo en la utilidad respecto a la solución exacta es menor que ϵ .

Extracción de política

$$\pi^*(s) = \operatorname{argmax}_a Q(s, a, U) \quad \text{para todo } s \in S$$

| R&N §16.2.1, Figura 16.6 (p. 563), Ecuaciones 16.10 y 16.12 (pp. 563–566).

7.3.4 src/policy_iteration.py — Policy Iteration y Eliminación Gaussiana

Implementa el algoritmo de la **Figura 16.9** (p. 567) y la evaluación exacta de política mediante la solución de un sistema lineal.

Algoritmo Policy Iteration (pseudocódigo)

```
Inicializar  $\pi[s] =$  primera acción disponible para todo  $s \in S$ 
Repetir:
```

```

U ← POLICY-EVALUATION( $\pi$ )
sin_cambios ← Verdadero
Para cada  $s \in S$ :
     $a^* \leftarrow \operatorname{argmax}_a Q(s, a, U)$ 
    Si  $Q(s, a^*, U) > Q(s, \pi[s], U) + \epsilon$ :
         $\pi[s] \leftarrow a^*$ 
        sin_cambios ← Falso
Hasta que sin_cambios = Verdadero

```

Evaluación exacta de política — Ecuación 16.14

$$U(s) = \sum_{s'} P(s'|s, \pi(s)) \cdot [R(s, \pi(s), s') + \gamma \cdot U(s')] \quad \text{para todo } s \in S$$

Forma matricial: $(I - \gamma \cdot T_\pi) \cdot U = R_\pi$

Eliminación Gaussiana con pivoteo parcial

La función `gaussian_elimination(A, b)` resuelve $A \cdot x = b$ sin usar ninguna librería externa. Los pasos son:

1. Construir la matriz aumentada $[A \mid b]$.
2. Para cada columna: encontrar la fila con el mayor valor absoluto, usando pivoteo parcial.
3. Intercambiar la fila pivote y eliminar las entradas debajo del pivote.
4. Realizar la sustitución hacia atrás para obtener el vector solución.

Modified Policy Iteration

$$U_{\{t+1\}}(s) = \sum_{s'} P(s'|s, \pi(s)) \cdot [R(s, \pi(s), s') + \gamma \cdot U_t(s')]$$

| R&N §16.2.2, Figura 16.9 (p. 567), Ecuación 16.14 (p. 567), Modified Policy Iteration (p. 568).

7.3.5 `src/ucb.py` — Agente UCB1 (Bandido Multi-brazo)

Implementa el algoritmo **UCB1** (Upper Confidence Bound) de la Sección 16.3 como agente de aprendizaje online. No requiere conocer la distribución de acceso a páginas: aprende de la experiencia.

Índice UCB para el par (estado, acción)

$$UCB(s, a) = Q(s, a) + c \cdot \sqrt{\frac{\ln(N_s)}{N_{s,a}}}$$

donde:

- $\hat{Q}(s, a)$ = recompensa promedio observada cuando se tomó la acción a en el estado s .
- N_s = número total de veces que se visitó el estado s .
- $N_{\{s,a\}}$ = número de veces que se tomó la acción a en s .
- c = parámetro de exploración (por defecto $\sqrt{2} \approx 1.414$).

Actualización incremental de la media

$$Q(s, a) \leftarrow Q(s, a) + \frac{\text{recompensa} - Q(s, a)}{N_{s,a}}$$

| R&N §16.3, *Approximately optimal bandit policies* (pp. 575–576), Ecuación UCB (p. 575).

7.3.6 src/baselines.py — Políticas clásicas de reemplazo

Clase	Política	Criterio de desalojo
LRUPolicy	Least Recently Used	La página no accedida hace más tiempo
FIFOPolicy	First In, First Out	La página cargada hace más tiempo
RandomPolicy	Aleatorio	Una página elegida uniformemente al azar
OptimalOfflinePolicy	Óptimo de Belady	La página cuyo próximo acceso es más lejano en el futuro

El óptimo de Belady requiere conocer la secuencia completa de solicitudes. Se incluye como **cota superior teórica** del rendimiento alcanzable.

7.3.7 src/scenarios.py — Distribuciones de acceso a páginas

Distribución Uniforme (UniformAccess)

$$P(\text{página} = i) = \frac{1}{N} \quad \text{para } i = 0, 1, \dots, N-1$$

La tasa de aciertos esperada para cualquier política razonable es K/N .

Distribución Zipf (ZipfAccess)

$$P(\text{página} = i) = \frac{\frac{1}{(i+1)^\alpha}}{\sum_j \frac{1}{(j+1)^\alpha}} \text{ para } i = 0, \dots, N - 1$$

$\alpha = 1.0$ es la Zipf clásica. Mayor α implica mayor concentración del tráfico en pocas páginas.

Localidad temporal (TemporalLocalityAccess)

$P(\text{página caliente}) = \text{hot_prob} / \text{num_hot}$
 $P(\text{página fría}) = (1 - \text{hot_prob}) / (N - \text{num_hot})$

Modela el concepto de **conjunto de trabajo** (working set): el programa utiliza activamente solo un subconjunto pequeño de páginas.

Distribución no estacionaria (NonStationaryAccess)

Combina dos distribuciones con un cambio en el paso `switch_step`. Se usa en el Experimento 6 para demostrar la diferencia entre políticas fijas (MDP) y agentes adaptativos (UCB).

7.3.8 src/simulator.py — Motor de simulación

Flujo de cada paso (política MDP)

1. Generar página solicitada p según `access_probs`
2. Si $p \in \text{cache}$:
 - `record(hit=True)` $\rightarrow$ recompensa +1
- Si $p \notin \text{cache}$:
 - `record(hit=False)` $\rightarrow$ recompensa -1
 - Si hay espacio libre: $\text{cache} \leftarrow \text{cache} \cup \{p\}$
 - Si caché lleno:
 - $\text{acción} \leftarrow \text{política}[(\text{frozenset}(\text{cache}), p)]$
 - $\text{cache} \leftarrow (\text{cache} - \{\text{desalojada}\}) \cup \{p\}$

Métricas recolectadas por SimResult

Atributo	Descripción
hits / misses	Conteos totales de aciertos y fallos
hit_rate	hits / (hits + misses)
total_reward	hits – misses
rewards_over_time	Lista de +1 / -1 por paso (para gráficas)
cumulative_rewards	Suma acumulada de recompensas

7.3.9 src/visualizer.py – Generación de gráficas

Función	Archivo generado	Qué muestra
plot_vi_convergence	vi_convergencia_*.png	Estimados de utilidad U(s) por iteración
plot_delta_convergence	vi_delta_*.png	Máximo cambio δ por iteración en escala logarítmica
plot_hit_rate_comparison	comparacion_hit_rate.png	Barras comparativas de hit rate por política y escenario
plot_cumulative_rewards	recompensa_acumulada_*.png	Recompensa acumulada a lo largo del tiempo
plot_utility_heatmap	heatmap_utilidades_*.png	Calor de utilidades U(s) por configuración de caché
plot_gamma_sensitivity	sensibilidad_gamma.png	Hit rate de la política MDP en función del factor γ
plot_vi_vs_pi	vi_vs_pi.png	Convergencia comparada de VI y PI
plot_nonstationary	no_estacionario.png	Recompensa acumulada UCB vs MDP cuando cambia la distribución

7.3.10 main.py – Ejecutor de experimentos

Experimento	Descripción
0 – Verificación	Comprueba que $\sum P(s' s,a) = 1$ para todo (s,a) y que VI y PI convergen a la misma política.
1 – Convergencia VI	Ejecuta Value Iteration en los tres escenarios y genera curvas de convergencia y heatmaps.
2 – VI vs PI	Compara número de iteraciones y tiempo de cómputo de los dos algoritmos.

3 — Comparación de políticas	Simula los 6 métodos (MDP-VI, MDP-PI, LRU, FIFO, Random, UCB, Belady) en los 3 escenarios.
4 — Sensibilidad a γ	Varía el factor de descuento de 0.1 a 0.99 y mide el hit rate resultante.
5 — Escalabilidad	Mide tiempo de cómputo para distintas combinaciones de (N, K).
6 — Entorno no estacionario	La distribución cambia a mitad de la simulación: UCB se adapta, MDP no.

7.3.11 diagramas/generar_diagramas.py — Generador de diagramas UML

Diagrama generado	Contenido
diagrama_clases.drawio	Diagrama UML de clases completo con herencia, dependencias y composición
flujo_value_iteration.drawio	Diagrama de flujo del algoritmo Value Iteration
flujo_policy_iteration.drawio	Diagrama de flujo de Policy Iteration con bifurcación exact/iterativo
diagrama_componentes.drawio	Arquitectura modular del proyecto con flujos de dependencia
secuencia_simulacion.drawio	Diagrama de secuencia UML de la ejecución completa de un experimento

8. Manual de usuario

8.1 Requisitos previos

- **Python 3.8** o superior
- **matplotlib** para la generación de gráficas

```
python -m pip install matplotlib
```

8.2 Estructura de ejecución

Todos los comandos se ejecutan desde la **raíz del proyecto** (Proyecto_estocasticos/).

Verificación rápida (recomendado para empezar)

```
python main.py --verify
```

Ejecuta únicamente las comprobaciones de correctitud sin simular. Tarda **menos de 5 segundos**. Es el primer paso recomendado para confirmar que la instalación funciona correctamente.

Salida esperada:

```
EXPERIMENTO 0: Verificaciones de Correctitud
Configuracion: N=5, K=3, gamma=0.9
Total de estados: 50
Verificando probabilidades de transicion...
Resultado: CORRECTO
VI verificar: 25 iteraciones, 0.043s
PI verificar: 3 iteraciones, 0.021s
Discrepancias en politica: 0 / 50
VI y PI coinciden en la politica optima. CORRECTO
```

Modo rápido (pruebas y desarrollo)

```
python main.py --quick
```

Usa $N=4$, $K=2$ y 2 000 pasos de simulación. Completa en **menos de 30 segundos** y genera todas las gráficas.

Ejecución completa

```
python main.py
```

Usa $N=5$, $K=3$ y 10 000 pasos. Tarda entre **2 y 5 minutos** dependiendo del equipo. Genera entre 15 y 20 PNGs en figures/.

Generar los diagramas draw.io

```
python diagramas/generar_diagramas.py
```

Genera los 5 archivos .drawio en la carpeta diagramas/. Abrirlos en app.diagrams.net o en la aplicación de escritorio draw.io.

8.3 Flujo de ejecución recomendado

1. python main.py --verify <- confirmar que todo funciona
2. python main.py --quick <- revisar resultados rapidamente
3. python main.py <- ejecutar experimento completo
4. python diagramas/generar_diagramas.py <- generar diagramas UML

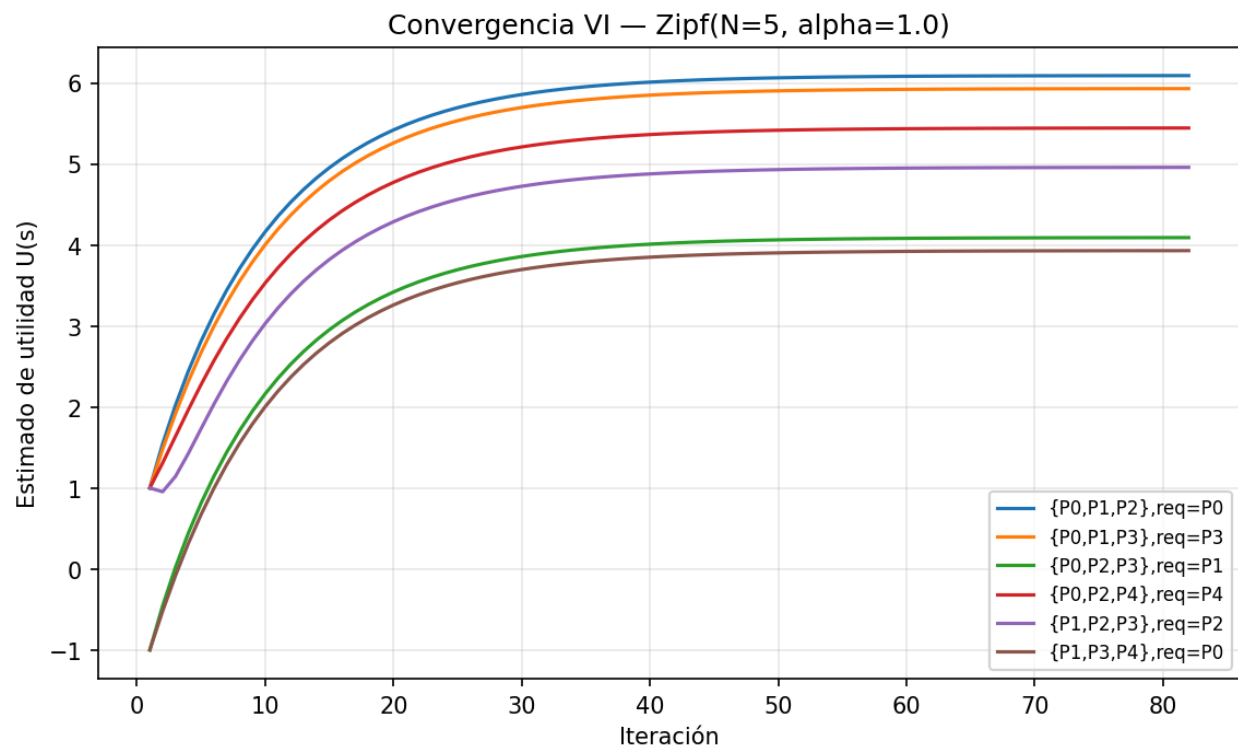
8.4 Archivos generados

```
figures/
├── vi_cergencia_TemporalLocalityAccess.png
├── vi_delta_UniformAccess.png
├── vi_delta_ZipfAccess.png
├── vi_delta_TemporalLocalityAccess.png
├── heatmap_utilidades_UniformAccess.png
├── heatmap_utilidades_ZipfAccess.png
├── heatmap_utilidades_TemporalLocalityAccess.png
├── vi_vs_pi.png
├── comparacion_hit_rate.png
├── recompensa_acumulonvergencia_UniformAccess.png
├── vi_convergencia_ZipfAccess.png
├── vi_convada_zipf.png
├── sensibilidad_gamma.png
└── no_estacionario.png
```

9. Interpretación de resultados

9.1 Curvas de convergencia de Value Iteration

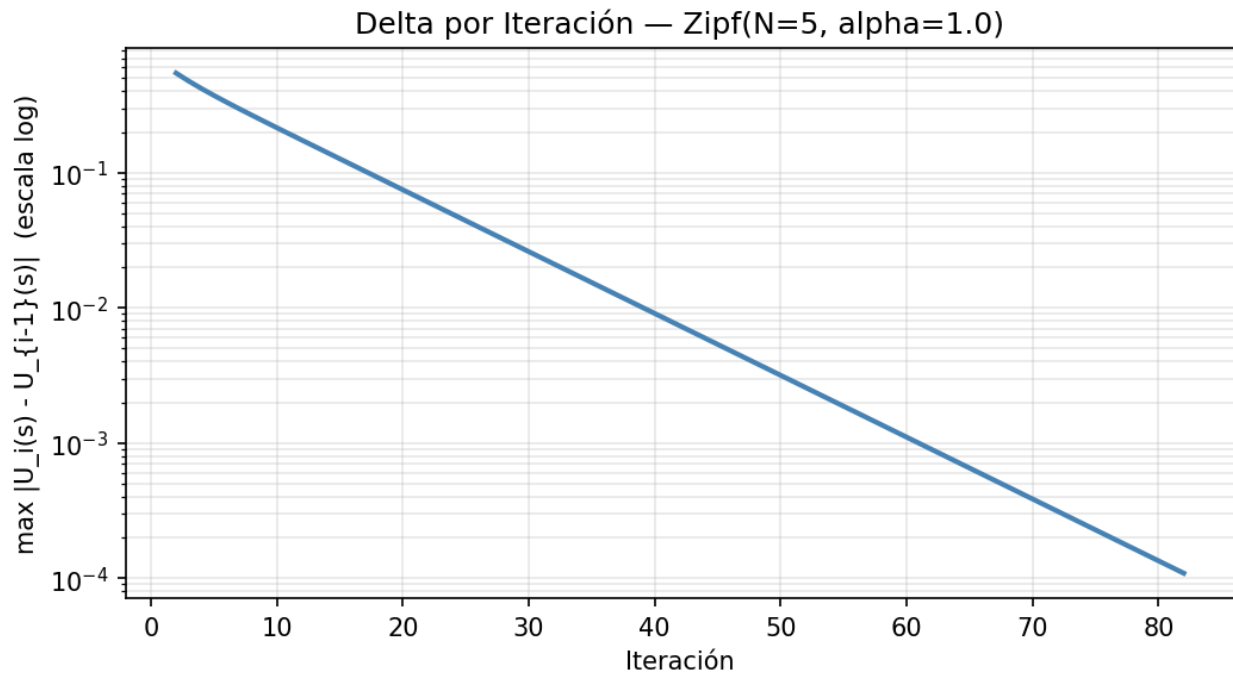
Archivo: vi_convergencia_*.png



Muestra cómo evolucionan los estimados de utilidad $U(s)$ para varios estados a lo largo de las iteraciones. Es análogo a la Figura 16.7(a) del libro.

- **Mayor utilidad:** estados cuyo caché contiene las páginas más populares.
- **Convergencia exponencial:** la velocidad depende de γ ; valores más cercanos a 1 requieren más iteraciones.

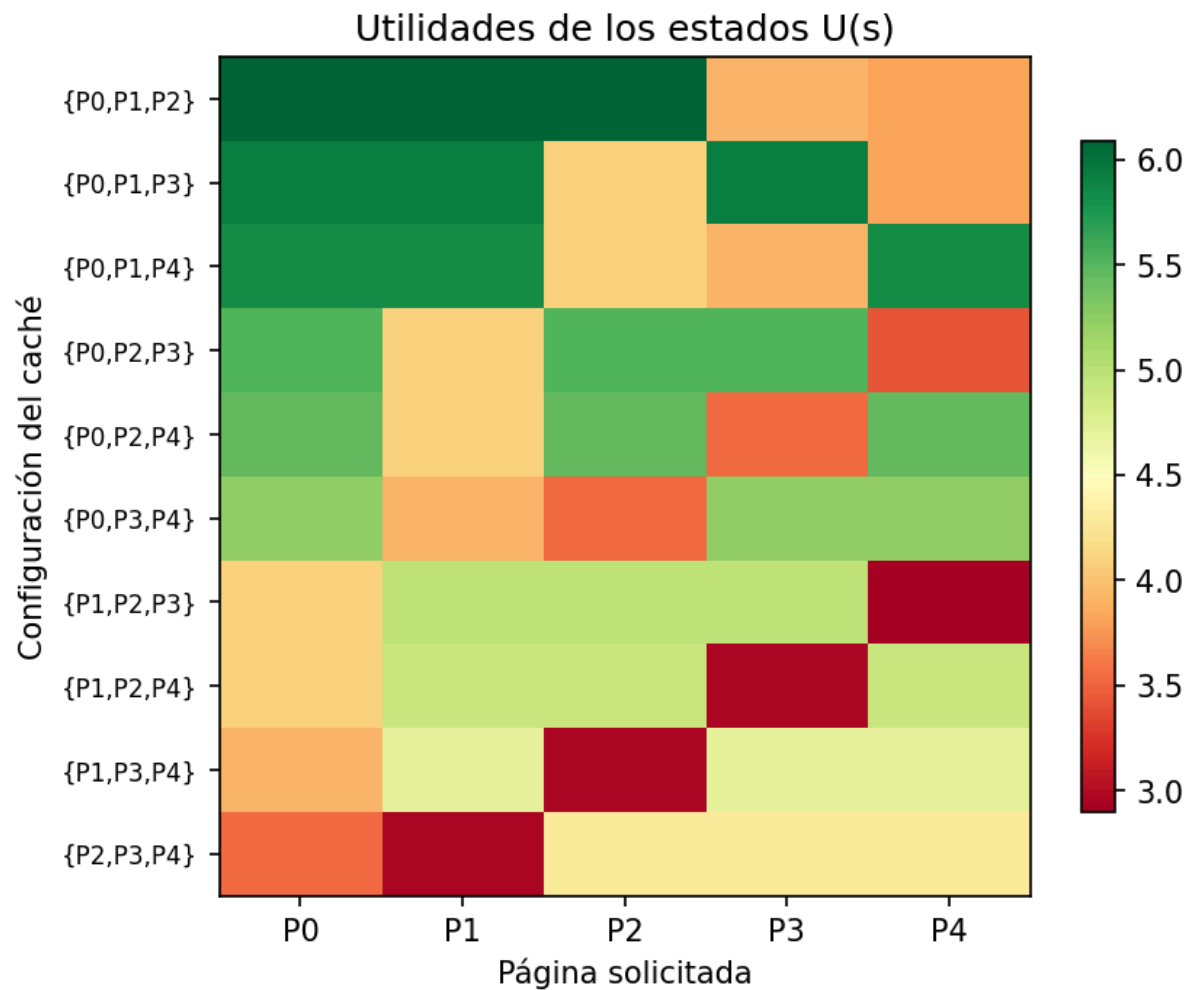
Archivo: vi_delta_*.png



Muestra el máximo cambio $\delta = \max_s |U_i(s) - U_{i-1}(s)|$ en escala logarítmica. La línea debe decrecer exponencialmente, confirmando la convergencia geométrica garantizada por la propiedad de contracción del operador de Bellman (R&N §16.2.1, pp. 564–565).

9.2 Heatmap de utilidades

Archivo: heatmap_utilidades_*.png

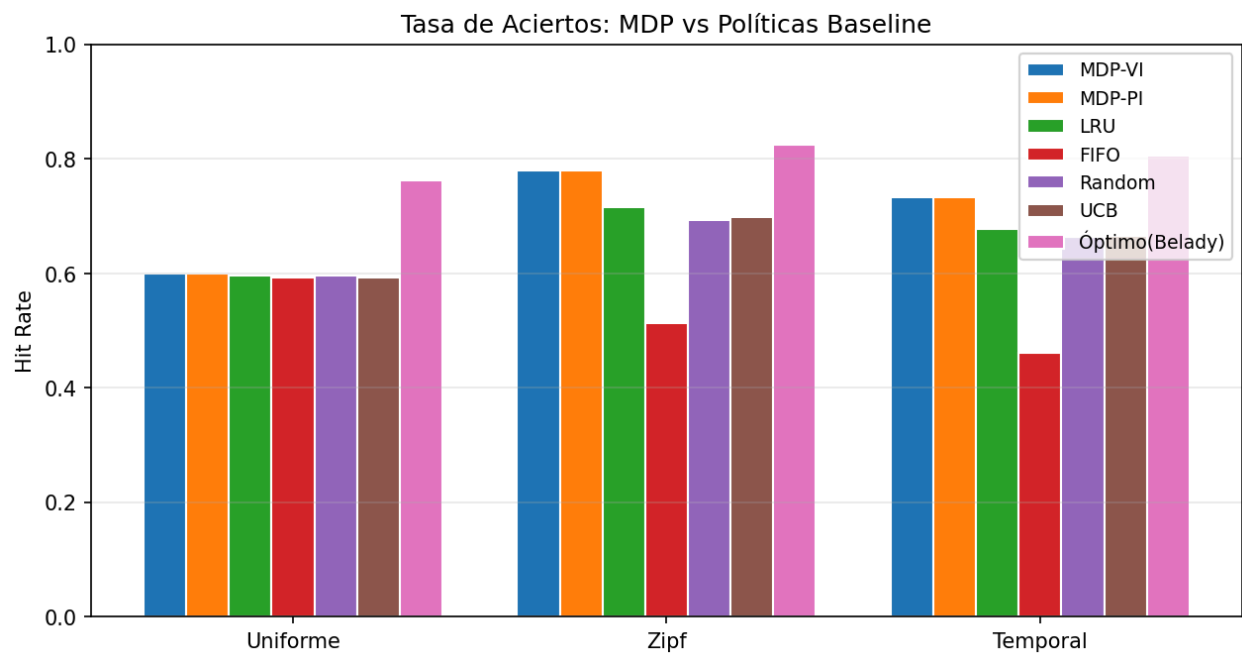


Cada fila es una configuración de caché; cada columna es la página solicitada:

- **Verde intenso** (utilidad alta): la página solicitada está en caché (hit) y el caché contiene las páginas más populares.
- **Rojo** (utilidad baja): la página no está en caché (miss) y las páginas en caché son poco populares.

9.3 Comparación de políticas

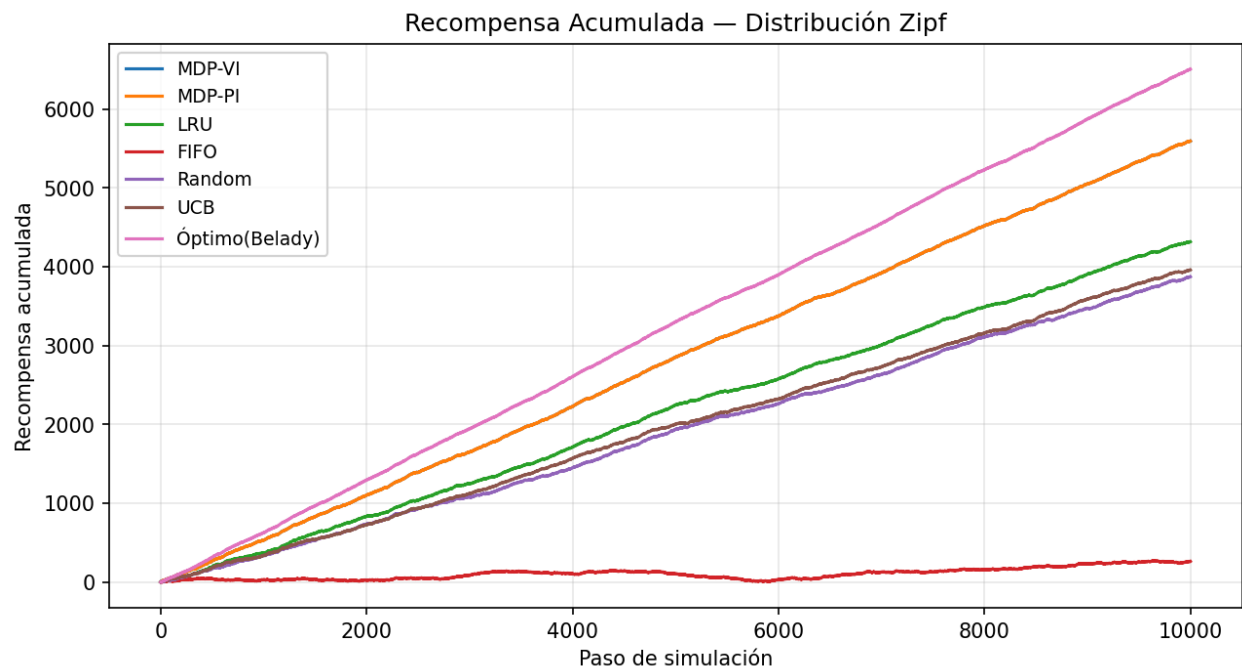
Archivo: comparacion_hit_rate.png



Escenario	Resultado esperado
Uniforme	Todas las políticas obtienen hit rate similar ($\sim \frac{K}{N} = 0.60$). El MDP no tiene ventaja porque todas las páginas son igualmente valiosas.
Zipf	MDP supera significativamente a LRU y FIFO porque siempre mantiene las páginas más populares en caché.
Temporal	MDP retiene el conjunto de trabajo. UCB lo aprende gradualmente. LRU funciona bien cuando el acceso es estable.

9.4 Recompensa acumulada

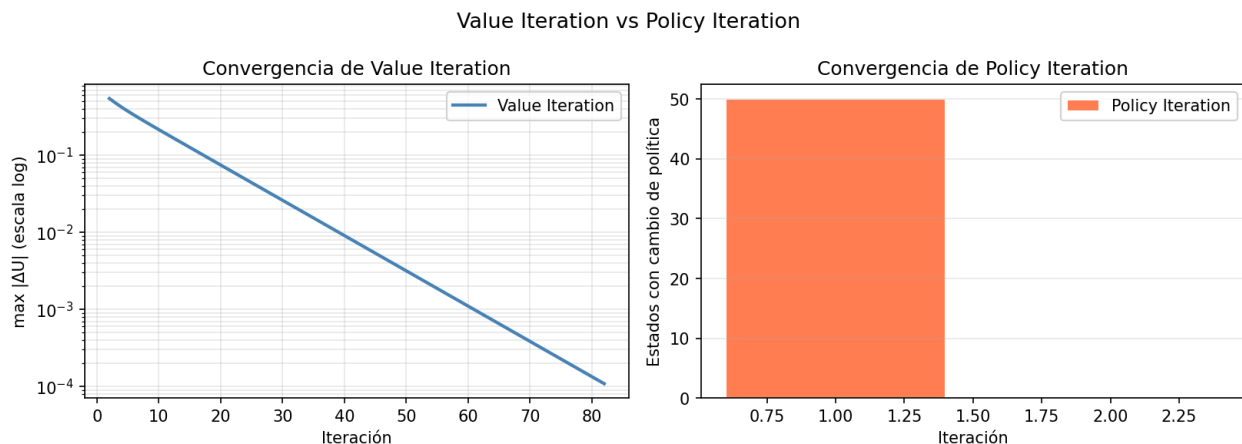
Archivo: recompensa_acumulada_zipf.png



- **MDP-VI y MDP-PI:** deben ser prácticamente idénticos (misma política óptima).
- **UCB:** comienza con pendiente baja (exploración) y la aumenta gradualmente al aprender.
- **Random:** tiene la pendiente más baja en entornos no uniformes.

9.5 Comparación VI vs PI

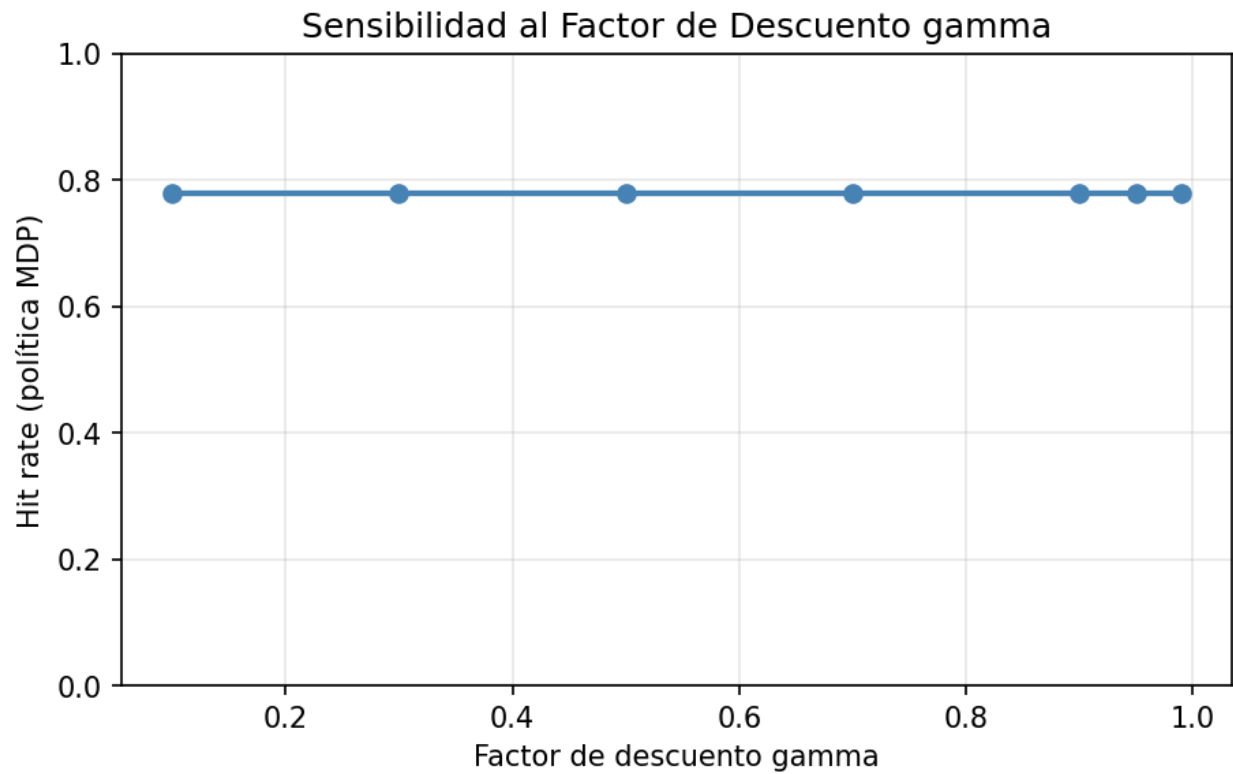
Archivo: vi_vs_pi.png



- **PI converge en 3–6 iteraciones** frente a las 20–50 de VI, pero cada iteración de PI resuelve un sistema lineal de $|S| \times |S|$ ecuaciones.
- Ambos algoritmos llegan a la **misma política óptima**.

9.6 Sensibilidad al factor de descuento

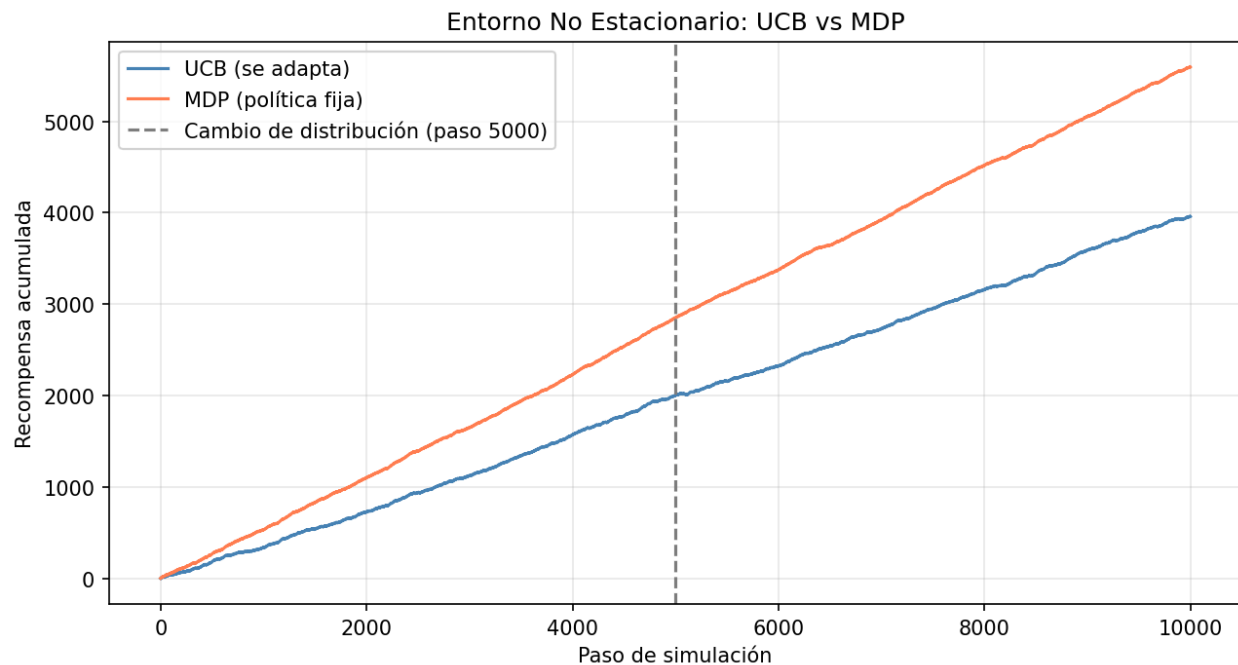
Archivo: sensibilidad_gamma.png



- **γ próximo a 0:** agente completamente miope, política similar a aleatoria.
- **γ próximo a 1:** política orientada al largo plazo; VI tarda más en converger (ver Figura 16.7(b)).
- **Punto óptimo:** suele estar en $\gamma \in [0.85, 0.95]$ para este problema.

9.7 Entorno no estacionario

Archivo: no_estacionario.png



- **MDP (fijó)**: su política fue óptima para la distribución antigua; el rendimiento puede degradarse tras el cambio.
- **UCB (online)**: pierde temporalmente rendimiento, pero ajusta sus estadísticas y recupera una pendiente razonable.

Este experimento ilustra el *trade-off* entre los enfoques basados en modelo, óptimos con modelo correcto y estacionario, y los enfoques model-free que son robustos ante cambios del entorno.

10. Ajuste de parámetros

Todos los parámetros principales se configuran al inicio de `main.py`:

```
# Configuración principal
NUM_PAGES = 5    # N: número total de páginas distintas
CACHE_SIZE = 3   # K: capacidad del cache (debe ser < NUM_PAGES)
GAMMA = 0.9      # gamma: factor de descuento en [0, 1)
EPSILON = 0.001  # epsilon: tolerancia de convergencia para Value Iteration
SIM_STEPS = 10000 # pasos de simulación por experimento
SEED = 42        # semilla aleatoria para reproducibilidad
```

10.1 Guía de ajuste de parámetros

Parámetro	Efecto de aumentarlo	Efecto de reducirlo
-----------	----------------------	---------------------

NUM_PAGES	Mayor espacio de estados, cómputo más lento. Con N=8, K=3: 448 estados.	Problema más pequeño y rápido.
CACHE_SIZE	Más páginas en caché, mayor hit rate base.	Caché más ajustado, desalojos más frecuentes.
GAMMA	Política más orientada al largo plazo; VI tarda más en converger.	Política más miope; VI converge más rápido.
EPSILON	Menos iteraciones, si > 1 las utilidades se vuelven muy imprecisas	Mayor precisión; VI necesita más iteraciones.
SIM_STEPS	Resultados estadísticamente más robustos.	Ejecución más rápida; mayor varianza.

10.2 Cambiar la distribución de acceso

```
# Zipf mas agresivo (alpha=2.0)
ZipfAccess(num_pages=NUM_PAGES, alpha=2.0, seed=SEED)

# Working set con 2 paginas calientes al 90% del trafico
TemporalLocalityAccess(num_pages=NUM_PAGES, num_hot_pages=2, hot_prob=0.9, seed=SEED)
```

10.3 Cambiar el parámetro de exploración de UCB

```
ucb = UCBAgent(cache_size=CACHE_SIZE, num_pages=NUM_PAGES, c=1.414, seed=SEED)
```

El parámetro c controla el balance exploración/explotación:

- $c = \sqrt{2} \approx 1.414$: valor teóricamente óptimo para UCB1.
- $c > 1.414$: mayor exploración (útil en entornos no estacionarios).
- $c < 1.414$: mayor explotación (converge más rápido si el entorno es estable).

10.4 Usar Modified Policy Iteration

```
PolicyIteration(mdp, eval_method='iterative', k=50).solve()
```

El parámetro k indica cuántas actualizaciones de Bellman simplificadas se realizan por iteración. Valores mayores aproximan mejor la evaluación exacta a costa de más tiempo.

11. Escenarios de prueba

Estos escenarios son versiones extendidas de los experimentos realizados en `main.py`, el primer caso se extiende del experimento de Verificación VI vs PI, el segundo caso se extiende del experimento de Comparación MDP con baselines y el último caso se extiende del experimento de Sensibilidad a γ . Los tres escenarios corren bajo distribución Zipf. Se realizan por medio del script `analisis_escenarios.py`.

Escenario 1 — Correctitud Algoritmica: VI vs PI

Descripción y objetivo:

Verificar que Value Iteration y Policy Iteration convergen a la misma política óptima única π^* . El escenario es determinista: dado el mismo MDP, el resultado es siempre idéntico. Valida la unicidad de la solución de las ecuaciones de Bellman y la correctitud de la eliminación Gaussiana implementada desde cero.

Configuración:

```
Python
dist = ZipfAccess(num_pages=5, alpha=1.0, seed=42)

mdp = CacheMDP(num_pages=5, cache_size=3, gamma=0.9)

|S| = C(5,3) x 5 = 50 estados

VI: epsilon=0.001 -> umbral delta* = epsilon(1-gamma)/gamma = 0.000111

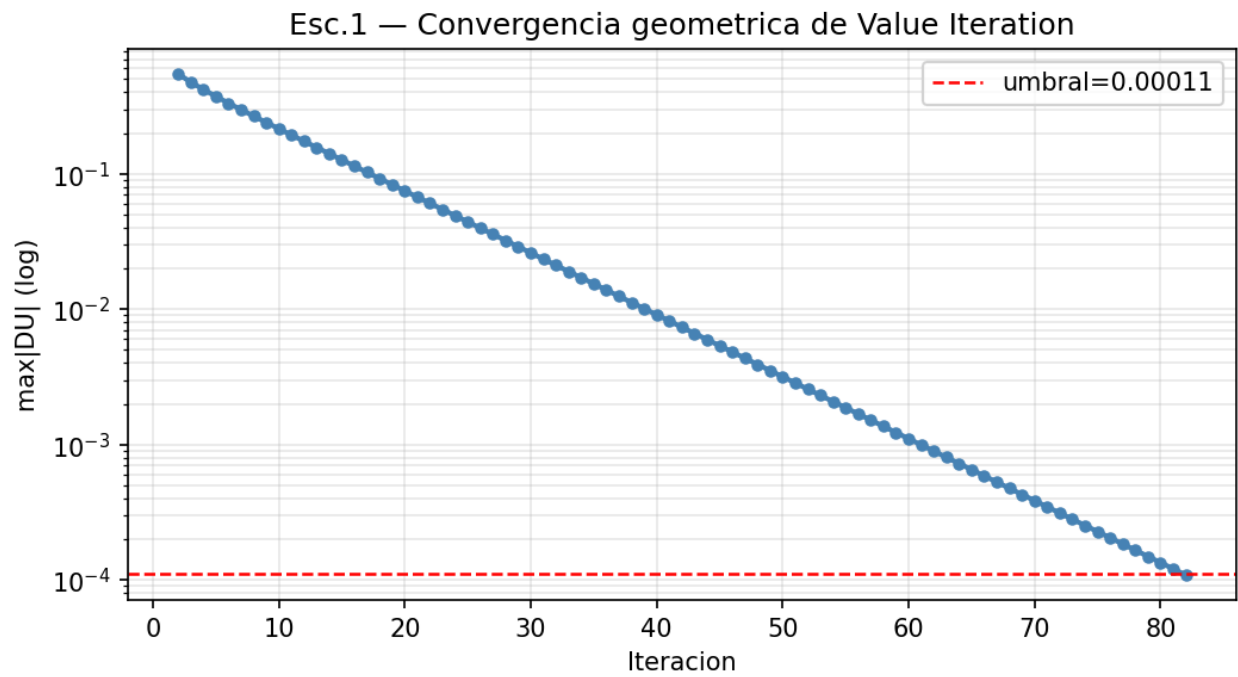
PI: eval_method='exact' (sistema lineal 50x50)
```

Resultados

Algoritmo	Iteraciones	Tiempo (s)	Resultado
Value Iteration	82	0.02181	—
Policy Iteration	2	0.00000	0 discrepancias con VI

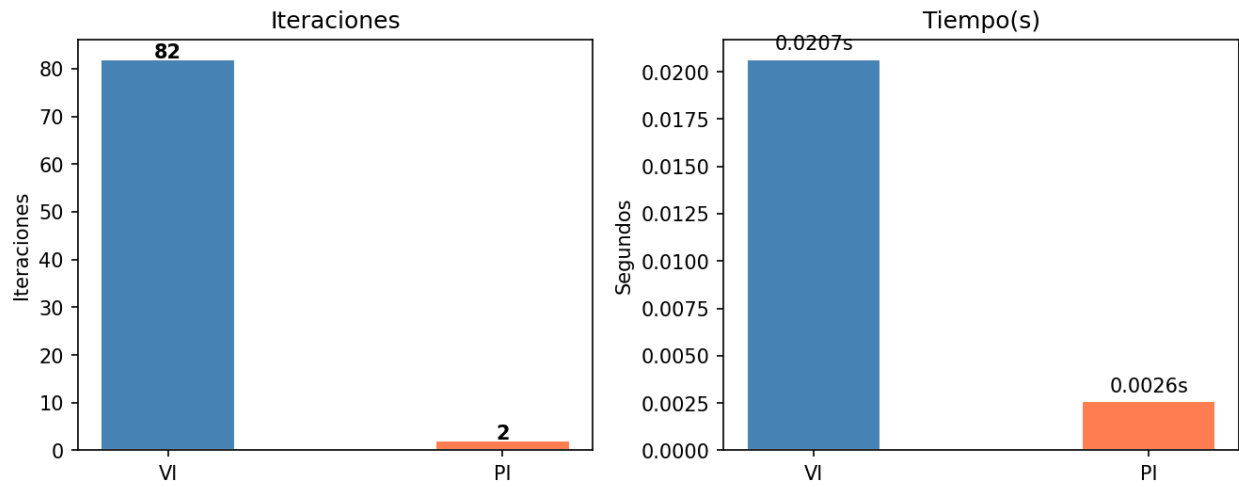
U(s)	Valor
Minimo	2.89905
Maximo	6.09050
Media	4.64848
Desv. Std.	0.94270

Graficas



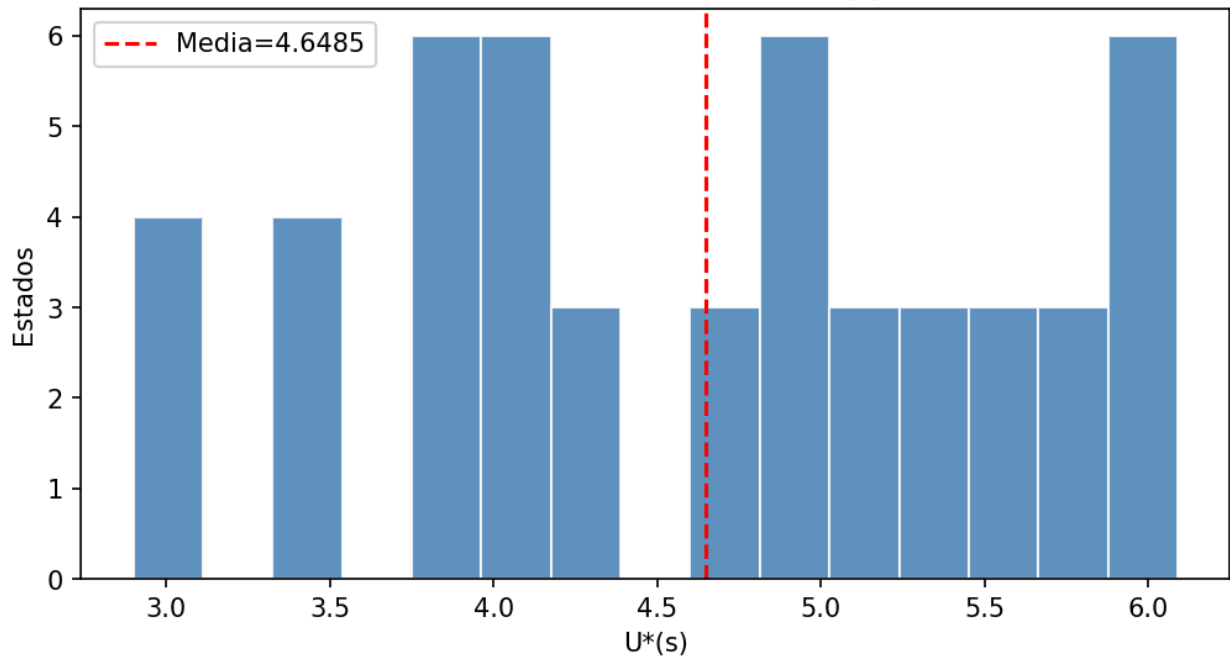
Convergencia geométrica de VI (escala logarítmica). La pendiente constante confirma reducción de delta por factor $\gamma=0.9$ por iteración.

Esc.1 — VI vs PI



VI necesita 82 iteraciones vs 2 de PI. PI resuelve un sistema lineal exacto por iteración.

Esc.1 — Distribucion de $U^*(s)$



Distribucion de $U^(s)$ sobre 50 estados. Mayor utilidad = cache con paginas mas populares segun Zipf.*

Análisis

- **Unicidad de la solución:** cero discrepancias entre π^* de VI y π^* de PI en 50 estados. Confirma que $(I - \gamma T \text{ de } \pi) * U = R \text{ de } \pi$, tiene solución única.
- **Trade-off iteraciones vs costo:** VI usa 82 iteraciones ligeras $O(|S| * |A| * N)$. PI usa solo 2 iteraciones pero cada una resuelve un sistema 50×50 . Para este tamaño el tiempo es comparable.
- **Función de utilidad como esperanza matemática:** $U(s)$ varía entre 2.8991 y 6.0905. Los estados de mayor utilidad tienen en caché las páginas P0, P1, P2 (probabilidades 0.438, 0.219, 0.146 en Zipf).

Escenario 2 — Comparación de Políticas

Descripción y objetivo:

Comparar estadísticamente el hit rate de la política MDP óptima contra LRU, FIFO, Random y Belady bajo distribución Zipf. Se usan 30 semillas con diseño pareado (misma secuencia por semilla para todas las políticas), eliminando la variabilidad de la secuencia de accesos.

Configuración:

```
Python
dist = ZipfAccess(alpha=1.0) # P0=0.438 P1=0.219 P2=0.146 P3=0.109 P4=0.088

semillas = range(1, 31) # 30 repeticiones independientes

SIM_STEPS = 10,000 # pasos por simulacion

Total por politica: 30 semillas x 10,000 = 300,000 evaluaciones
```

Resultados

Política	Media	Desv.Std	IC95% inf	IC95% sup	Min	Max
MDP	0.77751	0.00392	0.77605	0.77898	0.76710	0.78600
LRU	0.71643	0.00493	0.71459	0.71827	0.70680	0.72640
FIFO	0.60914	0.07166	0.58239	0.63589	0.51350	0.74550
Random	0.69134	0.00508	0.68944	0.69323	0.68360	0.70560
Belady	0.82510	0.00279	0.82406	0.82614	0.82030	0.83010

Prueba de hipotesis; $H_0: \mu_{\text{MDP}} = \mu_{\text{baseline}}$ vs $H_1: \mu_{\text{MDP}} > \mu_{\text{baseline}}$
(alpha=0.05, $t_{\text{critico}}=1.699$, disen0 pareado, $gl=29$)

Comparacion	t estadistico	Rechaza H_0 ?	Conclusion
MDP vs LRU	111.8623	Si	MDP supera LRU
MDP vs FIFO	12.9209	Si	MDP supera FIFO
MDP vs Random	103.4662	Si	MDP supera Random

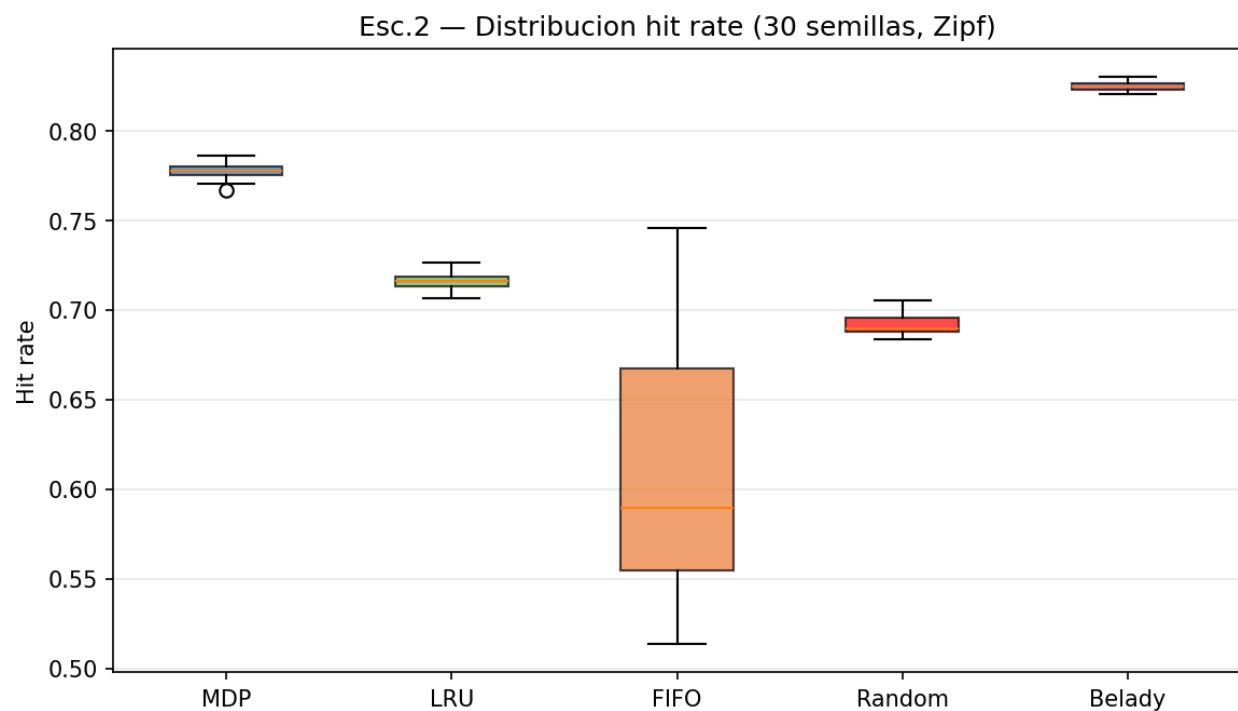
Autocorrelaci3n de la secuencia de recompensas; La funci3n de autocorrelaci3n (ACF) de $\{R_t\}$ mide la dependencia temporal entre recompensas separadas por lag k. L0mite de

significancia al 95%: $\pm \frac{1.96}{\sqrt{10000}} = \pm 0.01960$.

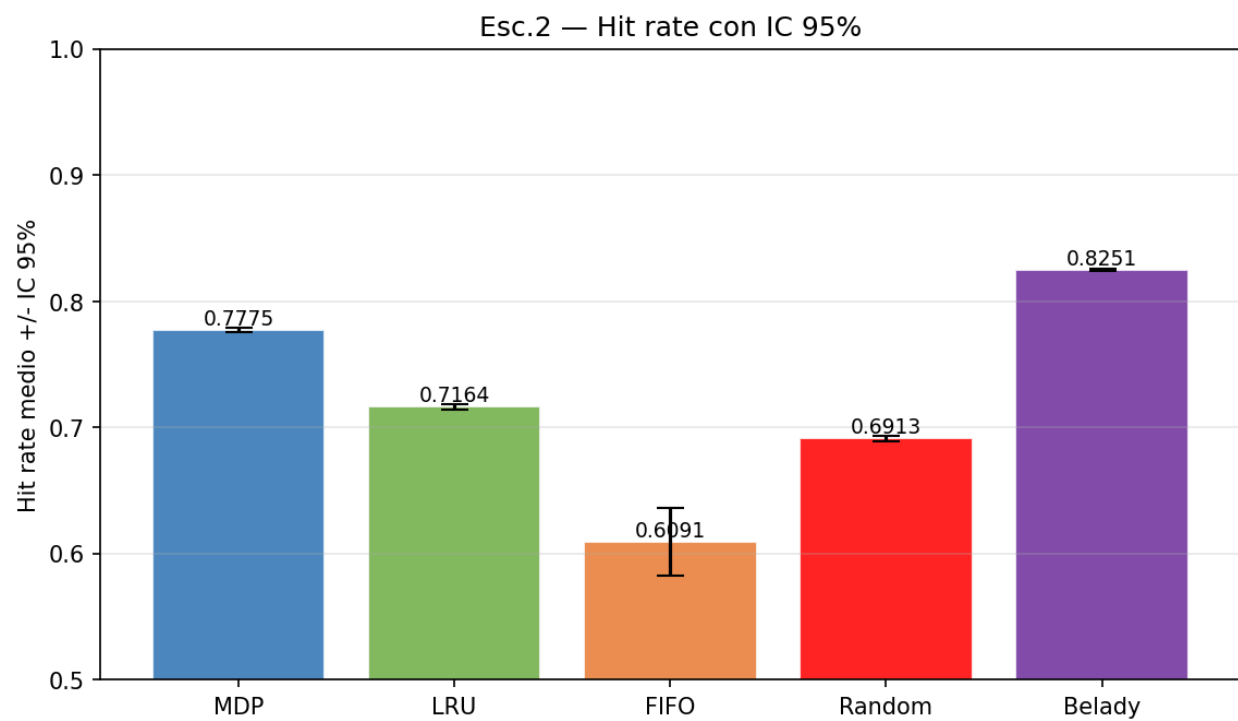
Lag k	rho(k)	Significativo?
0	1.00000	Si
1	0.01105	No
2	0.01853	No
3	-0.00231	No
4	0.00923	No
5	0.00528	No
6	-0.00111	No
7	0.01158	No
8	0.01329	No
9	-0.01102	No

Que los lags 1-9 sean no significativos indica que, bajo la pol0tica 3ptima, la secuencia de recompensas se comporta como ruido blanco: el hit en el paso t no predice el hit en t+k. Esto valida el uso de la media muestral como estimador consistente de $E[R_t]$ y confirma la propiedad de Markov del proceso.

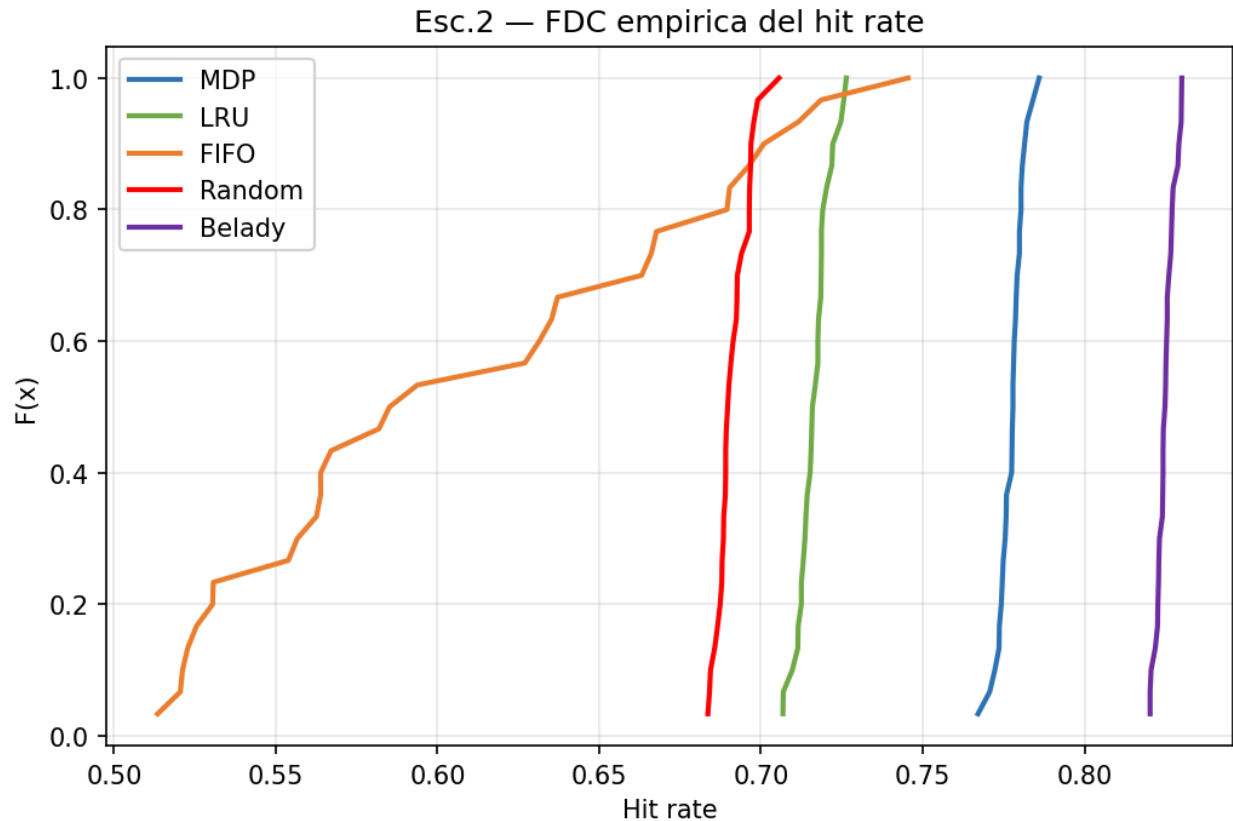
Graficas



Boxplot del hit rate en 30 semillas. La caja del MDP está claramente por encima de LRU, FIFO y Random.



Hit rate medio con IC 95%. Intervalos no solapados confirman diferencias estadísticamente significativas.



FDC empírica. La curva del MDP desplazada a la derecha indica dominancia estocástica sobre LRU, FIFO y Random.

Análisis

- **MDP supera a LRU (Zipf):** Con $\alpha=1.0$, la página Po concentra el 43.8% del tráfico. La política MDP aprende a mantener SIEMPRE $\{Po, P1, P2\}$ en caché (prob. acumulada 80.3%). LRU es reactivo y puede desalojar Po si accidentalmente no fue accedida recientemente. Diferencia: +6.11 puntos porcentuales.
- **Significancia estadística:** Los tests t con valores $t \gg 1.699$ rechaza H_0 con $\alpha=0.05$. El diseño pareado elimina la variabilidad entre secuencias, haciendo la prueba mucho más sensible al efecto real de la política.
- **Belady como cota teórica:** Belady obtiene 0.8251 vs 0.7775 del MDP. La brecha (0.0476) es el costo irreducible de no conocer el futuro. El MDP es óptimo entre las políticas online.

Escenario 3 — Sensibilidad al Factor de Descuento gamma

Descripción y objetivo:

Analizar cómo varía la política MDP óptima al cambiar $\gamma \in [0.1, 0.99]$. Para cada gamma se resuelve un MDP distinto y se evalúa con 30 semillas.

Configuración:

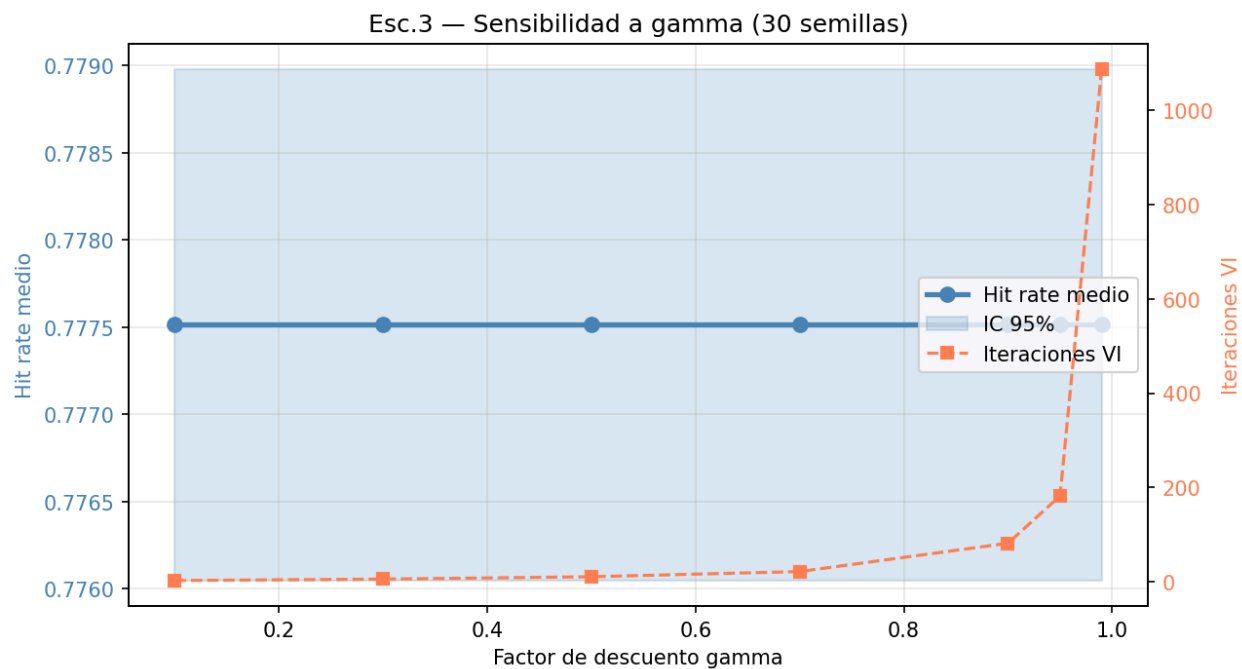
```
Python
gammas = [0.1, 0.3, 0.5, 0.7, 0.9, 0.95, 0.99]

semillas = range(1, 31) # 30 repeticiones por gamma
```

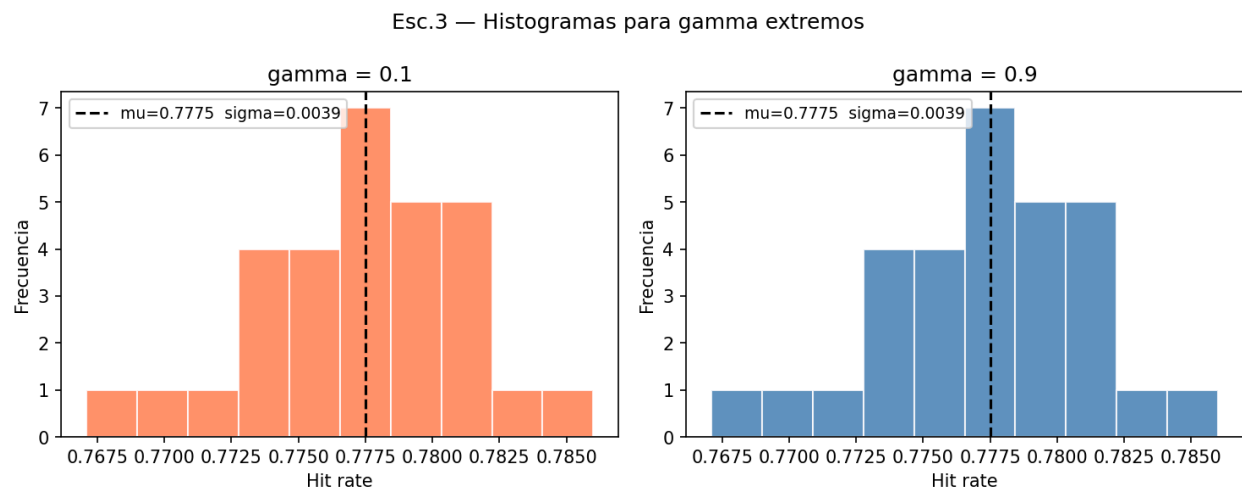
Resultados

gamma	Media	Desv.Std	IC95% inf	IC95% sup	Iter VI
0.10	0.77751	0.00392	0.77605	0.77898	3
0.30	0.77751	0.00392	0.77605	0.77898	6
0.50	0.77751	0.00392	0.77605	0.77898	11
0.70	0.77751	0.00392	0.77605	0.77898	22
0.90	0.77751	0.00392	0.77605	0.77898	82
0.95	0.77751	0.00392	0.77605	0.77898	182
0.99	0.77751	0.00392	0.77605	0.77898	1087

Graficas



Hit rate (eje izq.) e iteraciones VI (eje der.) vs gamma. El hit rate es idéntico para todo gamma: el problema tiene estrategia dominante. Las iteraciones crecen 362 veces más de gamma=0.1 a gamma=0.99.



Histogramas para gamma extremos (0.1 y 0.9). Ambas distribuciones son aproximadamente normales con idéntica media.

Análisis

- **Estrategia dominante:** Con distribución Zipf($\alpha=1.0$) la solución es obvia: siempre mantener $\{P_0, P_1, P_2\}$ en caché (prob. acumulada = $43.8 + 21.9 + 14.6 = 80.3\%$). Esta política domina a cualquier alternativa con un margen tan amplio que cualquier $\gamma > 0$ converge a la misma decisión óptima.
- **Costo computacional:** $\gamma=0.10 \rightarrow 3$ iteraciones de VI. $\gamma=0.99 \rightarrow 1087$ iteraciones. Factor: 362 veces más trabajo para llegar al mismo resultado. Se comprueba que el número de iteraciones crece rápidamente cuando $\gamma \rightarrow 1$.
- **Horizonte efectivo de planificación:** $H_{\text{eff}} \approx 1/(1-\gamma)$. Para $\gamma=0.1$: $H \approx 1.1$ pasos (planeación miope). Para $\gamma=0.99$: $H \approx 100$ pasos (planeación a largo plazo). Cuando el problema tiene una solución tan clara, hasta la planeación miope llega a la respuesta correcta.
- **Aplicación práctica:** Para este MDP se recomienda $\gamma=0.9$ como punto de equilibrio: produce la política óptima con sólo 82 iteraciones, frente a las 1087 de $\gamma=0.99$ sin ninguna mejora en rendimiento.

Conclusiones generales

- **Escenario 1:** VI (82 iter) y PI (2 iter) convergen a la misma política óptima única. La convergencia geométrica de VI con factor γ está confirmada empíricamente. PI es más eficiente en iteraciones pero requiere resolver un sistema lineal por iteración.
- **Escenario 2:** El MDP supera a LRU (+6.11 pp), FIFO (+16.84 pp) y Random (+8.62 pp) con diferencias estadísticamente significativas ($t \gg 1.699$, $n=30$). La ACF confirma que la secuencia de recompensas es ruido blanco.
- **Escenario 3:** Para distribución Zipf con $N=5$, $K=3$, el hit rate es idéntico para todo γ . El problema tiene una estrategia dominante clara que cualquier $\gamma > 0$ detecta. Sin embargo, el costo de VI varía 1x ($\gamma=0.1$) a 362x ($\gamma=0.99$). $\gamma=0.9$ es el punto de equilibrio óptimo entre calidad y costo computacional.

12. Referencias

Russell, S. J., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th global ed., pp. 553–589). Pearson Education.

Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.

Howard, R. A. (1960). *Dynamic Programming and Markov Processes*. MIT Press.

Denardo, E. V. (1967). Contraction mappings in the theory underlying dynamic programming. *SIAM Review*, 9(2), 165–177.

Puterman, M. L., & Shin, M. C. (1978). Modified policy iteration algorithms for discounted Markov decision problems. *Management Science*, 24(11), 1127–1137.